

ARGUS

Automated Reporting Governance & Unified Surveillance Platform \[4pt]
{Dokumentacja Architektoniczna i Projektowa (Software Design
Document)}

Platforma BI — Enterprise Quality Assurance

Wersja dokumentacji 1.0 · 18 lipca 2026

Contents

0. Wprowadzenie do repozytorium	6
ARGUS — Automated Reporting Governance & Unified Surveillance Platform . .	6
1. O projekcie	6
2. Struktura repozytorium	6
3. Jak korzystać z tego repozytorium	7
4. Kluczowe decyzje architektoniczne (skrót)	7
5. Stack technologiczny (skrót)	8
6. Status dokumentu	8
00. Technical Feasibility Study	9
Analiza techniczna metod integracji z platformą BI oraz metod ekstrakcji danych z raportów	9
1. Cel dokumentu	9
2. Metodologia oceny	9
3. Część I — Metody integracji z platformą raportową (dostęp do raportu, sprawdzenie dostępności i renderowania)	10
3.1 Browser Automation (Playwright)	10
3.2 Selenium (alternatywa dla Playwright)	11
3.3 Power BI REST API	11
3.4 XMLA Endpoint (Power BI Premium/Premium-per-User)	12
3.5 SSRS Web Services (SOAP) i SSRS REST API (Report Server 2017+)	13
3.6 Eksport raportów (Export-to-File: PDF / Excel / CSV / XML)	14
3.7 Analiza statyczna HTML/DOM (bez pełnego silnika przeglądarki, np. requests + BeautifulSoup)	14
3.8 Inne rozważane podejścia	15
3.9 Tabela porównawcza — integracja z platformą raportową	15
4. Rekomendowana architektura integracji	16
Uzasadnienie wyboru Browser Automation jako warstwy bazowej	16
Rola API i eksportów jako “drugiej linii”	16
5. Strategia odporności na zmiany raportów (Resilience Strategy)	17
6. Część II — Value Extraction Engine: metody odczytu wartości KPI z raportu	17
6.1 Analiza DOM (po wyrenderowaniu przez przeglądarkę)	17
6.2 Wykorzystanie natywnego API / zapytań do modelu danych (XMLA/DAX, SSRS SOAP dataset)	17
6.3 Eksport danych (CSV/XML/Excel) i parsowanie strukturalne	18
6.4 Analiza HTML (statyczny parser, bez wykonania JS)	18
6.5 Browser Automation jako mechanizm ekstrakcji (odczyt po renderowaniu — accessibility tree, innerText, atrybuty)	18
6.6 Computer Vision (detekcja układu, template matching, analiza wykresu)	18
6.7 OCR (Optical Character Recognition)	19
6.8 Tabela porównawcza — Value Extraction Engine	19
7. Rekomendowana architektura Value Extraction Engine — strategia wielowarstwowego fallbacku	20
8. Otwarte pytania do wyjaśnienia w Fazie 0 (Discovery / PoC)	21
9. Podsumowanie rekomendacji	21
01. Project Overview	22

1. Nazwa projektu i uzasadnienie	22
2. Wizja produktu	22
3. Cele projektu	23
3.1 Cele biznesowe	23
3.2 Cele techniczne	23
4. Zakres projektu	23
4.1 W zakresie (In Scope)	23
4.2 Poza zakresem (Out of Scope) — w bieżącej fazie	24
5. Interesariusze (Stakeholders)	24
6. Kluczowe założenia projektowe	24
7. Ograniczenia projektu	25
8. Słownik pojęć (Glossary)	25
9. Kryteria sukcesu projektu	25
02. Business Problem	27
1. Kontekst biznesowy	27
2. Obecny proces (AS-IS)	27
2.1 Charakterystyka procesu	27
3. Konsekwencje biznesowe obecnego stanu	28
3.1 Ryzyko operacyjne i decyzyjne	28
3.2 Koszt operacyjny	28
3.3 Brak wiedzy organizacyjnej o jakości danych w czasie	28
4. Analiza przyczyn źródłowych (Root Cause)	28
5. Stan docelowy (TO-BE)	29
6. Uzasadnienie inwestycji (Business Case — jakościowe)	29
7. Powiązanie z dalszą dokumentacją	29
03. Architecture	30
1. Zasady architektoniczne (Architectural Principles)	30
1.1 SOLID	30
1.2 Clean Architecture	30
1.3 Separation of Concerns	30
1.4 Dependency Injection	31
1.5 Open/Closed jako zasada nadrzędna dla rozszerzalności platformowej	31
2. Warstwy architektury (Clean Architecture)	31
3. Kluczowe wzorce projektowe	31
3.1 Connector Pattern (Report Platform Connector Layer)	31
3.2 Strategy + Chain of Responsibility (Value Extraction Engine)	32
3.3 Rule Engine Pattern (Business Rule Engine)	32
3.4 Dependency Injection / Composition Root	32
3.5 Repository Pattern	32
3.6 Odporność i resilience (Circuit Breaker, Retry, Timeout)	32
4. Główne moduły systemu i ich odpowiedzialności	32
4.1 Diagram zależności modułów (skrót koncepcyjny)	35
5. Zagadnienia przekrojowe (Cross-Cutting Concerns)	36
5.1 Bezpieczeństwo	36
5.2 Konfiguracja	36
5.3 Logowanie i audyt	36
5.4 Obserwowalność (Observability)	36
6. Rozszerzalność — scenariusz “nowa platforma raportowa”	36
7. Powiązanie z dalszą dokumentacją	37
04. Functional Requirements	38
1. Aktorzy systemu	38
2. Lista przypadków użycia	38
3. Wymagania funkcjonalne	39
3.1 Katalog raportów i nawigacja (powiązane UC-01, UC-02)	39
3.2 Definiowanie testów i Validation Profile (powiązane UC-03-UC-07)	39
3.3 Wykonanie testów (powiązane UC-08, UC-09)	40
3.4 Wyniki, historia i metryki (powiązane UC-10-UC-14)	41

3.5 AI i wsparcie decyzyjne (powiązane UC-15)	41
3.6 Powiadomienia (powiązane UC-16)	42
3.7 Administracja, bezpieczeństwo i audyt (powiązane UC-17-UC-20)	42
4. Wymagania GUI — opis ekranów	42
4.1 Ekran główny — Dashboard	43
4.2 Katalog Raportów (Report Catalog)	43
4.3 Konfiguracja Validation Profile	43
4.4 Uruchamianie testów	43
4.5 Historia i wyniki	43
4.6 Analiza AI (AI Insights)	43
4.7 Panel administracyjny	43
5. Powiązanie z dalszą dokumentacją	44
05. Non-Functional Requirements	45
1. Wydajność (Performance)	45
2. Skalowalność (Scalability)	45
3. Bezpieczeństwo (Security)	46
4. Dostępność i niezawodność (Availability & Reliability)	47
5. Utrzymywalność (Maintainability)	47
6. Obserwowalność (Observability)	47
7. Zgodność i audytowalność (Compliance)	48
8. Użyteczność (Usability)	48
9. Przenośność (Portability)	48
10. Podsumowanie — macierz wymagań krytycznych	48
06. AI Module	50
1. Zasada nadrzędna	50
2. Rola AI — co AI może robić	50
3. Czego AI nie może robić	51
4. Wymóg wyjaśnialności (Explainability)	51
5. Architektura modułu AI	51
5.1 Umiejscowienie w architekturze	51
5.2 Komponenty wewnętrzne AI Analysis Engine	52
5.3 Wybór dostawcy modelu	52
6. Dane wykorzystywane przez AI i ochrona danych wrażliwych	52
7. Ryzyka związane z AI i mitygacje	53
8. Powiązanie z dalszą dokumentacją	53
07. Data Model	54
1. Cel dokumentu	54
2. Słownik encji	54
2.1 BusinessArea (Obszar biznesowy / zakładka)	54
2.2 Report (Raport)	54
2.3 ValidationProfile	55
2.4 TestDefinition	55
2.5 SqlValidationQuery	55
2.6 BusinessRule	55
2.7 TestExecution (Przebieg testowy)	55
2.8 TestResult (Wynik pojedynczego testu)	56
2.9 HealthScoreSnapshot	56
2.10 AIInsight	56
2.11 Notification	57
2.12 User / Role	57
2.13 AuditLogEntry	57
3. Diagram relacji (skrót)	57
4. Strategia retencji i partycjonowania	58
5. Powiązanie z dalszą dokumentacją	58
08. API	59
1. Zasady projektowe API	59

2. Uwierzytelnianie i autoryzacja	59
3. Zasoby główne i endpointy	60
3.1 Reports — katalog raportów	60
3.2 Validation Profiles	60
3.3 Test Definitions (w ramach profilu)	60
3.4 Test Executions — wykonania testów	60
3.5 Health & Confidence Scores	61
3.6 AI Insights	61
3.7 Notifications	61
3.8 Administracja	61
4. Przykładowy kontrakt — wynik przebiegu testowego	62
5. Obsługa błędów	62
6. Webhooks / integracje wychodzące (rozszerzenie przyszłościowe)	63
7. Powiązanie z dalszą dokumentacją	63
09. Python Architecture	64
1. Stack technologiczny — pełna lista i uzasadnienie	64
2. Struktura katalogów (Clean Architecture / Ports & Adapters)	67
3. Dependency Injection — Composition Root	68
4. Model wykonawczy i współbieżność	68
4.1 Dwuwarstwowy model harmonogramu i wykonania	68
4.2 Asynchroniczność w kodzie	69
5. Wymuszanie granic architektonicznych w CI	69
6. Zarządzanie kosztem wywołań AI	69
7. Powiązanie z dalszą dokumentacją	70
10. Deployment	71
1. Środowiska	71
2. Topologia infrastruktury (on-prem, zgodnie z założeniem projektowym)	71
2.1 Uzasadnienie kluczowych decyzji topologicznych	72
3. CI/CD — pipeline (GitHub Actions)	72
4. Zarządzanie sekretami i kontem technicznym	73
5. Skalowanie	73
6. Disaster Recovery i kopie zapasowe	74
7. Integracja z monitoringiem organizacyjnym	74
8. Powiązanie z dalszą dokumentacją	75
11. Roadmap	76
1. Podejście do wdrożenia	76
2. Faza 0 — Discovery & Proof of Concept	76
3. Faza 1 — MVP (Minimum Viable Product)	77
4. Faza 2 — Scale-out i pełna funkcjonalność testowa	77
5. Faza 3 — Moduł AI	77
6. Faza 4 — Pełne wdrożenie produkcyjne (skala 1000 raportów)	78
7. Faza 5 — Ciągłe doskonalenie (Continuous Improvement)	78
8. Zależności między fazami (skrót)	79
9. Powiązanie z dalszą dokumentacją	79
12. Risks	80
1. Metodologia	80
2. Rejestr ryzyk	80
3. Macierz Prawdopodobieństwo × Wpływ (skrót wizualny)	84
4. Powiązanie z dalszą dokumentacją	84
13. Diagramy UML	85
13.1 Diagram przypadków użycia (Use Case Diagram)	85
13.2 Diagram klas (Class Diagram)	90
13.3 Diagram aktywności (Activity Diagram)	95
13.4 Diagram sekwencji (Sequence Diagram)	100
13.5 Diagram komponentów (Component Diagram)	104

13.6 Diagram wdrożenia (Deployment Diagram)	108
13.7 Diagram encji (Entity Relationship Diagram — ERD)	112

Klasa systemu	Enterprise / Mission-critical
Domena	Bankowość — Platforma BI (Power BI, SQL Server Reporting Services)
Typ dokumentu	Dokumentacja architektoniczna i projektowa (Software Design Document)
Status	Do akceptacji przez Menadżera oraz Dział IT
Wersja dokumentacji	1.0
Data	2026-07-18
Zawartość	Repozytorium BI-Quality-Assurance-Platform/ — README, 13 dokumentów architektonicznych (docs/) oraz 7 diagramów UML w PlantUML (uml/)

0. Wprowadzenie do repozytorium

ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Klasa systemu: Enterprise / Mission-critical

Domena: Bankowość — Platforma BI (Power BI, SQL Server Reporting Services)

Typ dokumentu: Repozytorium dokumentacji architektonicznej i projektowej

Status: Do akceptacji przez Menadżera oraz Dział IT

Wersja dokumentacji: 1.0

Data: 2026-07-18

1. O projekcie

W organizacji funkcjonuje wewnętrzna Platforma BI udostępniająca ok. 300 raportów (docelowo ≥ 1000), zbudowanych w technologii **Microsoft Power BI** oraz **SQL Server Reporting Services (SSRS)**. Obecnie poprawność raportów jest weryfikowana ręcznie przez analityków, co jest procesem czasochłonnym, podatnym na błędy i niemożliwym do skalowania wraz ze wzrostem liczby raportów.

ARGUS to platforma klasy enterprise, której zadaniem jest **autonomiczne, ciągłe i audytowalne monitorowanie jakości raportów BI** — dostępności, poprawności renderowania, zgodności wartości KPI ze źródłem danych (hurtownią) oraz reguł biznesowych — wspierana (ale nie zastępowana) przez moduł AI, który interpretuje wyniki, wykrywa anomalie i tłumaczy je zrozumiałym językiem dla użytkownika biznesowego.

2. Struktura repozytorium

BI-Quality-Assurance-Platform/

—	README.md ← ten dokument
—	docs/ ← dokumentacja architektoniczna (Markdown)
—	00_Technical_Feasibility_Study.md ← analiza techniczna metod integracji i ekstrakcji danych
—	01_Project_Overview.md ← wizja, cele, zakres, interesariusze
—	02_Business_Problem.md ← problem biznesowy i uzasadnienie inwestycji
—	03_Architecture.md ← architektura systemu (Clean Architecture, moduły)
—	04_Functional_Requirements.md ← wymagania funkcjonalne, przypadki użycia, GUI
—	05_NonFunctional_Requirements.md ← wydajność, bezpieczeństwo, skalowalność, SLA
—	06_AI_Module.md ← rola i granice modułu AI
—	07_Data_Model.md ← model danych i słownik encji
—	08_API.md ← projekt API REST
—	09_Python_Architecture.md ← architektura kodu Python, stack technologiczny

10_Deployment.md	← środowiska, CI/CD, infrastruktura, DR
11_Roadmap.md	← plan wdrożenia w fazach
12_Risks.md	← rejestr ryzyk
uml/	← diagramy PlantUML (źródła .puml)
UseCase.puml	
ClassDiagram.puml	
ActivityDiagram.puml	
SequenceDiagram.puml	
ComponentDiagram.puml	
DeploymentDiagram.puml	
ERD.puml	
ARGUS-Documentation-Complete.pdf	← skompilowana dokumentacja (wszystkie rozdziały + diagramy)

3. Jak korzystać z tego repozytorium

Rola	Od czego zacząć
Sponsor / Project Manager	01_Project_Overview.md → 02_Business_Problem.md → 11_Roadmap.md → 12_Risks.md
Architekt IT / Security	00_Technical_Feasibility_Study.md → 03_Architecture.md → 05_NonFunctional_Requirements.md → 10_Deployment.md
Zespół deweloperski	03_Architecture.md → 07_Data_Model.md → 08_API.md → 09_Python_Architecture.md → uml/
Analitik biznesowy / właściciel raportów	04_Functional_Requirements.md → 06_AI_Module.md

Diagramy w katalogu uml/ są renderowalne dowolnym silnikiem zgodnym z PlantUML (rozszerzenie VS Code, plantuml.jar, CI). W dokumencie PDF diagramy są już wyrenderowane i opatrzone opisem oraz uzasadnieniem projektowym.

4. Kluczowe decyzje architektoniczne (skrót)

1. **Warstwa integracji jako zestaw wymiennych Connectorów** (IReportConnector) — dodanie nowej platformy raportowej (np. Tableau, Qlik) w przyszłości wymaga wyłącznie implementacji nowego connectora, bez zmian w rdzeniu systemu (Open/Closed Principle).
2. **Browser Automation (Playwright) jako uniwersalna warstwa integracyjna “najniższego wspólnego mianownika”**, uzupełniona o natywne API (Power BI REST API / XMLA, SSRS Web Services) tam, gdzie są dostępne i opłacalne — patrz uzasadnienie w 00_Technical_Feasibility_Study.md.
3. **Value Extraction Engine oparty na strategii wielowarstwowego fallbacku** (DOM/HTML → API/Export → Computer Vision/OCR), aby system był odporny na drobne zmiany wizualne raportów.
4. **AI jako warstwa interpretacyjna, nigdy decyzyjna** — o poprawności raportu decyduje wyłącznie deterministyczny silnik walidacji.
5. **Clean Architecture + Dependency Injection** — separacja domeny od infrastruktury, testowalność, łatwość utrzymania w horyzoncie wieloletnim.

5. Stack technologiczny (skrót)

Pełne uzasadnienie każdego wyboru — patrz 09_Python_Architecture.md. * Python 3.13+, * FastAPI, * Playwright, * SQLAlchemy 2.x + Alembic, * PostgreSQL, * Redis, * Celery + Redis/RabbitMQ (orkiestracja harmonogramu i workerów), * Pydantic v2, * LangChain/Semantic Kernel (wyłącznie moduł AI) + Azure OpenAI, * React + TypeScript (GUI), * Docker/Docker Compose + Kubernetes (docelowo), * pytest, * Ruff, * MyPy, * GitHub Actions, * Prometheus, * Grafana, * OpenTelemetry.

6. Status dokumentu

Niniejsza dokumentacja stanowi **Software Design Document** w rozumieniu procesu SDLC organizacji i jest podstawą do: - oceny przez Architekturę IT i Bezpieczeństwo, - estymacji przez zespół deweloperski, - rozpoczęcia fazy Proof of Concept (patrz 11_Roadmap.md, Faza 0).

Dokument nie zawiera kodu implementacyjnego — jest to dokumentacja projektowa poprzedzająca development.

00. Technical Feasibility Study

Analiza techniczna metod integracji z platformą BI oraz metod ekstrakcji danych z raportów

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Studium wykonalności technicznej

Odbiorcy: Architektura IT, Zespół Bezpieczeństwa, Zespół Deweloperski

Status: Do akceptacji

1. Cel dokumentu

Dokument odpowiada na dwa fundamentalne pytania techniczne, od których zależy cała dalsza architektura systemu:

1. **W jaki sposób ARGUS ma technicznie “dotrzeć” do raportu** opublikowanego na Platformie BI (Power BI oraz SSRS/Report Server), tak aby móc stwierdzić, czy raport się otwiera i renderuje poprawnie?
2. **W jaki sposób ARGUS ma odczytać konkretną wartość liczbową (KPI)** z wyrenderowanego raportu, aby porównać ją z wynikiem zapytania SQL do hurtowni danych?

Odpowiedzi na te pytania nie są oczywiste, ponieważ Power BI i SSRS to dwie różne technologie, o różnym stopniu otwartości API, różnym modelu autoryzacji i różnej charakterystyce renderowania (Power BI — SPA renderowany po stronie klienta w iframe/JS; SSRS — częściowo server-side HTML/ReportViewer). Dokument analizuje wszystkie realistyczne podejścia integracyjne, ocenia je względem jednolitego zestawu kryteriów i rekomenduje architekturę docelową.

2. Metodologia oceny

Każde podejście oceniane jest według siedmiu kryteriów, spójnych z wymaganiami z briefu projektowego:

Kryterium	Pytanie oceniające
Wymagania	Czego technicznie/organizacyjnie potrzeba, aby wdrożyć podejście?
Zalety	Co zyskujemy?
Ograniczenia	Czego podejście nie potrafi lub gdzie zawodzi?
Wpływ na wydajność	Jaki jest koszt czasowy/zasobowy per raport?
Odporność na zmiany raportów	Czy drobna zmiana layoutu raportu psuje integrację?
Zastosowanie w środowisku bankowym	Czy podejście jest akceptowalne pod kątem bezpieczeństwa, sieci, compliance, licencji?
Złożoność implementacji / utrzymania	Ile kosztuje zbudowanie i utrzymanie w perspektywie 1000 raportów?

Ocena końcowa każdego podejścia wyrażona jest w skali: * **[W]** Wysoka przydatność * **[O]** Przydatność ograniczona * **[N]** Niska przydatność * **[-]** Nie dotyczy (n/d) dla danej platformy.

3. Część I — Metody integracji z platformą raportową (dostęp do raportu, sprawdzenie dostępności i renderowania)

3.1 Browser Automation (Playwright)

Opis: Sterowanie prawdziwą przeglądarką (Chromium/WebKit/Firefox) w trybie headless lub headed, nawigacja do adresu URL raportu, oczekiwanie na zdarzenia sieciowe/DOM sygnalizujące zakończenie renderowania, przechwytywanie stanu strony (DOM, zrzut ekranu, żądania sieciowe).

Kryterium	Ocena
Wymagania	Środowisko z zainstalowanymi silnikami przeglądarek, konto techniczne z dostępem SSO (Windows Integrated Authentication / Kerberos), dostęp sieciowy do pbi.bank.com.pl i bgkssrs, ewentualnie profil systemu Windows/Linux z obsługą NTLM/Kerberos (SPNEGO).
Zalety	Działa identycznie jak realny użytkownik — jest niezależne od tego, czy dany raport ma API. Obsługuje SSO tak samo jak przeglądarka pracownika. Pozwala zweryfikować pełny łańcuch: sieć → autoryzacja → renderowanie → interaktywność (np. slicery, filtry). Jedno podejście dla obu platform (PBI i SSRS) — ujednolica architekturę. Pozwala też wykonać zrzut ekranu jako dowód (evidence) w audycie.
Ograniczenia	Wyższe zużycie CPU/RAM niż podejścia API (pełny silnik przeglądarki na raport). Renderowanie Power BI jest asynchroniczne i oparte o WebSocket/JS — wymaga jawnych strategii oczekiwania (nie da się polegać wyłącznie na networkidle). Selektory DOM mogą się zmieniać wraz z aktualizacjami produktu Microsoft (mitygowalne, patrz sekcja 5).
Wpływ na wydajność	Otwarcie pojedynczego raportu Power BI: ok. 3-8 s (zależnie od złożoności wizualizacji i cache Power BI). SSRS: zwykle szybciej (1-4 s), bo renderowanie jest w większym stopniu server-side. Przy 1000 raportach wymaga równoległości (worker pool), a nie wykonania sekwencyjnego — patrz 05_NonFunctional_Requirements.md.
Odporność na zmiany	Średnia-wysoka, pod warunkiem stosowania odpornych strategii selekcji (role/aria-label, atrybuty semantyczne Power BI, a nie pozycja pikselowa czy klasy CSS generowane przez bundler).
Zastosowanie bankowe	Wysokie — Playwright jest open-source (Apache 2.0), nie wysyła telemetrii do zewnętrznych usług przy odpowiedniej konfiguracji, może działać w pełni offline w sieci wewnętrznej banku, wspiera autoryzację Windows (NTLM/Kerberos) natywnie przez kontekst przeglądarki uruchomiony na koncie domenowym / przekazanie tokenu.
Złożoność / utrzymanie	Średnia. Wymaga zespołu, który rozumie zarówno automatyzację przeglądarki, jak i specyfikę renderowania Power BI/SSRS. Utrzymanie selektorów jest powtarzalnym kosztem, ale mitygowalnym przez centralizację (Page Object Pattern) i strategię wielopoziomowych selektorów.
Ocena łączna	[W] Wysoka — jedyne podejście uniwersalne dla obu platform

3.2 Selenium (alternatywa dla Playwright)

Opis: Klasyczny framework do automatyzacji przeglądarki, sterujący przez protokół WebDriver.

Kryterium	Ocena
Wymagania	Analogiczne do Playwright — silniki przeglądarek + sterowniki (ChromeDriver/GeckoDriver), zarządzane osobno od wersji przeglądarki.
Zalety	Bardzo dojrzały, ogromna baza wiedzy, wsparcie w praktycznie każdym języku (w tym Python — selenium). Szerokie wsparcie w narzędziach RPA korporacyjnych, co może ułatwić integrację z istniejącymi standardami banku, jeśli takie już funkcjonują.
Ograniczenia	Wolniejszy i mniej stabilny niż Playwright w scenariuszach z dynamicznym JS (Power BI = ciężki SPA). Brak wbudowanego auto-waitingu na poziomie akcji (częstsze flaky tests). Zarządzanie wersjami sterowników (*Driver binaries) to dodatkowy koszt operacyjny (mitygowalne przez Selenium Manager / Selenium Grid, ale nadal bardziej złożone niż w Playwright). Brak natywnego przechwytywania ruchu sieciowego (wymaga dodatkowych proxy typu BrowserMob).
Wpływ na wydajność	Porównywalny rząd wielkości do Playwright, ale zazwyczaj 20–40% wolniejszy w testach z dużą liczbą interakcji z powodu mniej efektywnego protokołu komunikacji (WebDriver classic vs. CDP/BiDi używany przez Playwright).
Odporność na zmiany	Podobna do Playwright — zależy od jakości strategii selektorów, nie od samego narzędzia.
Zastosowanie bankowe	Wysokie, tak samo dopuszczalne licencyjnie (Apache 2.0) i sieciowo jak Playwright.
Złożoność / utrzymanie	Wyższa niż Playwright — więcej "ruchomych części" (menedżer sterowników, Selenium Grid do równoległości, mniej ergonomiczne API asynchroniczne).
Ocena łączna	[O] Przydatność ograniczona — Playwright dominuje Selenium w praktycznie każdym kryterium dla tego zastosowania (nowoczesny stack, natywna równoległość, natywne auto-waiting, natywne przechwytywanie sieci potrzebne do detekcji błędów renderowania)

3.3 Power BI REST API

Opis: Oficjalne REST API Microsoftu (api.powerbi.com w chmurze lub Power BI Report Server REST API on-prem) umożliwiające operacje na poziomie metadanych: listy raportów, workspace'y, odświeżanie datasetów, eksport raportu do pliku, sprawdzanie statusu odświeżenia.

Kryterium	Ocena
Wymagania	Rejestracja aplikacji w Azure AD / Entra ID (service principal) lub — w wariantcie Power BI Report Server (on-prem, częstszy w bankach) — API oparte o inny mechanizm autoryzacji (Windows Auth). Wymaga potwierdzenia, czy organizacja korzysta z Power BI Service (chmura) czy Power BI Report Server (on-prem) — adres pbi.bank.com.pl sugeruje wdrożenie on-prem / self-hosted , co ogranicza dostępność części funkcji REST API dostępnych wyłącznie w wariantcie chmurowym (np. część endpointów Admin API).

Kryterium	Ocena
Zalety	Brak potrzeby renderowania w przeglądarce dla operacji administracyjnych (np. sprawdzenie statusu ostatniego odświeżenia datasetu, listy raportów w workspace). Bardzo szybkie (odpowiedzi JSON, brak renderowania UI). Dobrze udokumentowane, stabilne kontraktowo (wersjonowane API).
Ograniczenia	REST API nie pozwala odczytać wartości z wizualizacji raportu (nie zwraca "co widzi użytkownik na wykresie") — to kluczowe ograniczenie dla wymogu odczytu KPI. Służy głównie do zarządzania (metadata, refresh, eksport). Dostępność i zakres endpointów różni się istotnie między Power BI Service a Power BI Report Server (on-prem) — wymaga wczesniej weryfikacji z zespołem administrującym platformą BI.
Wpływ na wydajność	Bardzo dobry — wywołania rzędu 100-300 ms.
Odporność na zmiany	Wysoka dla operacji, które obsługuje (kontrakt API), ale nie dotyczy odczytu KPI.
Zastosowanie bankowe	Zależne od modelu wdrożenia Power BI (cloud vs. on-prem) i polityk sieciowych dot. wywołań do <code>api.powerbi.com</code> (chmura Microsoft) — może wymagać dodatkowej zgody Security/Compliance na ruch wychodzący do chmury publicznej, co w bankowości bywa procesem długotrwałym.
Złożoność / utrzymanie	Niska dla zakresu, który obsługuje.
Ocena łączna	[O] Przydatność ograniczona — cenne wsparcie dla operacji administracyjnych i triggerowania odświeżeń, ale niewystarczające jako główny mechanizm odczytu dostępności i wartości KPI

3.4 XMLA Endpoint (Power BI Premium/Premium-per-User)

Opis: Endpoint analityczny (protokół XML for Analysis, jak w SSAS Tabular) pozwalający wykonywać zapytania **DAX/MDX bezpośrednio na modelu danych** leżącym pod raportem Power BI, z pominięciem warstwy wizualnej.

Kryterium	Ocena
Wymagania	Power BI Premium, Premium-per-User lub Fabric capacity (XMLA Endpoint nie jest dostępny w licencji Pro ani standardowo w Power BI Report Server on-prem w tej samej formie co w chmurze) — kluczowe pytanie licencyjne do wyjaśnienia z administratorem platformy BI na starcie projektu. Narzędzie klienckie obsługujące XMLA/DAX (np. biblioteka <code>Microsoft.AnalysisServices.AdomdClient</code> po stronie .NET; z poziomu Pythona — komunikacja przez <code>ADOMD.NET</code> via <code>pythonnet</code> lub pośredni serwis .NET).
Zalety	Najbardziej precyzyjna metoda odczytu wartości — zapytanie DAX zwraca dokładnie tę samą liczbę, którą silnik Power BI wyliczyłby dla wizualizacji, bez zależności od warstwy graficznej. Bardzo szybkie. Odporne na zmiany layoutu raportu (bo nie dotyka warstwy wizualnej), wrażliwe tylko na zmiany modelu danych.
Ograniczenia	Wymaga wiedzy o strukturze modelu (nazwy miar/tabel) dla każdego raportu — administrator definiujący Validation Profile musi znać definicję miary DAX, nie tylko "co widać na ekranie". Nie weryfikuje renderowania (czy wizualizacja faktycznie się wyświetla poprawnie użytkownikowi) — nie zastępuje testu dostępności/UI, jedynie uzupełnia test wartości. Zależność licencyjna (Premium) może wykluczać część raportów.
Wpływ na wydajność	Bardzo dobry (dziesiątki-setki ms na zapytanie).
Odporność na zmiany	Bardzo wysoka wobec zmian UI; niska wobec zmian modelu (przemianowanie miary łamie zapytanie — ale to zmiana rzadka i celowa, łatwa do zarządzania proceduralnie).

Kryterium	Ocena
Zastosowanie bankowe	Wysokie pod względem bezpieczeństwa (ruch wewnętrzny, ten sam mechanizm auth co reszta platformy), warunkowe pod względem licencji.
Złożoność / utrzymanie	Średnia-wysoka z Pythona (naturalny język docelowego projektu) ze względu na brak natywnej biblioteki Python dla XMLA/ADOMD — zwykle wymaga mostka .NET/ pythonnet lub osobnego mikroservisu.
Ocena łączna	[O] Przydatność ograniczona do podzbioru raportów (Premium) — rekomendowana jako opcjonalny, wysoko-precyzyjny connector “drugiej linii” dla raportów, dla których jest dostępna, nie jako rozwiązanie uniwersalne

3.5 SSRS Web Services (SOAP) i SSRS REST API (Report Server 2017+)

Opis: SSRS udostępnia klasyczne API SOAP (ReportService2010.asmx) do zarządzania katalogiem raportów oraz — od wersji 2017 — REST API (api/v2.0) do operacji na katalogu, a także URL Access (parametryzowany link renderujący raport w wybranym formacie: HTML, PDF, XML, CSV, Excel) poprzez ReportExecution2005 / /Reserve.aspx.

Kryterium	Ocena
Wymagania	Dostęp sieciowy do endpointów SOAP/REST Report Servera, autoryzacja Windows Integrated (NTLM/Kerberos) po stronie klienta HTTP — w Pythonie realizowalne (requests + requests-negotiate-sspi / requests-kerberos, lub zeep dla SOAP).
Zalety	Pozwala programowo wygenerować raport w formacie danych (np. XML lub CSV) zamiast HTML — co daje strukturalny, łatwy do sparsowania wynik bez potrzeby przeglądarki. Możliwość pobrania metadanych katalogu (lista raportów, parametry, harmonogramy) — przydatne dla automatycznej inwentaryzacji raportów w systemie ARGUS. Brak zależności od silnika przeglądarki dla tej ścieżki integracji.
Ograniczenia	URL Access renderuje cały raport na nowo w wybranym formacie eksportu — dla dużych raportów może być wolniejsze niż odczyt jednej wartości. Nie weryfikuje wizualnego renderowania w sensie “czy webowy interfejs użytkownika faktycznie poprawnie wyświetla stronę” (np. błędy CSS, błędy w kontrolce ReportViewer na stronie portalu, timeout w warstwie webowej niewidoczny z poziomu samego silnika raportowego) — jest to uzupełnienie, nie substytut testu end-to-end. SOAP API bywa postrzegane jako starzejące się (choć w środowiskach SSRS on-prem nadal standardowe i w pełni wspierane).
Wpływ na wydajność	Dobry dla metadanych (SOAP ListChildren itp. — setki ms); zmienny dla renderowania pełnego raportu w formacie eksportu (zależny od rozmiaru raportu, od setek ms do kilku-kilkunastu sekund dla dużych raportów tabelarycznych).
Odporność na zmiany	Wysoka dla warstwy API/katalogu; średnia dla parsowania wyeksportowanego formatu (zmiana struktury raportu = zmiana układu kolumn w eksporcie).
Zastosowanie bankowe	Wysokie — mechanizm natywny dla SSRS on-prem, używany od lat w środowiskach korporacyjnych, w pełni kompatybilny z Windows Integrated Authentication.
Złożoność / utrzymanie	Niska-średnia.
Ocena łączna	[W] Wysoka dla SSRS — rekomendowana jako natywny connector dla platformy SSRS, komplementarny do Browser Automation

3.6 Eksport raportów (Export-to-File: PDF / Excel / CSV / XML)

Opis: Zarówno Power BI, jak i SSRS oferują mechanizm eksportu raportu do pliku (poprzez UI, API lub URL Access). Eksport CSV/XML jest ustrukturyzowany i łatwy do sparsowania programowo; PDF/Excel wymaga dodatkowego parsowania.

Kryterium	Ocena
Wymagania	Dla SSRS — URL Access z parametrem <code>rs:Format</code> ; dla Power BI — Power BI REST API Export to File (asynchroniczne, wymaga odpytywania statusu zadania) lub eksport z poziomu UI przez Browser Automation.
Zalety	Format danych (CSV/XML) eliminuje potrzebę parsowania HTML/DOM — bardzo wysoka niezawodność odczytu wartości, gdy dostępny. Dobry “dowód” (artefakt) do celów audytu — plik można zarchiwizować.
Ograniczenia	Nie jest to test end-to-end doświadczenia użytkownika (nie wykrywa np. błędów renderowania konkretnej wizualizacji w warstwie webowej, błędów JS, timeoutu interfejsu). Eksport bywa wolniejszy niż odczyt pojedynczej wartości (generowany jest cały raport). Dla Power BI eksport przez REST API jest operacją asynchroniczną z limitami (rate limiting), co przy 1000 raportach wymaga starannego zarządzania kolejką.
Wpływ na wydajność	Średni-wysoki koszt czasowy per raport (generowanie pełnego dokumentu), ale wykonywalny w tle/asynchronicznie.
Odporność na zmiany	Wysoka dla formatów strukturalnych (CSV/XML), niska dla PDF (parsowanie PDF jest kruche wobec zmian layoutu).
Zastosowanie bankowe	Wysokie — mechanizm natywny, brak dodatkowych zależności bezpieczeństwa.
Złożoność / utrzymanie	Niska-średnia.
Ocena łączna	[W] Wysoka jako uzupełniająca metoda ekstrakcji wartości (patrz Część II), ograniczona jako jedyna metoda testu dostępności/renderowania

3.7 Analiza statyczna HTML/DOM (bez pełnego silnika przeglądarki, np. requests + BeautifulSoup)

Opis: Pobranie odpowiedzi HTTP strony raportu bez wykonywania JavaScript i sparsowanie surowego HTML.

Kryterium	Ocena
Wymagania	Biblioteka HTTP z obsługą Windows Auth (Kerberos/NTLM), parser HTML (BeautifulSoup, lxml).
Zalety	Bardzo szybkie i lekkie zasobowo (brak silnika przeglądarki).
Ograniczenia	Nieadekwatne dla Power BI — raporty Power BI to Single Page Application renderowany w całości przez JavaScript po stronie klienta; surowa odpowiedź HTTP nie zawiera wartości KPI (tylko szkielet aplikacji JS). Częściowo adekwatne dla SSRS w trybie klasycznego renderowania HTML, ale nowoczesny portal SSRS (od 2017) także w dużej mierze opiera się na komponentach JS (ReportViewer/Report Portal). W praktyce niewystarczające jako samodzielna metoda dla żadnej z platform w tym projekcie.
Wpływ na wydajność	Bardzo dobry, ale nieistotny wobec ograniczeń funkcjonalnych.
Odporność na zmiany	n/d — metoda niefunkcyjna dla Power BI.
Zastosowanie bankowe	Wysokie technicznie, ale niskie praktycznie z powodu ograniczeń funkcjonalnych.
Złożoność / utrzymanie	Niska, ale niska wartość dostarczana.

Kryterium	Ocena
Ocena łączna	[N] Niska — nie nadaje się jako samodzielna metoda integracji dla Power BI; ograniczone zastosowanie dla starszych, czysto server-side raportów SSRS

3.8 Inne rozważane podejścia

- **Power BI Embedded Analytics (powerbi-client JS SDK w kontrolowanej aplikacji embed)** — wymagałoby przebudowy sposobu publikacji raportów przez organizację (osadzenie raportów we własnej aplikacji embed) — poza zakresem projektu, który zakłada integrację z **istniejącą** platformą BI, a nie jej zastąpienie.
- **Komercyjne narzędzia RPA (UiPath, Automation Anywhere, Power Automate Desktop)** — funkcjonalnie zdolne do realizacji podobnego zadania (sterowanie przeglądarką), ale wprowadzają istotny koszt licencyjny, zależność od zewnętrznego środowiska wykonawczego (Robot) oraz mniejszą elastyczność integracji z natywnym stackiem Python/FastAPI wskazanym w projekcie; rozważane jako alternatywa, ale niższa transparentność kodu (low-code) utrudnia utrzymanie w modelu Clean Architecture/Test-Driven wymaganych dla systemu tej klasy.
- **Power BI Report Server On-Premises Data Gateway / PowerShell cmdlets (ReportingServicesTools)** — użyteczne do zadań administracyjnych (inwentaryzacja, harmonogramy), rozważane jako pomocnicze narzędzie dla modułu Configuration Repository przy imporcie listy raportów, nie jako mechanizm testowy.
- **Sonda syntetyczna na poziomie sieci (health check TCP/HTTP endpointu)** — zbyt płytka (potwierdza jedynie, że serwer odpowiada na warstwie sieciowej, nie że raport się renderuje poprawnie) — rozważana wyłącznie jako uzupełniający, tani “sygnał wczesnego ostrzegania” w module Monitoring, nie jako metoda walidacji raportu.

3.9 Tabela porównawcza — integracja z platformą raportową

Podejście	PBI	SSRS	Wydajność	Odporność na zmiany	Bankowość	Złożoność	Rekomendacja
Browser Automation (Playwright)	[W]	[W]	[O]	[W]	[W]	[O]	Warstwa bazowa — obowiązkowa
Selenium	[W]	[W]	[O]	[W]	[W]	[N]	Odrzucone na rzecz Playwright
Power BI REST API	[O]	[-]	[W]	[W]	[O]	[W]	Wsparcie administracyjne / trigger odświeżeń
XMLA Endpoint	[O]*	[-]	[W]	[W]	[W]	[O]	Opcjonalny connector “drugiej linii” (tylko Premium)

Podejście	PBI	SSRS	Wydajność	Odporność na zmiany	Bankowość	Złożoność	Rekomendacja
SSRS Web Services / URL Access Eksport (CSV/XML)	[–]	[W]	[W]	[O]	[W]	[W]	Natywny connector SSRS
Statyczny HTML/ DOM (bez JS)	[O]	[W]	[O]	[W]	[W]	[O]	Uzupełniająca metoda ekstrakcji wartości
	[N]	[O]	[W]	[N]	[W]	[W]	Odrzucone

* zależne od licencji Premium/PPU/Fabric.

4. Rekomendowana architektura integracji

ARGUS przyjmuje architekturę wielowarstwową (multi-connector), w której Browser Automation (Playwright) pełni rolę uniwersalnej warstwy bazowej, a natywne API (SSRS Web Services, Power BI REST API, opcjonalnie XMLA) są wykorzystywane tam, gdzie podnoszą precyzję lub wydajność, bez zastępowania testu end-to-end.

Uzasadnienie wyboru Browser Automation jako warstwy bazowej

1. **Jest to jedyne podejście działające identycznie dla obu technologii (PBI i SSRS)**, co pozwala zbudować **jeden spójny rdzeń silnika testowego** i sprawdzić różnice platformowe wyłącznie do warstwy Connectora (zgodnie z zasadą Open/Closed — patrz 03_Architecture.md).
2. **Jest to jedyne podejście, które realnie weryfikuje to, co widzi użytkownik końcowy** — łącznie z autoryzacją SSO, czasem ładowania, błędami JS, błędami wizualizacji, a nie tylko dostępność API. Cel biznesowy projektu to zastąpienie analityka, który “otwiera raport i patrzy” — Browser Automation jest jedynym podejściem odwzorowującym dokładnie tę czynność.
3. **Nie wymaga żadnych dodatkowych licencji** (Premium/PPU) ani zmiany modelu publikacji raportów — działa z dnia na dzień na już istniejącej platformie BI, bez ingerencji w sposób jej wdrożenia.
4. **Skaluje się horyzontalnie** — worker pool przeglądarek headless można uruchamiać równolegle na wielu instancjach/kontenerach, co jest kluczowe dla celu 1000 raportów.

Rola API i eksportów jako “drugiej linii”

Tam, gdzie są one dostępne, natywne API **nie zastępują** testu Browser Automation, lecz **wzbogacają Validation Profile o dodatkowy, bardziej precyzyjny kanał odczytu wartości** (patrz Część II, sekcja 6) oraz ociążają warstwę przeglądarkową z zadań czysto administracyjnych (np. wymuszenie odświeżenia datasetu przed testem, pobranie metadanych katalogu SSRS do automatycznej inwentaryzacji raportów w Configuration Repository).

Ta hybrydowa strategia jest realizowana architektonicznie przez wzorzec **Strategy + Chain of Responsibility** wewnątrz Value Extraction Engine — patrz sekcja 7 poniżej oraz 03_Architecture.md.

5. Strategia odporności na zmiany raportów (Resilience Strategy)

Ponieważ liczba raportów (docelowo 1000) uniemożliwia ręczne utrzymanie sztywnych, "kruchych" selektorów, ARGUS wdraża następujące zasady projektowe od samego początku:

1. **Selektory semantyczne, nie pozycyjne/wizualne** — wykorzystanie atrybutów `aria-label`, ról ARIA, nazw elementów Power BI (np. nazwa wizualizacji nadana przez autora raportu) zamiast współrzędnych pikselowych czy generowanych klas CSS.
2. **Multi-selector fallback** — dla każdego punktu ekstrakcji Validation Profile może definiować więcej niż jedną strategię odnalezienia elementu (np. "po nazwie wizualizacji" → "po pozycji w siatce" → "po indeksie w kolejności renderowania"), próbowane sekwencyjnie.
3. **Confidence Score** ekstrakcji — każda odczytana wartość otrzymuje wskaźnik pewności odczytu (np. czy selektor trafił jednoznacznie w jeden element, czy zastosowano fallback niższego poziomu) — widoczny w GUI i wykorzystywany przez moduł AI do priorytetyzacji przeglądu.
4. **Automatyczna detekcja "podejrzanej zmiany strukturalnej"** — jeśli struktura DOM raportu zmieni się istotnie względem poprzedniego udanego przebiegu, system oznacza to jako alert dla administratora Validation Profile, zamiast cicho zwracać błędny odczyt.
5. **Wersjonowanie Validation Profile** — każda zmiana selektora/profilu jest wersjonowana i audytowalna (kto, kiedy, dlaczego zmienił definicję testu).

6. Część II — Value Extraction Engine: metody odczytu wartości KPI z raportu

To najbardziej newralgiczny komponent całego systemu — jego niezawodność determinuje zaufanie użytkowników biznesowych do wyników ARGUS.

6.1 Analiza DOM (po wyrenderowaniu przez przeglądarkę)

Kryterium	Ocena
Zalety	Odczyt dokładnie tego tekstu/liczby, który widzi użytkownik. Działa dla wizualizacji renderowanych jako tekst/tabela (karty KPI, tabele, matryce) — najczęstszy typ wizualizacji używany do prezentacji pojedynczych wskaźników.
Ograniczenia	Nie działa dla wartości renderowanych wyłącznie jako grafika wektorowa/Canvas bez odpowiadającego tekstu w DOM (niektóre typy wykresów Power BI renderują się częściowo na <code><canvas></code>). Wymaga strategii selekcji odpornej na zmiany (patrz sekcja 5).
Odporność na zmiany	Średnia-wysoka przy zastosowaniu selektorów semantycznych.
Wpływ na wydajność	Bardzo dobry — odczyt po stronie już załadowanej strony, rzędu dziesiątek ms.
Niezawodność	Wysoka dla kart KPI/tabel (dominujący przypadek użycia dla "wartości KPI" wg briefu).
Power BI	[W] Bardzo dobre dla kart, tabel, macierzy.
SSRS	[W] Bardzo dobre — SSRS renderuje tabele/textboxy jako elementy HTML/SVG z tekstem w DOM.

6.2 Wykorzystanie natywnego API / zapytań do modelu danych (XMLA/DAX, SSRS SOAP dataset)

Kryterium	Ocena
Zalety	Najwyższa możliwa precyzja — wartość pobrana bezpośrednio z silnika obliczeniowego, tożsama co do bitu z tym, co obliczyłaby wizualizacja. Brak zależności od layoutu wizualnego raportu.
Ograniczenia	Wymaga znajomości definicji miary/zapytania leżącego pod wizualizacją (utrzymywane w Validation Profile przez administratora/analityka). Dla Power BI ograniczone do licencji Premium/PPU/Fabric (XMLA). Nie potwierdza, że wizualizacja faktycznie renderuje tę wartość poprawnie na ekranie (możliwy błąd formatowania/mapowania w samej wizualizacji, niewykrywalny tą metodą).
Odporność na zmiany	Bardzo wysoka wobec zmian UI, niska wobec zmian nazw miar (rzadkie, kontrolowane zmiany).
Wpływ na wydajność	Bardzo dobry.
Niezawodność	Bardzo wysoka tam, gdzie dostępne.
Power BI	[O] Dobrze, warunkowe licencyjnie.
SSRS	[O] Częściowo — poprzez ReportExecution2005.Render do formatu danych (XML/CSV) można odczytać wartość bezpośrednio z zestawu danych raportu.

6.3 Eksport danych (CSV/XML/Excel) i parsowanie strukturalne

Kryterium	Ocena
Zalety	Bardzo wysoka niezawodność parsowania (dane w formie ustrukturyzowanej, nie tekstu wizualnego). Dobry “ślad audytowy” — plik eksportu można zachować jako dowód.
Ograniczenia	Generowanie pełnego eksportu bywa wolniejsze niż odczyt jednej wartości z DOM. Zmiana układu kolumn w raporcie wymaga aktualizacji mapowania w Validation Profile.
Odporność na zmiany	Wysoka (format danych), średnia (układ kolumn).
Wpływ na wydajność	Średni (generacja pełnego dokumentu), ale wykonywalne asynchronicznie/w tle.
Niezawodność	Wysoka.
Power BI	[O] Dobrze (Export to File REST API), z ograniczeniami rate-limit i asynchronicznością.
SSRS	[W] Bardzo dobre (URL Access rs:Format=CSV/XML — natywne, szybkie do wdrożenia).

6.4 Analiza HTML (statyczny parser, bez wykonania JS)

Jak w sekcji 3.7 — nieadekwatne dla Power BI (SPA renderowany przez JS), ograniczone dla SSRS. Odrzucone jako samodzielna metoda ekstrakcji.

6.5 Browser Automation jako mechanizm ekstrakcji (odczyt po renderowaniu — accessibility tree, innerText, atrybuty)

Rozszerzenie DOM Analysis (6.1) o pełne możliwości Playwright: odczyt drzewa dostępności (Accessibility Tree), które Power BI aktywnie utrzymuje dla potrzeb czytników ekranu — jest to **bardzo stabilne semantycznie źródło**, bo Microsoft utrzymuje je pod kątem zgodności z WCAG, niezależnie od zmian czysto wizualnych.

Power BI | **[W]** Bardzo dobre — accessibility tree Power BI jest zaskakująco stabilnym i mało wykorzystywanym źródłem odczytu wartości kart/tabel. |
SSRS | **[W]** Dobrze. |

6.6 Computer Vision (detekcja układu, template matching, analiza wykresu)

Kryterium	Ocena
Zalety	Jedyna metoda działająca dla wizualizacji renderowanych czysto graficznie (np. niektóre wykresy na <canvas> bez warstwy tekstowej w DOM), gdzie DOM/API zawodzą. Może wykrywać także anomalie wizualne niezwiązane z konkretną liczbą (np. “wykres jest pusty”, “brak legendy”, “nakładające się etykiety”).
Ograniczenia	Wysoka złożoność implementacyjna. Niższa precyzja numeryczna niż DOM/API (zwłaszcza dla wartości z wieloma miejscami po przecinku). Wrażliwe na zmiany kolorystyki/motywu raportu.
Odporność na zmiany	Średnia.
Wpływ na wydajność	Wyższy koszt obliczeniowy (przetwarzanie obrazu, ewentualnie model ML).
Niezawodność	Średnia — rekomendowane jako metoda ostatniej linii fallbacku , nie podstawowa.
Power BI	[O] Fallback dla wykresów canvas.
SSRS	[O] Rzadziej potrzebne (SSRS częściej renderuje tekst w DOM/SVG).

6.7 OCR (Optical Character Recognition)

Kryterium	Ocena
Zalety	Działa “na obrazie”, niezależnie od technologii renderującej — uniwersalny fallback ostatniej instancji (np. dla zrzutu ekranu całego raportu, gdy żadna inna metoda nie zidentyfikowała elementu).
Ograniczenia	Najniższa precyzja i niezawodność spośród wszystkich metod (ryzyko błędnego odczytu cyfr, separatorów dziesiętnych, jednostek). Najwyższy koszt czasowy. Wymaga starannej walidacji formatu liczby po odczycie (np. reguła “odczytana wartość musi pasować do wzorca liczby, inaczej odrzuć wynik i oznacz jako niepewny”).
Odporność na zmiany	Wysoka na zmiany strukturalne DOM (bo nie zależy od struktury), ale niska precyzja per se.
Wpływ na wydajność	Najwyższy koszt spośród analizowanych metod.
Niezawodność	Niska-średnia — wyłącznie jako ostateczny fallback z obowiązkowym oznaczeniem niskiego Confidence Score i skierowaniem do przeglądu (ewentualnie z udziałem modułu AI jako “podpowiedzi”, nigdy jako źródła automatycznej decyzji).
Power BI	[O] Fallback ostatniej instancji.
SSRS	[O] Fallback ostatniej instancji.

6.8 Tabela porównawcza — Value Extraction Engine

Metoda	Niezawodność	Wydajność	Odporność na zmiany	PBI	SSRS	Warstwa strategii
DOM Analysis (karty/tabele)	[W] Wysoka	[W]	[O]	[W]	[W]	Warstwa 1 — podstawowa
Accessibility Tree	[W] Wysoka	[W]	[W]	[W]	[W]	Warstwa 1 — podstawowa
API / model danych (XMLA/DAX, SSRS dataset)	[W] Bardzo wysoka	[W]	[W]	[O]*	[O]	Warstwa 0 — precyzyjna, gdy dostępna

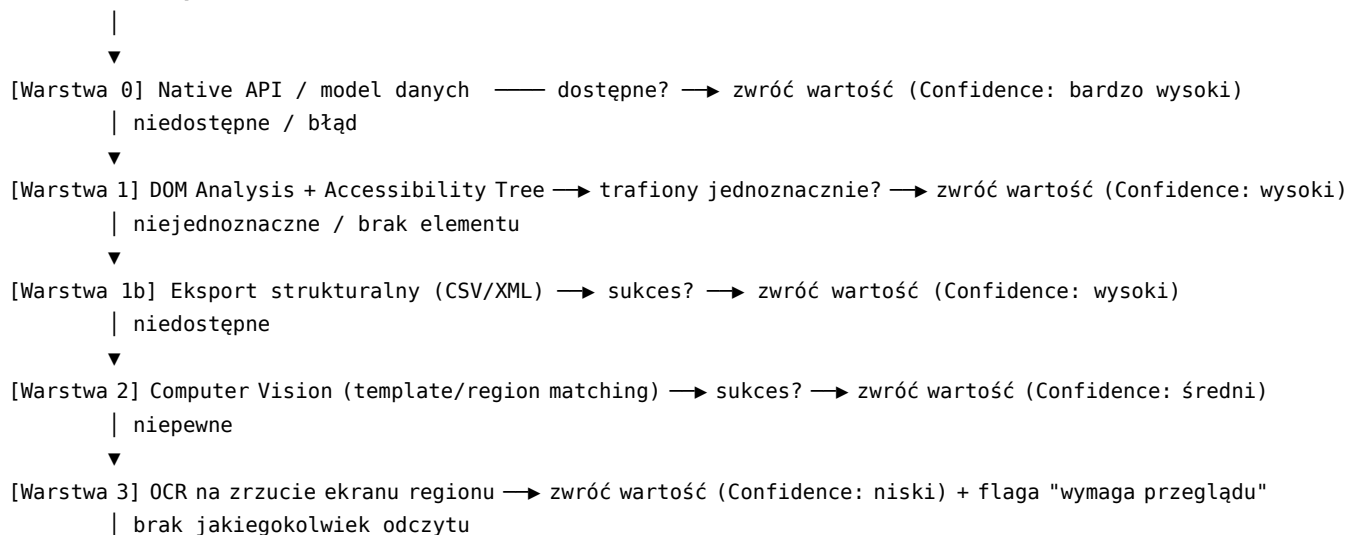
Metoda	Niezawodność	Wydajność	Odporność na zmiany	PBI	SSRS	Warstwa strategii
Eksport strukturalny (CSV/XML)	[W] Wysoka	[O]	[W]	[O]	[W]	Warstwa 1b — alternatywna
Computer Vision	[O] Średnia	[O]	[O]	[O]	[O]	Warstwa 2 — fallback
OCR	[N] Niska-średnia	[N]	[W]	[O]	[O]	Warstwa 3 — fallback ostateczny
Statyczny HTML (bez JS)	[N] Niska	[W]	[N]	[N]	[O]	Odrzucone

* *zależne od licencji.*

7. Rekomendowana architektura Value Extraction Engine — strategia wielowarstwowego fallbacku

System **nie wybiera jednej metody globalnie**, lecz stosuje **łańcuch strategii ekstrakcji per punkt danych**, skonfigurowany deklaratywnie w Validation Profile, zgodnie ze wzorcem projektowym **Chain of Responsibility** opakowanym w **Strategy Pattern** (każda strategia implementuje wspólny interfejs `IExtractionStrategy`):

Punkt ekstrakcji KPI w Validation Profile



Status: `EXTRACTION_FAILED` → alert + zadanie w kolejce przeglądu manualnego

Uzasadnienie:

1. **Determinizm i przewidywalność kosztu** — najdroższe metody (CV/OCR) uruchamiane są tylko wtedy, gdy tańsze i pewniejsze zawiodły, co optymalizuje koszt obliczeniowy przy skali 1000 raportów.
2. **Confidence Score jako obywatel pierwszej klasy** — każdy wynik niesie ze sobą informację o pewności odczytu, co jest kluczowe dla modułu AI (priorytetyzacja anomalii) oraz dla zaufania użytkownika biznesowego do automatyzacji.

3. **Odporność na zmiany raportów** — awaria jednej warstwy nie unieważnia całego testu, tylko degradowuje pewność wyniku i uruchamia kolejną strategię.
4. **Rozszerzalność (Open/Closed)** — dodanie nowej metody ekstrakcji (np. w przyszłości integracja z nowym modelem AI vision) to dodanie nowej klasy implementującej `IExtractionStrategy`, bez zmian w rdzeniu silnika.

Pełny opis wzorców i interfejsów — patrz `03_Architecture.md`, sekcja “Value Extraction Engine”.

8. Otwarte pytania do wyjaśnienia w Fazie 0 (Discovery / PoC)

Poniższe kwestie wymagają potwierdzenia przez zespół administrujący Platformą BI przed rozpoczęciem implementacji (patrz `11_Roadmap.md`, Faza 0):

1. Czy Power BI jest wdrożone jako **Power BI Report Server on-prem**, czy jako usługa hybrydowa z komponentem chmurowym (wpływa na dostępność REST API i XMLA)?
2. Czy dostępna jest licencja Premium/PPU/Fabric dla choćby części raportów (kwalifikacja do connectora XMLA jako opcji “drugiej linii”)?
3. Jaki jest dokładny mechanizm SSO — Kerberos/SPNEGO czy NTLM — oraz czy konto techniczne ARGUS będzie mogło działać w modelu delegacji (impersonacji) czy wyłącznie jako dedykowane konto serwisowe z bezpośrednim dostępem nadanym do raportów?
4. Czy istnieje (lub może powstać) scentralizowany katalog metadanych raportów (spis 300/1000 raportów wraz z adresami i obszarami biznesowymi), czy ARGUS musi zbudować własną inwentaryzację od zera w module Configuration Repository?
5. Jakie są polityki sieciowe dot. ruchu wychodzącego z serwerów ARGUS do `pbi.bank.com.pl` oraz `bgkssrs` (segmentacja sieci, wymagane wyjątki firewall)?

9. Podsumowanie rekomendacji

Obszar	Rekomendacja
Warstwa integracji z raportem	Playwright (Browser Automation) jako warstwa bazowa, uniwersalna dla PBI i SSRS
Connector SSRS — wzbogacenie	SSRS Web Services / URL Access do metadanych katalogu i eksportu strukturalnego
Connector Power BI — wzbogacenie	Power BI REST API do operacji administracyjnych; XMLA jako opcjonalny connector precyzyjny (Premium)
Odczyt wartości KPI	Silnik wielowarstwowego fallbacku: API/model danych → DOM/Accessibility Tree → Eksport strukturalny → Computer Vision → OCR
Zasada nadrzędna	Każda decyzja o niedostępności/błędzie raportu musi być deterministyczna i wyjaśnialna — AI komentuje, nie decyduje (patrz <code>06_AI_Module.md</code>)

01. Project Overview

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Przegląd projektu (Project Overview)

Odbiorcy: Sponsor, Project Manager, Architektura IT, Zespół Deweloperski

1. Nazwa projektu i uzasadnienie

ARGUS — od Argosa Panoptesa, stuokiego strażnika z mitologii greckiej, którego oczy nigdy nie zasypiały w komplecie. System ma pełnić dokładnie taką rolę: nieustannie “obserwować” setki, a docelowo tysiąc raportów jednocześnie, wychwytyjąc nieprawidłowości niewidoczne dla pojedynczego obserwatora działającego ręcznie.

Nazwa spełnia kryteria dobrej nazwy produktu enterprise: - **krótka i łatwa do zapamiętania** w każdym języku i kontekście (rozmowy, dokumentacja, commit messages), - **nośnik metafory zgodnej z funkcją systemu** (czujność, kompletność obserwacji, brak “martwych pól”), - **neutralna technologicznie** — nie sugeruje wyłącznie Power BI ani wyłącznie SSRS, co jest zgodne z zasadą niezależności platformowej systemu, - **dobrze skracalna** w nazewnictwie technicznym repozytoriów/usług: argus-core, argus-connectors, argus-ai, argus-ui, argus-api.

Pełna nazwa formalna do użytku w dokumentach zarządczych: **ARGUS — Automated Reporting Governance & Unified Surveillance Platform.**

2. Wizja produktu

Żaden błąd w raporcie BI nie dotrze do decydenta biznesowego, zanim nie zostanie automatycznie wykryty, zweryfikowany względem źródła danych i zgłoszony — szybciej i pewniej, niż mógłby to zrobić zespół analityków pracujący ręcznie.

ARGUS ma zastąpić czasochłonny, ręczny proces kontroli jakości raportów Platformy BI systemem, który:

- działa **w sposób ciągły i deterministyczny**, a nie okazjonalny i uznaniowy,
 - **skala się liniowo** wraz ze wzrostem liczby raportów (z ok. 300 do docelowo ≥ 1000 , a w przyszłości potencjalnie więcej),
 - **buduje historię i ślad audytowy** jakości każdego raportu w czasie — czego dziś praktycznie nie ma,
 - **wspiera, a nie zastępuje** kompetencje analityczne zespołu — poprzez moduł AI, który tłumaczy wyniki i wskazuje prawdopodobne przyczyny, pozostawiając ostateczną ocenę deterministycznemu silnikowi reguł, a decyzję biznesową — człowiekowi.
-

3. Cele projektu

3.1 Cele biznesowe

ID	Cel biznesowy	Miernik sukcesu
BC-01	Redukcja czasu i kosztu ręcznej weryfikacji raportów	Skrócenie czasu potrzebnego na pełen przegląd katalogu raportów z dni do minut/godzin
BC-02	Wzrost zaufania do danych prezentowanych w raportach BI	Spadek liczby incydentów “błędny raport wykryty przez biznes zamiast przez IT”
BC-03	Skalowalność procesu QA wraz ze wzrostem katalogu raportów	System obsługujący ≥ 1000 raportów bez wzrostu nakładu pracy ludzkiej proporcjonalnego do liczby raportów
BC-04	Budowa historii jakości danych jako aktywa organizacji	Dostępna, przeszukiwalna historia wyników testów dla każdego raportu od dnia wdrożenia
BC-05	Wsparcie zgodności i audytowalności (compliance)	Pełny ślad audytowy: kto, kiedy, jaki test zdefiniował/zmienił, jaki był wynik i dlaczego

3.2 Cele techniczne

ID	Cel techniczny	Powiązany dokument
TC-01	Architektura niezależna od konkretnej technologii raportowej (rozszerzalna o nowe platformy)	03_Architecture.md
TC-02	Jednolity mechanizm testowy dla Power BI i SSRS	00_Technical_Feasibility_Study.md
TC-03	Odporność na drobne zmiany wizualne raportów (self-healing/fallback)	00_Technical_Feasibility_Study.md, sekcja 7
TC-04	Pełna separacja logiki domenowej od infrastruktury (Clean Architecture, SOLID, DI)	03_Architecture.md
TC-05	Wyjaśnialność każdej decyzji o błędzie/poprawności raportu	06_AI_Module.md
TC-06	Bezpieczeństwo klasy bankowej (SSO, RBAC, audyt, least privilege)	05_NonFunctional_Requirements.md

4. Zakres projektu

4.1 W zakresie (In Scope)

- Automatyczne otwieranie i sprawdzanie dostępności/renderowania raportów Power BI i SSRS.
- Odczyt wskazanych wartości KPI z raportów oraz porównanie ich z wynikami zapytań SQL do hurtowni danych.
- Definiowanie, wersjonowanie i wykonywanie testów biznesowych (Validation Profiles) przez administratora/analityka.
- Harmonogramowanie testów (cykliczne i na żądanie).
- Przechowywanie historii wyników testów wraz z metrykami (Health Score, Confidence Score).
- Moduł AI wspierający interpretację wyników, wykrywanie anomalii i generowanie czytelnych podsumowań.
- Aplikacja webowa (GUI) do przeglądu raportów, konfiguracji, historii i wyników.

- Powiadomienia o wykrytych problemach (np. e-mail, integracja z kanałem powiadomień organizacji).
- Mechanizmy bezpieczeństwa, audytu i monitoringu zgodne ze standardami środowiska bankowego.

4.2 Poza zakresem (Out of Scope) — w bieżącej fazie

- Automatyczne podejmowanie decyzji biznesowych na podstawie wyników testów (decyzje pozostają po stronie ludzi, wspieranych przez AI — patrz 06_AI_Module.md).
- Integracja z platformami raportowymi innymi niż Power BI i SSRS w fazie MVP (przewidziana architektonicznie, ale poza zakresem pierwszego wdrożenia — patrz 11_Roadmap.md).

5. Interesariusze (Stakeholders)

Rola	Odpowiedzialność w projekcie	Kluczowe zainteresowania
Sponsor biznesowy	Zatwierdzenie budżetu i priorytetów	ROI, redukcja ryzyka operacyjnego
Project Manager	Harmonogram, zasoby, komunikacja	Terminowość, zakres, ryzyka
Architekt IT / Enterprise Architecture	Zgodność z architekturą korporacyjną	Standardy, integracje, dług technologiczny
Zespół Bezpieczeństwa / Security Office	Ocena ryzyka bezpieczeństwa	Zarządzanie kontem technicznym, RBAC, audyt, dane wrażliwe
Administrator Platformy BI	Udostępnienie dostępów, wiedza o środowisku PBI/SSRS	Stabilność platformy, obciążenie generowane przez testy
Zespół Deweloperski	Implementacja	Jakość dokumentacji wejściowej, jasność wymagań
Analitycy biznesowi / Właściciele raportów	Definiowanie testów i reguł biznesowych, konsumenci wyników	Trafność testów, minimalizacja fałszywych alarmów
Data Governance / Compliance	Nadzór nad jakością i audytowalnością danych	Ślad audytowy, retencja danych, zgodność regulacyjna

6. Kluczowe założenia projektowe

1. System działa **wewnątrz infrastruktury organizacji** (on-prem), z dedykowanym kontem technicznym mającym dostęp do Platformy BI oraz hurtowni danych — brak zależności od usług chmurowych publicznych jako wymogu obowiązkowego (opcjonalna integracja z Azure OpenAI jako usługą regulowaną/kontraktową — patrz 06_AI_Module.md).
2. Autoryzacja użytkowników do samych raportów pozostaje bez zmian (SSO/Windows Authentication) — ARGUS **nie zastępuje** i nie modyfikuje mechanizmu autoryzacji Platformy BI; korzysta z własnego konta technicznego o kontrolowanym, najniższym niezbędnym zakresie uprawnień.
3. Docelowa skala — **1000 raportów** — jest wymaganiem projektowym od dnia pierwszego, nawet jeśli faza MVP obejmuje mniejszy podzbiór (patrz 11_Roadmap.md). Architektura musi być zaprojektowana pod tę skalę, a nie dostosowywana post factum.
4. Administrator/analitik biznesowy **musi** zdefiniować test dla raportu, aby ten był objęty monitoringiem — ARGUS nie zgaduje reguł biznesowych automatycznie (poza wspierającą rolę AI w sugerowaniu konfiguracji, patrz 06_AI_Module.md).
5. AI pełni rolę **wyłącznie wspierającą** — nie podejmuje decyzji o poprawności raportu (zasada nadrzędna projektu — patrz 06_AI_Module.md).

7. Ograniczenia projektu

- Zależność od stabilności i dostępności API/mechanizmów udostępnianych przez Power BI i SSRS — zmiany po stronie Microsoft (aktualizacje produktu) mogą wymagać adaptacji connectorów (mitygowane architekturą modułową — patrz 03_Architecture.md).
- Zależność od jakości i kompletności definicji testów dostarczanych przez administratorów — system jest tak dobry, jak zdefiniowane Validation Profiles.
- Środowisko bankowe wprowadza dodatkowe wymagania bezpieczeństwa i proceduralne (zarządzanie zmianą, testy penetracyjne, akceptacje Security Office), które wydłużają cykl wdrożenia względem środowisk nieregulowanych.
- Skala docelowa (1000 raportów) wymaga inwestycji w infrastrukturę wykonawczą (worker pool przeglądarek) — koszt operacyjny musi być uwzględniony w planowaniu budżetu (patrz 10_Deployment.md).

8. Słownik pojęć (Glossary)

Pojęcie	Definicja
Validation Profile	Zestaw konfiguracji dla pojedynczego raportu: adres, typ platformy, testy, sposób odnajdywania KPI, zapytania SQL, dopuszczalne odchylenia, reguły biznesowe, harmonogram.
Connector	Moduł implementujący integrację z konkretną platformą raportową (Power BI, SSRS, w przyszłości inne), zgodny ze wspólnym interfejsem IReportConnector.
Value Extraction Engine	Moduł odpowiedzialny za odczyt wartości KPI z wyrenderowanego raportu, działający w oparciu o strategię wielowarstwowego fallbacku.
Health Score	Zagregowany, liczbowy wskaźnik "zdrowia" raportu w danym momencie/okresie, wyliczany na podstawie wyników testów.
Confidence Score	Wskaźnik pewności pojedynczego odczytu/testu — informuje, jak bardzo system "ufa" danemu wynikowi (np. z uwagi na warstwę fallbacku, która go dostarczyła).
Test Orchestrator	Moduł koordynujący wykonanie zestawu testów dla raportu/grupy raportów.
Business Rule	Reguła biznesowa wykraczająca poza proste porównanie liczby (np. "suma kategorii = wartość całkowita", "wartość nie może być ujemna").
Obszar biznesowy (zakładka)	Grupa raportów w Platformie BI odpowiadająca jednostce/tematyce biznesowej (np. Sprzedaż, Kredyty, Ryzyko).
AI Analysis Engine	Moduł AI interpretujący wyniki testów — wykrywanie anomalii, klastrowanie błędów, generowanie wyjaśnień; nie podejmuje decyzji o poprawności.

9. Kryteria sukcesu projektu

Kryterium	Docelowa wartość
Pokrycie katalogu raportów testami	100% raportów posiada zdefiniowany Validation Profile w perspektywie 12 miesięcy od startu produkcyjnego
Czas pełnego cyklu testowego dla całego katalogu	≤ 4 godziny dla 1000 raportów (harmonogram nocny/wsadowy — patrz 05_NonFunctional_Requirements.md)
Dostępność systemu (SLA)	≥ 99.5% w godzinach pracy organizacji
Redukcja czasu manualnej weryfikacji	≥ 80% w stosunku do stanu obecnego
Trafność detekcji (niska liczba fałszywych alarmów)	Utrzymanie wskaźnika fałszywych alarmów na poziomie akceptowalnym przez biznes, mierzone i raportowane cyklicznie

Powiązanie z uzasadnieniem biznesowym — patrz 02_Business_Problem.md. Plan realizacji w fazach — patrz 11_Roadmap.md.

02. Business Problem

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Analiza problemu biznesowego i uzasadnienie inwestycji

Odbiorcy: Sponsor, Project Manager, Interesariusze biznesowi

1. Kontekst biznesowy

Organizacja utrzymuje wewnętrzną **Platformę BI** stanowiącą centralny punkt dostępu do raportów zarządczych i operacyjnych. Platforma obejmuje obecnie **ok. 300 raportów**, pogrupowanych tematycznie według obszarów biznesowych (zakładek — np. Sprzedaż, Kredyty, Ryzyko), zbudowanych w dwóch technologiach:

- **Microsoft Power BI** (<https://pbi.bank.com.pl/reports/powerbi/...>),
- **SQL Server Reporting Services / Report Server** (<https://bgkssrs/Reports/report/...>).

Trajektoria wzrostu liczby raportów wskazuje na docelowy katalog **co najmniej 1000 raportów**, co oznacza ponad trzykrotny wzrost względem stanu obecnego — bez odpowiadającego wzrostu zasobów ludzkich dedykowanych do ich kontroli jakości.

2. Obecny proces (AS-IS)

Weryfikacja poprawności raportu odbywa się dziś **w pełni manualnie**, według powtarzalnego, lecz nieskalowalnego schematu:

1. Analityk otwiera raport w przeglądarce.
2. Czeki na jego załadowanie (czas zależny od złożoności raportu i obciążenia platformy).
3. Wizualnie ocenia, czy raport w ogóle się otworzył i wyrenderował bez błędów.
4. Odczytuje wzrokiem jedną lub kilka wartości KPI z raportu.
5. Otwiera osobne narzędzie (np. SSMS) i uruchamia odpowiadające zapytanie SQL na hurtowni danych.
6. Porównuje wynik zapytania z wartością odczytaną z raportu — ręcznie, “na oko” lub w arkuszu kalkulacyjnym.
7. Przechodzi do kolejnego raportu i powtarza cały cykl.

2.1 Charakterystyka procesu

Cecha	Opis
Czasochłonność	Pojedynczy cykl (otwarcie + odczyt + zapytanie SQL + porównanie) trwa realistycznie kilka-kilkanaście minut na raport, w zależności od złożoności.
Powtarzalność	Proces identyczny dla każdego raportu, a mimo to wykonywany ręcznie za każdym razem — klasyczny kandydat do automatyzacji.

Cecha	Opis
Podatność na błędy	Porównanie “na oko” wartości liczbowych (formatowanie, zaokrąglenia, jednostki) jest podatne na przeoczenia, zwłaszcza przy wielogodzinnej, monotonnej pracy.
Brak śladu audytowego	Wynik ręcznej weryfikacji zwykle nie jest nigdzie systematycznie rejestrowany — nie istnieje historyczna, przeszukiwalna baza “czy i kiedy raport X był ostatnio poprawny”.
Brak powtarzalnej częstotliwości	Ze względu na koszt czasowy, pełny przegląd katalogu raportów odbywa się rzadko lub wybiórczo, a nie w sposób ciągły.
Skalowalność	Czas procesu rośnie liniowo wraz z liczbą raportów, podczas gdy zasoby ludzkie dedykowane do tego zadania nie rosną w tym samym tempie — przy 1000 raportach proces w obecnej formie staje się praktycznie niewykonalny w rozsądnym cyklu.

3. Konsekwencje biznesowe obecnego stanu

3.1 Ryzyko operacyjne i decyzyjne

Raporty BI w środowisku bankowym są podstawą podejmowania decyzji biznesowych (sprzedażowych, kredytowych, ryzykowych). Błędny raport, który nie zostanie wykryty na czas, może prowadzić do:

- podjęcia decyzji biznesowej na podstawie nieaktualnych lub błędnych danych,
- utraty zaufania odbiorców biznesowych do Platformy BI jako całości (efekt “raz oszukany, zawsze podejrzliwy” — spadek adopcji nawet poprawnych raportów),
- wydłużonego czasu wykrycia i naprawy błędu, jeśli zostanie on zauważony dopiero przez użytkownika biznesowego, a nie przez IT/analityków w toku rutynowej kontroli.

3.2 Koszt operacyjny

Manualna weryfikacja skaluje się liniowo z liczbą raportów, co oznacza, że wzrost katalogu z 300 do 1000 raportów (założenie docelowe) **potroiłby** nakład pracy potrzebny do utrzymania obecnego poziomu kontroli — albo, co bardziej prawdopodobne przy niezmiennych zasobach, **proporcjonalnie obniżyłby rzeczywiste pokrycie kontrolą** (mniejszy odsetek raportów faktycznie regularnie sprawdzanych).

3.3 Brak wiedzy organizacyjnej o jakości danych w czasie

Brak systematycznego rejestrowania wyników weryfikacji oznacza, że organizacja nie dysponuje danymi do odpowiedzi na pytania typu: “Który raport najczęściej ulega awarii?”, “Czy jakość raportów w obszarze Kredyty pogarsza się w czasie?”, “Jak długo trwało wykrycie ostatniej rozbieżności?” — a to są pytania kluczowe dla governance danych i priorytetyzacji działań naprawczych.

4. Analiza przyczyn źródłowych (Root Cause)

Proces manualny nie jest wynikiem złej woli czy braku kompetencji zespołu — jest **strukturalnie nieskalowalny**, ponieważ:

1. Nie istnieje jeden, ujednolicony mechanizm dostępu programistycznego do “tego, co widzi użytkownik” w obu technologiach raportowych jednocześnie (stąd konieczność dogłębnej analizy technicznej — patrz 00_Technical_Feasibility_Study.md).

2. Testy porównawcze (raport vs. hurtownia) wymagają wiedzy specjalistycznej (SQL, znajomość modelu danych) połączonej z dostępem do samego raportu — dziś skoncentrowanej w głowach pojedynczych analityków, a nie w reużywalnej, wersjonowanej konfiguracji.
3. Brak platformy, która pozwoliłaby **raz zdefiniować** regułę testową i **wielokrotnie, automatycznie** ją wykonywać w czasie.

5. Stan docelowy (TO-BE)

Po wdrożeniu ARGUS proces wygląda następująco:

1. Administrator/analityk definiuje **Validation Profile** dla raportu — jednorazowo, z możliwością aktualizacji w miarę zmian.
2. **Scheduler** automatycznie uruchamia testy zgodnie ze zdefiniowaną częstotliwością (lub na żądanie).
3. System **automatycznie** otwiera raport, sprawdza dostępność i renderowanie, odczytuje wartości KPI, wykonuje odpowiadające zapytania SQL do hurtowni i porównuje wyniki — bez udziału człowieka w pętli wykonawczej.
4. Wyniki są zapisywane w historii, agregowane w **Health Score**, a w razie wykrycia rozbieżności — moduł AI generuje czytelne wyjaśnienie i wskazuje prawdopodobną przyczynę, a **Notification Service** informuje odpowiednie osoby.
5. Analityk/administrator poświęca czas **wyłącznie** na przypadki faktycznie wymagające uwagi (odsiane i priorytetyzowane przez system), zamiast na rutynowe sprawdzanie raportów działających poprawnie.

6. Uzasadnienie inwestycji (Business Case — jakościowe)

Wymiar	Stan obecny (AS-IS)	Stan docelowy (TO-BE z ARGUS)
Zakres kontroli	Wybiórczy, ograniczony przepustowością ludzką	Pełne pokrycie katalogu raportów (docelowo 1000)
Częstotliwość	Nieregularna, zależna od dostępności analityka	Regularna, harmonogramowana, konfigurowalna per raport
Czas wykrycia błędu	Godziny-dni (zależnie od priorytetu ręcznego przeglądu)	Minuty-godziny od wystąpienia (zgodnie z harmonogramem testów)
Ślad audytowy	Brak lub fragmentaryczny	Pełna, przeszukiwalna historia + audyt zmian konfiguracji
Koszt krańcowy dodania kolejnego raportu do kontroli	Rosnący liniowo wraz z liczbą raportów (koszt osobowy)	Marginalny (konfiguracja Validation Profile), niezależny od skali infrastruktury wykonawczej
Wiedza organizacyjna o jakości danych w czasie	Anegdotyczna	Ustrukturyzowana, mierzalna (Health Score, trendy)

Niniejszy dokument stanowi jakościowe uzasadnienie biznesowe. Ilościowa estymacja kosztów wdrożenia i utrzymania (infrastruktura, zespół, licencje) powinna zostać wykonana przez Project Managera we współpracy z Architekturą IT na bazie stacku technologicznego zdefiniowanego w 09_Python_Architecture.md oraz planu faz w 11_Roadmap.md, jako odrębne ćwiczenie budżetowe — poza zakresem niniejszego dokumentu architektonicznego.

7. Powiązanie z dalszą dokumentacją

- Szczegółowa analiza techniczna sposobu realizacji — 00_Technical_Feasibility_Study.md.
- Wymagania funkcjonalne wynikające z opisanego procesu — 04_Functional_Requirements.md.
- Plan wdrożenia fazowego, minimalizujący ryzyko i czas do pierwszej wartości biznesowej — 11_Roadmap.md.
- Rejestr ryzyk projektu, w tym ryzyk związanych z akceptacją zmiany procesu przez organizację — 12_Risks.md.

03. Architecture

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Architektura systemu

Odbiorcy: Architektura IT, Zespół Deweloperski, Security Office

1. Zasady architektoniczne (Architectural Principles)

Architektura ARGUS jest projektowana w oparciu o pięć nadrzędnych zasad wskazanych w wymaganiach projektowych, konsekwentnie stosowanych na wszystkich poziomach systemu:

1.1 SOLID

- **S — Single Responsibility.** Każdy moduł (patrz sekcja 4) ma dokładnie jedną, jasno zdefiniowaną odpowiedzialność (np. Scheduler wyłącznie harmonogramuje, nie wykonuje testów; Value Extraction Engine wyłącznie odczytuje wartości, nie waliduje ich poprawności biznesowej).
- **O — Open/Closed.** System jest **otwarty na rozszerzenie, zamknięty na modyfikację** — kluczowy wymóg projektowy: dodanie nowej platformy raportowej (np. Tableau w przyszłości) wymaga wyłącznie dodania nowej klasy implementującej IReportConnector, bez zmian w silniku orkiestracji, walidacji czy GUI.
- **L — Liskov Substitution.** Każdy IReportConnector (Power BI, SSRS, przyszłe) jest w pełni wymienny z punktu widzenia Test Orchestrator — orkiestrator nie zawiera logiki `if platform == "PowerBI"`.
- **I — Interface Segregation.** Interfejsy są wąskie i zorientowane na konkretny kontrakt (IReportConnector, IExtractionStrategy, ISqlValidator, INotificationChannel) zamiast jednego wszechogarniającego interfejsu.
- **D — Dependency Inversion.** Warstwy wyższe (domena, logika aplikacyjna) zależą od **abstrakcji**, nie od konkretnych implementacji infrastrukturalnych (np. domena zależy od IReportRepository, a nie bezpośrednio od SQLAlchemy) — realizowane przez Dependency Injection (sekcja 3.4).

1.2 Clean Architecture

System jest podzielony na koncentryczne warstwy, gdzie zależności wskazują **zawsze do wewnątrz** (Dependency Rule): warstwa Infrastruktury zależy od Domeny, nigdy odwrotnie.

1.3 Separation of Concerns

Rozdzielenie odpowiedzialności realizowane jest na trzech poziomach jednocześnie: - **Poziom modułów** — 17 modułów o rozłącznych odpowiedzialnościach (sekcja 4). - **Poziom warstw** — Presentation / Application / Domain / Infrastructure (sekcja 2). - **Poziom przekrojowy (cross-cutting)** — bezpieczeństwo, logowanie, audyt, konfiguracja jako aspekty niezależne od logiki biznesowej (sekcja 5).

1.4 Dependency Injection

Wszystkie zależności między komponentami są **wstrzykiwane**, a nie tworzone wewnątrz klas (new/bezpośrednia instancjacja). W warstwie API realizowane natywnym mechanizmem `depends()` FastAPI; w warstwie domenowej i serwisowej — poprzez kontener DI (rekomendacja: `dependency-injector`, patrz `09_Python_Architecture.md`) konfigurowany w warstwie kompozycji (Composition Root), oddzielnie dla środowisk (dev/test/prod) i dla testów jednostkowych (mock/stub connectorów).

1.5 Open/Closed jako zasada nadrzędna dla rozszerzalności platformowej

To wymaganie było explicite podniesione w briefie projektowym i jest traktowane jako **twardy wymóg architektoniczny, a nie sugestia**: żaden nowy typ platformy raportowej nie powinien wymagać zmiany kodu poza katalogiem `connectors/`. Mechanizm rejestracji connectorów opiera się na wzorcu **Registry + Factory**, konfigurowanym deklaratywnie (typ platformy → implementacja connectora), tak by dodanie wpisu konfiguracyjnego i nowej klasy było wystarczające.

2. Warstwy architektury (Clean Architecture)

PRESENTATION LAYER
Dashboard (Web GUI), REST API (FastAPI), Notification Output
APPLICATION LAYER (Use Cases / Orchestration)
Test Orchestrator, Scheduler, AI Analysis Engine (orkiestracja)
DOMAIN LAYER (rdzeń biznesowy, niezależny od frameworków)
Validation Engine, Business Rule Engine, Health/Confidence Score, encje domenowe: Report, ValidationProfile, TestResult
INFRASTRUCTURE LAYER (adaptery do świata zewnętrznego)
Report Platform Connectors (PBI/SSRS), Browser Automation Engine, Value Extraction Engine, SQL Validation Engine, Results/Configuration/Validation Profile Repository (Postgres), Notification Service, Audit & Logging, Monitoring

Reguła zależności: warstwa Domain nie importuje niczego z Infrastructure ani Presentation. Warstwa Application zależy od abstrakcji zdefiniowanych w Domain (porty), a konkretne implementacje (adaptery) dostarczane są przez Infrastructure i wstrzykiwane w warstwie kompozycji (`main.py` / bootstrap aplikacji). Jest to wariant architektury **Ports & Adapters (Hexagonal)** zagnieżdżony w Clean Architecture — Domain i Application definiują porty (interfejsy), Infrastructure implementuje adaptery.

3. Kluczowe wzorce projektowe

3.1 Connector Pattern (Report Platform Connector Layer)

Ilustracja koncepcyjna interfejsu – nie jest to kod produkcyjny do implementacji wprost

```
class IReportConnector(Protocol):
    async def check_availability(self, report_ref: ReportReference) -> AvailabilityResult: ...
```

```
async def open_report(self, report_ref: ReportReference) -> RenderedReportHandle: ...
async def list_catalog(self) -> list[ReportMetadata]: ...
```

Konkretne implementacje: PowerBICONNECTOR, SSRSConnector — obie implementują ten sam kontrakt, ale wewnętrznie korzystają z różnych mechanizmów niskopoziomowych opisanych w 00_Technical_Feasibility_Study.md (Browser Automation jako wspólna baza + odpowiednie API natywne jako wzbogacenie). Wybór connectora następuje na podstawie pola platform_type w Validation Profile, poprzez ConnectorFactory, bez logiki warunkowej rozproszonej po systemie.

3.2 Strategy + Chain of Responsibility (Value Extraction Engine)

Opisany szczegółowo w 00_Technical_Feasibility_Study.md, sekcja 7. Architektonicznie: IExtractionStrategy jako wspólny interfejs, ExtractionChain jako obiekt porządkujący sekwencję strategii per punkt ekstrakcji zdefiniowany w Validation Profile, z jednolitym obiektem wyniku ExtractedValue(value, confidence_score, method_used, raw_evidence).

3.3 Rule Engine Pattern (Business Rule Engine)

Reguły biznesowe (np. “suma kategorii = wartość całkowita ± tolerancja”, “wartość nie może być ujemna”, “wartość bieżąca odchyła się od średniej historycznej o więcej niż X%”) są modelowane jako **deklaratywne, konfigurowalne obiekty reguł** (IBusinessRule.evaluate(context) -> RuleResult), a nie zaszyty w kodzie warunek — pozwala to administratorowi definiować nowe reguły bez zmiany kodu aplikacji, poprzez konfigurację lub bibliotekę predefiniowanych typów reguł rozszerzaną w cyklu deweloperskim.

3.4 Dependency Injection / Composition Root

Cała aplikacja (worker wykonawczy, API, scheduler) posiada jeden punkt kompozycji zależności (Composition Root), w którym konkretne implementacje (repozytoria SQLAlchemy, connectory, kanały powiadomień) są wiązane z interfejsami domenowymi. Ułatwia to: - podmianę implementacji w testach (np. FakeReportConnector zamiast prawdziwego Playwright w testach jednostkowych Test Orchestratora), - niezależne skalowanie i wdrażanie komponentów (worker od API), - czytelność zależności dla nowego dewelopera dołączającego do zespołu.

3.5 Repository Pattern

Dostęp do danych trwałych (raporty, profile, wyniki testów) odbywa się wyłącznie przez interfejsy repozytoriów zdefiniowane w warstwie Domain (IReportRepository, IValidationProfileRepository, IResultsRepository), implementowane w Infrastructure przy użyciu SQLAlchemy. Domena nigdy nie zna szczegółów ORM.

3.6 Odporność i resilience (Circuit Breaker, Retry, Timeout)

Ze względu na zależność od zewnętrznych, czasem niestabilnych platform (Power BI, SSRS), warstwa Infrastructure stosuje wzorce odpornościowe: **Retry z backoff wykładniczym** dla przejściowych błędów sieciowych, **Circuit Breaker** wyłączający tymczasowo testy dla platformy zgłaszającej masowe awarie (aby nie “zalewać” niedziałającego serwera i nie generować fałszywych alarmów o pojedynczych raportach, gdy przyczyna leży w całej platformie), oraz jawne **Timeouty** na każdym poziomie (nawigacja, ekstrakcja, zapytanie SQL).

4. Główne moduły systemu i ich odpowiedzialności

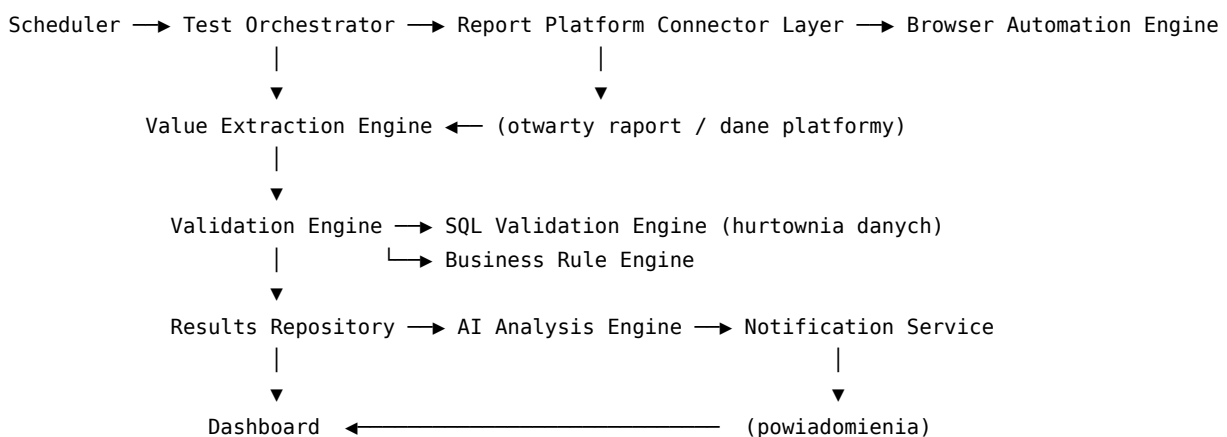
#	Moduł	Warstwa	Odpowiedzialność
1	Scheduler	Application	Harmonogramowanie wykonania testów zgodnie z częstotliwością zdefiniowaną per Validation Profile (cron-like), inicjowanie przebiegów wsadowych (batch) oraz uruchomień na żądanie (manual trigger z GUI). Nie wykonuje testów samodzielnie — deleguje do Test Orchestratora.
2	Test Orchestrator	Application	Koordinuje pełny cykl życia pojedynczego przebiegu testowego: pobranie Validation Profile → wybór connectora → otwarcie raportu → ekstrakcja wartości → walidacja → zapis wyniku → wyzwolenie analizy AI i powiadomień. Odpowiada za równoległość i kolejkovanie przebiegów (worker pool).
3	Configuration Repository	Infrastructure	Przechowuje konfigurację systemową i katalog raportów (metadane: adres, obszar biznesowy/zakładka, typ platformy, właściciel). Źródło prawdy dla inwentaryzacji raportów.
4	Validation Profile Repository	Infrastructure	Przechowuje i wersjonuje Validation Profiles — definicje testów, KPI, zapytań SQL, tolerancji, reguł biznesowych, harmonogramów, wraz z pełną historią zmian (audytowalność).
5	Test Definition Engine	Domain/Application	Udostępnia model i walidację poprawności definicji testu podczas jego tworzenia/edycji przez administratora (np. sprawdzenie spójności zapytania SQL, poprawności selektora KPI) — zapobiega zapisaniu niekompletnego/błédnego profilu.
6	Browser Automation Engine	Infrastructure	Cienka, reużywalna warstwa zarządzająca cyklem życia instancji przeglądarki (Playwright): pule kontekstów, zarządzanie sesją SSO, nawigacja, oczekiwanie na stabilizację renderowania, przechwytywanie zrzutów ekranu i logów sieciowych jako dowodów. Wykorzystywana przez connectory, nieeksponowana bezpośrednio na zewnątrz.

#	Moduł	Warstwa	Odpowiedzialność
7	Report Platform Connector Layer	Infrastructure	Zestaw connectorów (PowerBICconnector, SSRSConnector, ...) implementujących IReportConnector — jedyne miejsce w systemie znające specyfikę danej platformy raportowej.
8	Value Extraction Engine	Infrastructure/Domain	Odczyt wartości KPI z otwartego raportu przy użyciu strategii wielowarstwowego fallbacku (patrz 00_Technical_Feasibility_Study.md). Zwraca wartość wraz z Confidence Score i użytą metodą.
9	Validation Engine	Domain	Rdzeń deterministycznej oceny poprawności: porównuje wartości odczytane z raportu z wynikami SQL Validation Engine w granicach dopuszczalnych odchyleń zdefiniowanych w profilu; agreguje wyniki pojedynczych testów w wynik przebiegu. To jedyny moduł uprawniony do wydania werdyktu PASS/FAIL/WARNING.
10	SQL Validation Engine	Infrastructure	Bezpieczne (parametryzowane, z kontrolą uprawnień konta technicznego) wykonywanie zapytań SQL zdefiniowanych w Validation Profile na hurtowni danych i zwracanie wyników do Validation Engine.
11	Business Rule Engine	Domain	Wykonuje reguły biznesowe wykraczające poza proste porównanie dwóch liczb (sekcja 3.3).
12	Results Repository	Infrastructure	Trwały, zoptymalizowany pod odczyt historyczny magazyn wyników testów, Health Score i Confidence Score w czasie (patrz 07_Data_Model.md — strategia retencji/partycjonowania).
13	AI Analysis Engine	Application (usługa wspierająca)	Interpretuje wyniki dostarczone przez Validation Engine: wykrywa anomalie na tle historii, grupuje podobne błędy, wskazuje najbardziej prawdopodobne przyczyny, generuje czytelne uzasadnienia i sugestie działań naprawczych. Nigdy nie zmienia ani nie nadpisuje werdyktu Validation Engine (patrz 06_AI_Module.md).

#	Moduł	Warstwa	Odpowiedzialność
14	Dashboard	Presentation	Aplikacja webowa GUI — przegląd raportów, konfiguracja, historia, wyniki AI, Health/Confidence Score (patrz 04_Functional_Requirements.md).
15	Notification Service	Infrastructure	Dystrybucja powiadomień o wykrytych problemach do zdefiniowanych odbiorców/kanałów (e-mail, integracja z systemem powiadomień organizacji), z regułami deduplikacji i eskalacji.
16	Audit & Logging	Cross-cutting/ Infrastructure	Rejestruje wszystkie zdarzenia istotne z punktu widzenia audytu: zmiany konfiguracji/profilu, wykonania testów, decyzje systemu, działania użytkowników GUI/API. Niezależny, tylko-do-zapisu (append-only) log audytowy odrębny od logów technicznych.
17	Monitoring	Cross-cutting/ Infrastructure	Metryki operacyjne systemu (czas wykonania testów, obciążenie worker pool, wskaźniki błędów connectorów, dostępność platform BI) eksponowane do Prometheus/Grafana; integralna część NFR (patrz 05_NonFunctional_Requirements.md).

4.1 Diagram zależności modułów (skrót koncepcyjny)

Pełny, formalny diagram komponentów z relacjami — patrz `uml/ComponentDiagram.puml`. Poniżej skrót tekstowy głównego przepływu zależności:



Wszystkie moduły → Audit & Logging, Monitoring (cross-cutting)

Configuration / Validation Profile Repository ↔ Test Definition Engine, Test Orchestrator, Dashboard

5. Zagadnienia przekrojowe (Cross-Cutting Concerns)

5.1 Bezpieczeństwo

- Konto techniczne o **najmniejszym niezbędnym zakresie uprawnień** (least privilege) do Platformy BI i hurtowni danych, zarządzane zgodnie z polityką zarządzania sekretami organizacji (patrz 10_Deployment.md).
- Autoryzacja i autentykacja użytkowników GUI/API oparta o istniejący mechanizm SSO organizacji (Windows Authentication / Azure AD / AD FS — do potwierdzenia z zespołem IAM), z modelem RBAC (role: Administrator, Analityk/Właściciel testu, Przeglądający).
- Pełny szczegół — patrz 05_NonFunctional_Requirements.md, sekcja Bezpieczeństwo.

5.2 Konfiguracja

Konfiguracja aplikacji zarządzana zgodnie z zasadą **12-Factor App** — parametry środowiskowe (adresy platform, connection stringi, sekrety) wstrzykiwane przez zmienne środowiskowe / zarządzany magazyn sekretów, nigdy hardkodowane; walidowane przy starcie aplikacji (Pydantic Settings — patrz 09_Python_Architecture.md).

5.3 Logowanie i audyt

Rozdzielone strumienie: **logi techniczne** (poziom DEBUG/INFO/WARNING/ERROR, do celów operacyjnych i debugowania, skorelowane przez trace_id zgodnie z OpenTelemetry) oraz **log audytowy** (append-only, dotyczący zdarzeń biznesowych: kto zmienił Validation Profile, kto uruchomił test ręcznie, jaki był werdykt) — ten drugi podlega odrębnym wymogom retencji zgodnym z polityką organizacji.

5.4 Obserwowalność (Observability)

Trzy filary: **metryki** (Prometheus/Grafana), **logi** (ustrukturyzowane, JSON, agregowane centralnie), **trace’y rozproszone** (OpenTelemetry) — pozwalające prześledzić pojedynczy przebieg testowy przez wszystkie moduły, co jest kluczowe przy debugowaniu problemów wydajnościowych na skalę 1000 raportów.

6. Rozszerzalność — scenariusz “nowa platforma raportowa”

Aby zilustrować zgodność z zasadą Open/Closed, poniżej przedstawiono pełną listę zmian wymaganych do dodania obsługi nowej, hipotetycznej platformy raportowej (np. Tableau) w przyszłości:

1. Implementacja nowej klasy TableauConnector zgodnej z IReportConnector.
2. Implementacja (opcjonalnie, jeśli potrzebna specyficzna strategia) nowej klasy IExtractionStrategy dla specyfiki renderowania Tableau.
3. Rejestracja nowego typu platformy w ConnectorFactory (jedna linia konfiguracji/rejestracji).
4. Dodanie wartości TABLEAU do enumeracji typu platformy w modelu domenowym (platform_type).

Żadna z powyższych zmian nie ingeruje w Test Orchestrator, Validation Engine, Business Rule Engine, Results Repository, Dashboard ani AI Analysis Engine — potwierdzając, że koszt krańcowy dodania nowej platformy jest liniowo mały i izolowany.

7. Powiązanie z dalszą dokumentacją

- Szczegółowe uzasadnienie wyboru technologii integracji — 00_Technical_Feasibility_Study.md.
- Model danych wspierający powyższą architekturę — 07_Data_Model.md.
- Kontrakty API pomiędzy warstwą Presentation a Application — 08_API.md.
- Konkretna implementacja w Pythonie (struktura katalogów, biblioteki) — 09_Python_Architecture.md.
- Diagram komponentów i diagram wdrożenia — uml/ComponentDiagram.puml, uml/DeploymentDiagram.puml.

04. Functional Requirements

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Wymagania funkcjonalne, przypadki użycia, wymagania GUI

Odbiorcy: Zespół Deweloperski, Analitycy biznesowi, QA

1. Aktorzy systemu

Aktor	Opis
Administrator systemu	Zarządza konfiguracją globalną, kontami technicznymi, uprawnieniami (RBAC), harmonogramami, katalogiem platform.
Analityk / Właściciel raportu	Definiuje i utrzymuje Validation Profiles dla przypisanych raportów, przegląda wyniki i analizy AI dla swoich obszarów.
Przeglądający (Viewer)	Przegląda dashboard, historię, Health Score — bez prawa edycji konfiguracji.
System (Scheduler)	Aktor niebędący człowiekiem — automatycznie inicjuje wykonania testów zgodnie z harmonogramem.
AI Analysis Engine	Aktor wspierający — dostarcza interpretacje i sugestie w ramach zdefiniowanych granic (patrz 06_AI_Module.md).

2. Lista przypadków użycia

Pełny diagram graficzny — patrz `uml/UseCase.puml`. Poniżej lista referencyjna z ID używanym w wymaganiach funkcjonalnych.

ID	Nazwa przypadku użycia	Aktor główny
UC-01	Przegląd katalogu raportów wg obszarów biznesowych	Przeglądający, Analityk, Administrator
UC-02	Wyszukiwanie raportu	Przeglądający, Analityk, Administrator
UC-03	Definiowanie Validation Profile dla raportu	Analityk, Administrator
UC-04	Definiowanie testu SQL	Analityk, Administrator
UC-05	Definiowanie testu wartości pola (KPI)	Analityk, Administrator
UC-06	Definiowanie reguły biznesowej	Analityk, Administrator
UC-07	Ustawienie harmonogramu wykonania testów	Analityk, Administrator
UC-08	Ręczne uruchomienie testu / grupy testów	Analityk, Administrator
UC-09	Automatyczne wykonanie testów wg harmonogramu	System (Scheduler)
UC-10	Przegląd wyniku pojedynczego przebiegu testowego	Przeglądający, Analityk, Administrator

ID	Nazwa przypadku użycia	Aktor główny
UC-11	Przegląd historii wyników raportu	Przeglądający, Analityk, Administrator
UC-12	Przegląd Health Score raportu/ obszaru	Przeglądający, Analityk, Administrator
UC-13	Przegląd Confidence Score odczytu	Analityk, Administrator
UC-14	Filtrowanie i przegląd błędów	Przeglądający, Analityk, Administrator
UC-15	Przegląd analizy i wyjaśnienia wygenerowanego przez AI	Przeglądający, Analityk, Administrator
UC-16	Otrzymanie powiadomienia o wykrytym problemie	Analityk, Administrator
UC-17	Zarządzanie użytkownikami i rolami (RBAC)	Administrator
UC-18	Zarządzanie katalogiem platform / connectorów	Administrator
UC-19	Przegląd logu audytowego	Administrator
UC-20	Przegląd metryk operacyjnych systemu (Monitoring)	Administrator

3. Wymagania funkcjonalne

Priorytety wg MoSCoW: **M**ust have / **S**hould have / **C**ould have / **W**on't have (na tym etapie).

3.1 Katalog raportów i nawigacja (powiązane UC-01, UC-02)

ID	Wymaganie	Priorytet
FR-001	System musi prezentować raporty pogrupowane według obszarów biznesowych (zakładek), odzwierciedlając strukturę istniejącej Platformy BI.	M
FR-002	System musi umożliwiać wyszukiwanie raportu po nazwie, adresie, obszarze biznesowym i typie platformy.	M
FR-003	System musi rozróżniać i jednoznacznie oznaczać typ platformy raportu (Power BI / SSRS) w każdym miejscu prezentacji.	M
FR-004	System powinien umożliwiać filtrowanie katalogu wg statusu ostatniego testu (OK / Błąd / Ostrzeżenie / Brak testów).	S

3.2 Definiowanie testów i Validation Profile (powiązane UC-03-UC-07)

ID	Wymaganie	Priorytet
FR-010	System musi umożliwiać utworzenie Validation Profile dla dowolnego raportu z katalogu, zawierającego: adres, typ platformy, zestaw testów, sposób odnajdywania KPI, zapytania SQL, dopuszczalne odchylenia, reguły biznesowe, częstotliwość.	M

ID	Wymaganie	Priorytet
FR-011	System musi wymuszać, że raport bez zdefiniowanego co najmniej jednego testu nie jest objęty automatycznym harmonogramem (zgodnie z założeniem projektowym — testy muszą być zdefiniowane).	M
FR-012	System musi umożliwiać definiowanie testów typu SQL (zapytanie do hurtowni + oczekiwany wynik/zakres).	M
FR-013	System musi umożliwiać definiowanie testów typu "wartość pola raportu" (wskazanie elementu KPI + oczekiwana wartość lub reguła porównania z SQL).	M
FR-014	System musi umożliwiać zdefiniowanie dopuszczalnego odchylenia (tolerancji) dla porównań liczbowych (wartość bezwzględna i/ lub procentowa).	M
FR-015	System musi umożliwiać definiowanie reguł biznesowych wykraczających poza proste porównanie (np. sumy kontrolne, zakresy, trend historyczny).	M
FR-016	System musi umożliwiać administratorowi aktualizację (wersjonowaną) istniejącego Validation Profile, z zachowaniem historii poprzednich wersji.	M
FR-017	System musi umożliwiać ustawienie indywidualnej częstotliwości wykonania testów per raport (np. co godzinę, dziennie, tygodniowo, na żądanie).	M
FR-018	System powinien umożliwiać kopiowanie/klonowanie Validation Profile jako punkt startowy dla podobnego raportu.	S
FR-019	AI Analysis Engine może zaproponować wstępną konfigurację Validation Profile (np. wykryte kandydackie elementy KPI) — wymaga akceptacji człowieka przed aktywacją (patrz 06_AI_Module.md).	C

3.3 Wykonanie testów (powiązane UC-08, UC-09)

ID	Wymaganie	Priorytet
FR-020	System musi automatycznie wykonywać testy zgodnie z harmonogramem zdefiniowanym w Validation Profile, bez interakcji użytkownika.	M
FR-021	System musi umożliwiać ręczne uruchomienie testu dla pojedynczego raportu na żądanie użytkownika uprawnionego.	M
FR-022	System musi umożliwiać ręczne uruchomienie testów dla grupy raportów (np. cały obszar biznesowy) na żądanie.	S

ID	Wymaganie	Priorytet
FR-023	System musi wykonywać testy równoległe (worker pool), z konfigurowalnym limitem współbieżności, aby zapewnić realizację pełnego cyklu dla 1000 raportów w oknie czasowym określonym w 05_NonFunctional_Requirements.md.	M
FR-024	System musi rejestrować pełny cykl życia przebiegu testowego (start, poszczególne etapy, zakończenie, czas trwania) niezależnie od jego wyniku (w tym w przypadku awarii/timeoutu).	M

3.4 Wyniki, historia i metryki (powiązane UC-10-UC-14)

ID	Wymaganie	Priorytet
FR-030	System musi prezentować wynik każdego przebiegu testowego ze statusem (PASS / FAIL / WARNING / ERROR technicznym) i szczegółami (wartości porównywane, odchylenie, użyta metoda ekstrakcji).	M
FR-031	System musi przechowywać pełną historię wyników dla każdego raportu, przeszukiwalną wg zakresu dat.	M
FR-032	System musi wyliczać i prezentować Health Score — zagregowany wskaźnik jakości raportu w czasie (per raport, per obszar biznesowy, globalnie).	M
FR-033	System musi wyliczać i prezentować Confidence Score dla każdej odczytanej wartości, informujący o pewności metody ekstrakcji.	M
FR-034	System musi umożliwiać filtrowanie listy błędów wg obszaru biznesowego, platformy, typu błędu, zakresu dat, statusu.	M
FR-035	System powinien umożliwiać eksport wyników (np. CSV) do celów raportowania zarządczego poza systemem.	S

3.5 AI i wsparcie decyzyjne (powiązane UC-15)

ID	Wymaganie	Priorytet
FR-040	System musi prezentować wygenerowane przez AI wyjaśnienie każdego wykrytego błędu/anomalii w języku naturalnym, wraz z uzasadnieniem opartym na konkretnych danych (nigdy gołosłowne).	M
FR-041	System musi grupować/klastrować podobne błędy w celu redukcji szumu informacyjnego (np. "17 raportów w obszarze Kredyty wskazuje ten sam wzorzec błędu").	S

ID	Wymaganie	Priorytet
FR-042	System musi jednoznacznie oznaczać w GUI, że dana treść pochodzi od AI (interpretacja wspierająca), odróżniając ją wizualnie od deterministycznego werdyktu Validation Engine.	M
FR-043	System nie może pozwolić AI na zmianę statusu PASS/FAIL wygenerowanego przez Validation Engine — wymaganie o charakterze ograniczającym (guardrail), zweryfikowane testami akceptacyjnymi.	M

3.6 Powiadomienia (powiązane UC-16)

ID	Wymaganie	Priorytet
FR-050	System musi wysyłać powiadomienia do wskazanych odbiorców po wykryciu błędu/rozbieżności zgodnie z regułami zdefiniowanymi w Validation Profile lub globalnie.	M
FR-051	System musi umożliwiać deduplikację powiadomień (np. brak ponownego alertu o tym samym, wciąż trwającym problemie w każdym cyklu harmonogramu).	S
FR-052	System powinien umożliwiać konfigurację progu eskalacji (np. powiadomienie do przełożonego po N nieudanych cyklach z rzędu).	C

3.7 Administracja, bezpieczeństwo i audyt (powiązane UC-17-UC-20)

ID	Wymaganie	Priorytet
FR-060	System musi wspierać model RBAC z co najmniej rolami: Administrator, Analityk/Właściciel, Przeglądający.	M
FR-061	System musi rejestrować w logu audytowym każdą zmianę konfiguracji/Validation Profile wraz z autorem i znacznikiem czasu.	M
FR-062	System musi umożliwiać administratorowi przegląd logu audytowego z poziomu GUI.	M
FR-063	System musi umożliwiać administratorowi zarządzanie listą platform/connectorów oraz podstawowymi parametrami połączeń (bez ekspozycji sekretów w GUI).	M
FR-064	System musi eksponować metryki operacyjne (stan worker pool, czas wykonania, wskaźnik błędów) dostępne dla zespołu utrzymania.	M

4. Wymagania GUI — opis ekranów

Poniższy opis stanowi wymagania funkcjonalne warstwy prezentacji na poziomie koncepcyjnym (nie jest to projekt graficzny/UI design — ten powstaje w odrębnym etapie projektowym)

z zespołem UX).

4.1 Ekran główny — Dashboard

- Podsumowanie stanu całego katalogu: liczba raportów OK / z błędem / z ostrzeżeniem / bez zdefiniowanych testów.
- Zagregowany Health Score globalny i per obszar biznesowy (wizualizacja trendu w czasie).
- Lista/kafelki najnowszych wykrytych problemów wymagających uwagi, posortowane wg istotności (z udziałem priorytetyzacji AI — patrz 06_AI_Module.md).
- Skróty nawigacyjne do Katalogu Raportów, Historii, Konfiguracji.

4.2 Katalog Raportów (Report Catalog)

- Widok listy/siatki raportów, grupowanie wg obszarów biznesowych (zakładek) odzwierciedlających strukturę Platformy BI.
- Filtrowanie: platforma (PBI/SSRS), status ostatniego testu, obszar biznesowy, właściciel.
- Wyszukiwarka pełnotekstowa (nazwa, adres, tagi).
- Widok szczegółu raportu: metadane, aktualny Validation Profile, ostatnie wyniki, Health Score, link do historii.

4.3 Konfiguracja Validation Profile

- Formularz definicji: dane raportu (readonly, z katalogu), lista testów (SQL / wartość pola), harmonogram, tolerancje, reguły biznesowe.
- Edytor testu SQL z podglądem/walidacją składni przed zapisem (Test Definition Engine).
- Edytor testu wartości pola — wskazanie elementu KPI (wspierany podglądem raportu / listą wykrytych elementów przez Value Extraction Engine w trybie podglądu).
- Historia wersji profilu z możliwością porównania (diff) i przywrócenia poprzedniej wersji.

4.4 Uruchamianie testów

- Możliwość ręcznego uruchomienia testu dla pojedynczego raportu lub grupy (obszar biznesowy / wybór wielokrotny).
- Podgląd statusu w toku (live/near-live), z możliwością odświeżenia.

4.5 Historia i wyniki

- Chronologiczna lista przebiegów testowych per raport, z możliwością rozwinięcia szczegółów (porównywane wartości, użyta metoda ekstrakcji, Confidence Score).
- Wykres trendu Health Score w czasie.
- Eksport danych historycznych.

4.6 Analiza AI (AI Insights)

- Lista wygenerowanych przez AI wyjaśnień/anomalii, wyraźnie oznaczonych jako treść wspierająca (nie decyzyjna).
- Grupowanie podobnych błędów (klastry) z rekomendowanymi działaniami.
- Możliwość oznaczenia przez użytkownika trafności sugestii AI (feedback loop do dalszego doskonalenia promptów/modelu — patrz 06_AI_Module.md).

4.7 Panel administracyjny

- Zarządzanie użytkownikami i rolami.
- Zarządzanie katalogiem platform/connectorów.
- Podgląd logu audytowego z filtrowaniem.

- Podgląd metryk operacyjnych (link/embed do Grafany lub widok uproszczony wewnątrz aplikacji).
-

5. Powiązanie z dalszą dokumentacją

- Model danych wspierający powyższe funkcje — `07_Data_Model.md`.
- Kontrakty API realizujące powyższe przypadki użycia — `08_API.md`.
- Diagramy UML (Use Case, Activity, Sequence) — katalog `uml/`.
- Wymagania niefunkcjonalne (wydajność wykonania testów przy skali 1000 raportów) — `05_NonFunctional_Requirements.md`.

05. Non-Functional Requirements

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Wymagania нефункционалне

Odbiorcy: Architektura IT, Security Office, Zespół Deweloperski, Zespół Utrzymania

1. Wydajność (Performance)

ID	Wymaganie	Kryterium akceptacji
NFR-P-01	Pełny cykl testowy dla całego katalogu raportów (docelowo 1000) musi zakończyć się w oknie czasowym akceptowalnym operacyjnie.	≤ 4 godziny dla 1000 raportów przy standardowej konfiguracji worker pool (do doprecyzowania liczby workerów w PoC — patrz 11_Roadmap.md, Faza 0).
NFR-P-02	Pojedynczy przebieg testowy (otwarcie raportu + ekstrakcja + walidacja SQL) nie powinien przekraczać rozsądnego czasu jednostkowego.	Mediana ≤ 15 s dla raportu o przeciętnej złożoności (Power BI), ≤ 10 s (SSRS); wartości referencyjne do potwierdzenia w PoC.
NFR-P-03	System musi wykonywać testy równoległe, ze skalowalną liczbą workerów.	Konfigurowalny worker pool, skalowalny horyzontalnie (patrz 10_Deployment.md).
NFR-P-04	Zapytania do hurtowni danych (SQL Validation Engine) muszą być zoptymalizowane i nie obciążać nadmiernie hurtowni.	Zapytania wykonywane z jasno zdefiniowanym timeoutem, monitorowane pod kątem czasu wykonania; rekomendacja: dedykowana pula połączeń o ograniczonym limicie równoległości do hurtowni.
NFR-P-05	Dashboard musi ładować podsumowanie stanu katalogu (1000 raportów) w czasie akceptowalnym dla użytkownika.	Czas ładowania widoku głównego ≤ 2 s przy typowym obciążeniu (dzięki zdenormalizowanym/zagregowanym widokom, patrz 07_Data_Model.md).

2. Skalowalność (Scalability)

ID	Wymaganie
NFR-S-01	Architektura musi wspierać wzrost katalogu z ok. 300 do ≥ 1000 raportów bez zmian architektonicznych — wyłącznie przez skalowanie horyzontalne warstwy wykonawczej (workery Browser Automation).
NFR-S-02	Warstwa API i baza danych muszą wspierać skalowanie horyzontalne (stateless API, connection pooling do PostgreSQL, możliwość read-replik dla obciążeń analitycznych/dashboardowych w przyszłości).

ID	Wymaganie
NFR-S-03	Kolejka zadań testowych (Scheduler → Test Orchestrator → workery) musi być odporna na skoki obciążenia (np. cały katalog uruchomiony ręcznie jednocześnie) poprzez mechanizm kolejkowania z limitem współbieżności (Celery/Redis lub RabbitMQ — patrz 09_Python_Architecture.md).
NFR-S-04	Model danych musi wspierać rosnący wolumen historii wyników (1000 raportów × wiele przebiegów dziennie × wieloletnia retencja) bez degradacji wydajności zapytań — strategia partycjonowania opisana w 07_Data_Model.md.

3. Bezpieczeństwo (Security)

System jest projektowany zgodnie z podejściem **Security by Design**, adekwatnym do środowiska instytucji finansowej.

ID	Wymaganie
NFR-SEC-01	Konto techniczne używane przez ARGUS do dostępu do Platformy BI i hurtowni danych musi działać zgodnie z zasadą najmniejszych uprawnień (least privilege) — wyłącznie odczyt raportów/danych niezbędnych do walidacji, bez uprawnień administracyjnych na platformie BI.
NFR-SEC-02	Sekrety (dane uwierzytelniające konta technicznego, connection stringi, klucze API) muszą być przechowywane w dedykowanym, zarządzanym magazynie sekretów (np. HashiCorp Vault / Azure Key Vault / rozwiązanie wewnętrzne banku), nigdy w kodzie, plikach konfiguracyjnych repozytorium ani zmiennych środowiskowych w postaci jawnej w obrazach kontenerów.
NFR-SEC-03	Dostęp użytkowników do GUI/API musi być uwierzytelniany przez istniejący mechanizm SSO organizacji, z autoryzacją opartą o RBAC (role: Administrator, Analityk/Właściciel, Przeglądający).
NFR-SEC-04	Cała komunikacja sieciowa (GUI ↔ API, API ↔ workery, workery ↔ Platforma BI/hurtownia) musi być szyfrowana (TLS) zgodnie z polityką organizacji.
NFR-SEC-05	Dane wrażliwe w logach technicznych muszą być maskowane (np. nie logujemy pełnych wyników zapytań SQL zawierających dane biznesowe, o ile nie jest to jawnie wymagane dla celów audytowych w kontrolowanym, ograniczonym dostępowo repozytorium).
NFR-SEC-06	System musi podlegać standardowym procesom bezpieczeństwa organizacji przed wdrożeniem produkcyjnym: testy podatności / testy penetracyjne, przegląd kodu pod kątem bezpieczeństwa (SAST), skanowanie zależności (SCA) w pipeline CI/CD.
NFR-SEC-07	Wszelkie treści generowane przez moduł AI wysyłane do zewnętrznego dostawcy modelu (jeśli wybrany wariant chmurowy — patrz 06_AI_Module.md) muszą być pozbawione danych osobowych i danych sklasyfikowanych jako wrażliwe zgodnie z polityką klasyfikacji danych organizacji; rekomendowany wariant Azure OpenAI w ramach dzierżawy (tenant) organizacji z kontraktowym brakiem wykorzystania danych do trenowania modeli, alternatywnie hostowany model open-weight w infrastrukturze wewnętrznej — decyzja do podjęcia z Security Office w Fazie 0.
NFR-SEC-08	System musi wspierać segmentację sieciową — komponenty wykonawcze (workery Browser Automation) uruchamiane w segmencie sieci z kontrolowanym dostępem wyłącznie do niezbędnych zasobów (Platforma BI, hurtownia danych), zgodnie z zasadą defense-in-depth.

4. Dostępność i niezawodność (Availability & Reliability)

ID	Wymaganie
NFR-A-01	Docelowa dostępność systemu (API + GUI) w godzinach pracy organizacji: ≥ 99.5% .
NFR-A-02	Awaria pojedynczego workera (Browser Automation) nie może wpływać na dostępność pozostałych przebiegów testowych — izolacja procesów wykonawczych.
NFR-A-03	System musi implementować mechanizmy odpornościowe (Retry z backoff, Circuit Breaker, Timeout) dla wszystkich integracji zewnętrznych (Platforma BI, hurtownia danych) — patrz 03_Architecture.md, sekcja 3.6.
NFR-A-04	Chwilowa niedostępność Platformy BI lub hurtowni danych musi skutkować jednoznacznym, odróżnialnym statusem technicznym (np. PLATFORM_UNAVAILABLE), a nie fałszywym wynikiem FAIL sugerującym błąd raportu.
NFR-A-05	System musi posiadać zdefiniowaną strategię Disaster Recovery (kopie zapasowe bazy danych, RPO/RTO — do sprecyzowania z zespołem infrastruktury, patrz 10_Deployment.md).

5. Utrzymywalność (Maintainability)

ID	Wymaganie
NFR-M-01	Kod musi być zgodny z zasadami SOLID/Clean Architecture opisanymi w 03_Architecture.md, zweryfikowanymi w code review.
NFR-M-02	Pokrycie testami jednostkowymi warstwy domenowej (Validation Engine, Business Rule Engine) — cel: ≥ 85% .
NFR-M-03	Każdy connector platformy raportowej musi posiadać dedykowany zestaw testów kontraktowych (contract tests) weryfikujących zgodność z interfejsem IReportConnector.
NFR-M-04	Statyczna analiza kodu (Ruff, MyPy w trybie strict) musi być częścią pipeline CI, blokującą merge w przypadku naruszeń.
NFR-M-05	Dokumentacja architektoniczna (niniejsze repozytorium) musi być aktualizowana równolegle ze zmianami architektonicznymi jako część definition of done dla zadań o charakterze architektonicznym.

6. Obserwowalność (Observability)

ID	Wymaganie
NFR-O-01	System musi eksponować metryki operacyjne w formacie kompatybilnym z Prometheus (czas wykonania testów, liczba sukcesów/błędów per connector, stan worker pool, opóźnienia kolejki).
NFR-O-02	System musi udostępniać dashboardy operacyjne w Grafanie dla zespołu utrzymania (odrębne od Dashboardu biznesowego dla użytkowników — patrz 04_Functional_Requirements.md).
NFR-O-03	System musi implementować rozproszone śledzenie (distributed tracing) zgodne z OpenTelemetry, umożliwiające prześledzenie pojedynczego przebiegu testowego przez wszystkie moduły.
NFR-O-04	Logi muszą być ustrukturyzowane (JSON) i zawierać trace_id/correlation_id umożliwiające korelację zdarzeń między modułami/workerami.

7. Zgodność i audytowalność (Compliance)

ID	Wymaganie
NFR-C-01	Każda zmiana Validation Profile musi być rejestrowana w logu audytowym z identyfikacją autora, znacznikiem czasu i pełną treścią zmiany (before/after).
NFR-C-02	Wyniki testów muszą być przechowywane w sposób niemodyfikowalny (append-only) przez okres retencji zgodny z polityką organizacji (do potwierdzenia z Data Governance — rekomendacja wyjściowa: minimum 24 miesiące, patrz 07_Data_Model.md).
NFR-C-03	System musi umożliwiać wygenerowanie raportu audytowego (kto, kiedy, co zmienił/uruchomił) na żądanie audytora wewnętrznego.
NFR-C-04	Decyzje modułu AI muszą być logowane wraz z uzasadnieniem i danymi wejściowymi, na których się opierały, dla celów wyjaśnialności (explainability) i audytu — patrz 06_AI_Module.md.

8. Użyteczność (Usability)

ID	Wymaganie
NFR-U-01	GUI musi być dostępne bez konieczności instalacji dodatkowego oprogramowania po stronie użytkownika (standardowa przeglądarka wspierana w organizacji).
NFR-U-02	Interfejs musi jednoznacznie odróżniać wizualnie treści generowane deterministycznie (wynik Validation Engine) od treści generowanych przez AI (interpretacja wspierająca) — wymaganie wynikające wprost z zasady nadrzędnej projektu.
NFR-U-03	GUI powinno być responsywne (wsparcie dla typowych rozdzielczości stacji roboczych organizacji); wsparcie mobilne nie jest wymagane w MVP.

9. Przenośność (Portability)

ID	Wymaganie
NFR-PO-01	Komponenty systemu muszą być konteneryzowane (Docker), umożliwiając uruchomienie w różnych środowiskach (Dev/Test/UAT/Prod) bez zmian w kodzie — wyłącznie przez konfigurację.
NFR-PO-02	System nie może zakładać twardej zależności od konkretnego dostawcy chmury — komponenty rdzeniowe (API, baza danych, workery) muszą działać on-prem zgodnie z założeniem projektowym; wyjątkiem jest opcjonalny, kontraktowo kontrolowany dostawca modelu AI (patrz NFR-SEC-07).

10. Podsumowanie — macierz wymagań krytycznych

Wymiar	Wymóg krytyczny	Ryzyko przy niespełnieniu
Wydajność	Pełny cykl 1000 raportów w akceptowalnym oknie czasowym	Brak realnej wartości biznesowej — system “nie nadąży” za katalogiem
Bezpieczeństwo	Least privilege konta technicznego + zarządzany magazyn sekretów	Brak akceptacji Security Office, ryzyko incydentu bezpieczeństwa

Wymiar	Wymóg krytyczny	Ryzyko przy niespełnieniu
Niezawodność	Odróżnienie awarii platformy od błędu raportu	Fałszywe alarmy, utrata zaufania biznesu do systemu
Audytowalność	Pełny, niemodyfikowalny ślad audytowy	Brak zgodności z wymogami compliance instytucji finansowej
Wyjaśnialność AI	Logowanie uzasadnień decyzji AI	Brak akceptacji dla wykorzystania AI w procesie decyzyjnym

Powiązanie: szczegóły infrastruktury realizującej powyższe wymagania — 10_Deployment.md; rejestr ryzyk związanych z niespełnieniem powyższych wymogów — 12_Risks.md.

06. AI Module

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform

Dokument: Rola, granice i architektura modułu AI

Odbiorcy: Architektura IT, Security Office, Compliance, Zespół Deweloperski

1. Zasada nadrzędna

AI nie podejmuje decyzji o poprawności raportu. Za poprawność odpowiada wyłącznie deterministyczny silnik walidacji (Validation Engine) oparty na jawnie zdefiniowanych testach.

Jest to najważniejsza zasada projektowa dotycząca modułu AI i jest traktowana jako **twardy wymóg architektoniczny (guardrail)**, weryfikowany technicznie (rozdzielenie odpowiedzialności w kodzie — AI Analysis Engine nie ma technicznej możliwości zapisu statusu PASS/FAIL) oraz proceduralnie (testy akceptacyjne, przegląd bezpieczeństwa).

Powód tej zasady jest fundamentalny dla kontekstu bankowego: modele językowe mogą halucynować, są niedeterministyczne i nie dają twardych gwarancji poprawności. Powierzenie im ostatecznej decyzji o poprawności danych finansowych byłoby nieakceptowalnym ryzykiem regulacyjnym i operacyjnym. AI dostarcza **wartości interpretacyjnej i przyspieszającej pracę człowieka**, nigdy nie zastępuje kontrolowanego, testowalnego procesu walidacji.

2. Rola AI — co AI może robić

Funkcja	Opis
Interpretacja wyników testów	Tłumaczenie technicznego wyniku Validation Engine (np. "KPI_SPRZEDAZ_TOTAL: raport=1 204 500,00, SQL=1 189 300,00, odchylenie=1.28%, próg=1.0%") na czytelne, biznesowe wyjaśnienie w języku naturalnym.
Wykrywanie anomalii na tle historii	Analiza trendu Health Score/wyników w czasie w celu wskazania nietypowych wzorców (np. "wartość KPI dla tego raportu odchyła się drugi dzień z rzędu, mimo że mieści się formalnie w tolerancji — warto zwrócić uwagę").
Wskazywanie najbardziej prawdopodobnych przyczyn	Na podstawie kontekstu (typ błędu, historia, powiązane raporty w tym samym obszarze/źródle danych) AI proponuje hipotezy przyczyny (np. "opóźnienie odświeżenia datasetu", "zmiana definicji miary", "problem z hurtownią danych widoczny również w innych raportach obszaru Kredyty").
Grupowanie podobnych błędów	Klastrowanie wielu jednostkowych alertów w jeden zrozumiały wzorec (np. "12 raportów w obszarze Sprzedaż wykazuje ten sam wzorec błędów — prawdopodobna wspólna przyczyna źródłowa"), redukując szum informacyjny dla analityka.

Funkcja	Opis
Generowanie czytelnych raportów dla użytkowników	Streszczenia okresowe (np. tygodniowe podsumowanie stanu jakości katalogu raportów) w języku naturalnym, na bazie zagregowanych, twardych danych z Results Repository.
Proponowanie działań naprawczych	Sugestie kolejnych kroków dla analityka/administradora (np. “sprawdź harmonogram odświeżania datasetu X”, “porównaj z raportem Y, który wykazuje tę samą anomalię”) — sugestie, nie polecenia wykonywane automatycznie.
Wspieranie konfiguracji (opcjonalnie, Could-have)	Propozycja wstępnej konfiguracji Validation Profile na podstawie analizy struktury raportu — zawsze wymaga świadomej akceptacji przez człowieka przed aktywacją (patrz 04_Functional_Requirements.md, FR-019).

3. Czego AI nie może robić

1. **Nie ustala statusu PASS/FAIL/WARNING** — ten status jest wyłączną kompetencją Validation Engine, obliczaną deterministycznie na podstawie zdefiniowanych testów i progów tolerancji.
2. **Nie modyfikuje wyników testów ani danych źródłowych.**
3. **Nie uruchamia automatycznie działań naprawczych** bez udziału i akceptacji człowieka (brak trybu “autonomicznej naprawy”).
4. **Nie zastępuje testu SQL ani reguły biznesowej** — nie “ocenia” poprawności liczby na podstawie własnej wiedzy ogólnej, wyłącznie interpretuje wynik już wyliczony deterministycznie.
5. **Nie ma dostępu zapisu** do Validation Profile, Results Repository czy statusu raportu — z perspektywy uprawnień systemowych AI Analysis Engine posiada **wyłącznie prawo odczytu** danych historycznych/wyników i prawo zapisu do odrębnej, oznaczonej przestrzeni “AI Insights” (nigdy do encji wyniku testu).

4. Wymóg wyjaśnialności (Explainability)

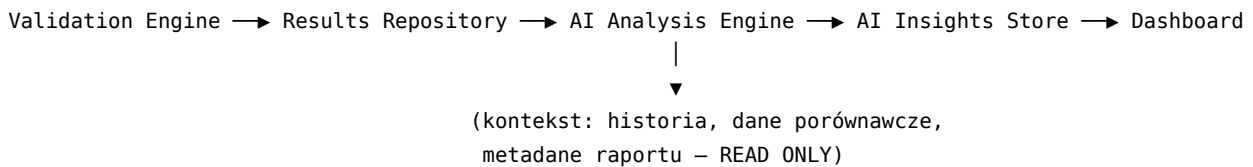
Każda wygenerowana przez AI interpretacja musi zawierać jawne uzasadnienie odwołujące się do konkretnych, sprawdzalnych danych (numer testu, wartości porównywane, historia, powiązane raporty) — nigdy ogólnikowe stwierdzenie bez podstawy.

Realizacja techniczna: - Prompt systemowy AI Analysis Engine wymusza strukturalną odpowiedź zawierającą sekcję evidence (dane źródłowe, na których oparto wyjaśnienie) obok sekcji explanation (treść dla użytkownika) — odpowiedź bez evidence jest odrzucana przez warstwę walidacji odpowiedzi AI przed wyświetleniem użytkownikowi. - Każda interpretacja AI jest logowana wraz z pełnym kontekstem wejściowym (dane wejściowe przekazane do modelu, wersja promptu, wersja modelu) w celu audytu i możliwości odtworzenia/wyjaśnienia decyzji post factum (NFR-C-04 w 05_NonFunctional_Requirements.md). - GUI jednoznacznie oznacza treści AI wizualnie (odrębny styl/etykieta “Wygenerowane przez AI — treść wspierająca”) — patrz 04_Functional_Requirements.md, NFR-U-02.

5. Architektura modułu AI

5.1 Umiejscowienie w architekturze

AI Analysis Engine działa jako usługa **downstream** względem Validation Engine i Results Repository — konsumuje już zakończone, deterministyczne wyniki testów (nigdy nie jest wywoływany w ścieżce krytycznej samej walidacji, co gwarantuje, że nawet całkowita niedostępność usługi AI **nie wpływa** na zdolność systemu do wykonywania i raportowania testów — jedynie na dostępność warstwy interpretacyjnej).



5.2 Komponenty wewnętrzne AI Analysis Engine

Komponent	Odpowiedzialność
Context Builder	Zbiera i przygotowuje kontekst dla zapytania do modelu: wynik bieżącego testu, historia N ostatnich przebiegów, dane powiązanych raportów tego samego obszaru/źródła — wyłącznie dane niewrażliwe (patrz sekcja 6). Komponent statystyczny/regulowy (niekoniecznie LLM) wykrywający odchylenia od wzorca historycznego — dostarcza sygnał wejściowy dla warstwy LLM, a także może działać niezależnie jako prosty detektor trendu.
Anomaly Detector	Warstwa komunikacji z modelem językowym (Azure OpenAI / OpenAI API), zarządzanie promptami, wymuszanie strukturalnej odpowiedzi (np. schema JSON z polami explanation, evidence, probable_causes, suggested_actions, confidence).
LLM Orchestration Layer	Waliduje odpowiedź modelu pod kątem zgodności ze schematem i obecności evidence przed zapisem/prezentacją — odrzuca i loguje odpowiedzi niespełniające wymogu wyjaśnialności.
Response Validator	Grupowanie podobnych błędów (może wykorzystywać embeddings + podobieństwo wektorowe lub reguły oparte na metadanych — decyzja implementacyjna do doprecyzowania w Fazie 2, patrz 11_Roadmap.md).
Clustering Engine	Odrębna tabela/przestrzeń przechowująca wygenerowane interpretacje, powiązana z wynikiem testu, ale niewpływająca na jego status (patrz 07_Data_Model.md).
AI Insights Store	

5.3 Wybór dostawcy modelu

Zgodnie z zaleceniem projektowym: **LangChain** (lub równoważny framework orkiestracji, np. lekki, autorski wrapper — decyzja w Fazie 0 zależna od faktycznej złożoności potrzebnych łańcuchów) wykorzystywany **wyłącznie w module AI**, oraz **Azure OpenAI** jako rekomendowany dostawca modelu w kontekście bankowym (dzierżawa Azure w ramach umowy korporacyjnej, kontraktowe gwarancje nieużywania danych do trenowania, rezydencja danych w UE, zgodność z politykami bezpieczeństwa organizacji) — alternatywnie hostowany model open-weight w infrastrukturze wewnętrznej, jeśli polityka Security Office wykluczy jakikolwiek ruch do usług chmurowych nawet w modelu kontraktowo kontrolowanym (do potwierdzenia w Fazie 0, patrz 00_Technical_Feasibility_Study.md, sekcja 8, oraz 12_Risks.md).

6. Dane wykorzystywane przez AI i ochrona danych wrażliwych

- AI Analysis Engine otrzymuje **wyłącznie zagregowane wyniki testów i metadane** (nazwy KPI, wartości liczbowe, statusy, historię, nazwy raportów/obszarów biznesowych) — **nigdy surowych danych klienckich** ani danych sklasyfikowanych jako wrażliwe/osobowe, zgodnie z zasadą minimalizacji danych.
- Definicje testów (zapytania SQL) mogą być przekazywane w ograniczonym zakresie wyłącznie jako metadane strukturalne (np. nazwa testowanej miary), nie jako pełna treść zapytania, jeśli zawiera ona elementy wrażliwe — zasada do doprecyzowania z Data Governance w Fazie 0.

- W przypadku wykorzystania zewnętrznego dostawcy modelu obowiązuje wymóg NFR-SEC-07 (05_NonFunctional_Requirements.md) — kontraktowa gwarancja nieużywania danych do trenowania, przetwarzanie w regionie zgodnym z polityką rezydencji danych.

7. Ryzyka związane z AI i mitygacje

Ryzyko	Mitygacja
Halucynacja modelu (wymyślone "fakty" nieoparte na danych)	Wymóg strukturalnej odpowiedzi z obowiązkowym polem evidence, walidacja odpowiedzi przed prezentacją (Response Validator), jednoznaczne oznaczenie treści jako wspierającej.
Nadmierne zaufanie użytkowników do sugestii AI ("automation bias")	Wyraźne oznaczenie wizualne treści AI jako odrębnej od werdyktu deterministycznego (NFR-U-02); szkolenie użytkowników; możliwość oznaczenia sugestii jako nietrafnej (feedback loop).
Wyciek danych wrażliwych do zewnętrznego dostawcy modelu	Minimalizacja danych przekazywanych do modelu (sekcja 6), wybór dostawcy zgodnego z polityką organizacji, możliwość wariantu w pełni on-prem.
Niedostępność usługi AI (zewnętrzne API)	AI Analysis Engine jest komponentem non-blocking — niedostępność nie wpływa na wykonanie i raportowanie testów (sekcja 5.1); degradacja funkcjonalna ograniczona do braku interpretacji.
Koszt operacyjny wywołań modelu przy skali 1000 raportów	Wywoływanie AI selektywnie — wyłącznie dla przypadków FAIL/WARNING lub wykrytych anomalii, nie dla każdego pojedynczego zielonego wyniku (optymalizacja kosztowa opisana w 09_Python_Architecture.md).

8. Powiązanie z dalszą dokumentacją

- Model danych dla AI Insights — 07_Data_Model.md.
- Endpointy API do pobierania interpretacji AI — 08_API.md.
- Wybór bibliotek (LangChain, klient Azure OpenAI) — 09_Python_Architecture.md.
- Ryzyka projektowe związane z AI jako pozycja w rejestrze ryzyk — 12_Risks.md.

07. Data Model

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Model danych i słownik encji **Odbiorcy:** Zespół Deweloperski, DBA, Architektura Danych

1. Cel dokumentu

Dokument opisuje logiczny model danych systemu ARGUS na poziomie encji biznesowych i ich relacji. Pełny diagram graficzny (encje, atrybuty, kardynalności) znajduje się w `uml/ERD.puml`. Fizyczny model bazy danych (typy kolumn, indeksy, migracje) jest przedmiotem implementacji w Alembic, na bazie niniejszego modelu logicznego — patrz `09_Python_Architecture.md`.

Silnik bazy danych: **PostgreSQL** (uzasadnienie wyboru — patrz `09_Python_Architecture.md`, sekcja Stack Technologiczny).

2. Słownik encji

2.1 BusinessArea (Obszar biznesowy / zakładka)

Odwzorowuje grupowanie raportów zgodnie ze strukturą istniejącej Platformy BI (zakładki: Sprzedaż, Kredyty, Ryzyko itd.).

Atrybut	Opis
id	Identyfikator
name	Nazwa obszaru biznesowego
description	Opis
owner	Właściciel biznesowy obszaru (opcjonalnie)

2.2 Report (Raport)

Reprezentuje pojedynczy raport z katalogu Platformy BI.

Atrybut	Opis
id	Identyfikator
name	Nazwa raportu
platform_type	POWER_BI SSRS (rozszerzalne — patrz <code>03_Architecture.md</code> , sekcja 6)
url	Adres raportu na Platformie BI
business_area_id	FK → BusinessArea
owner	Właściciel biznesowy raportu (analityk odpowiedzialny)
is_active	Czy raport jest aktywnie monitorowany
created_at, updated_at	Znaczniki czasu

2.3 ValidationProfile

Konfiguracja testowa przypisana do raportu — **wersjonowana** (każda zmiana tworzy nową wersję, poprzednie są zachowywane dla audytu).

Atrybut	Opis
id	Identyfikator
report_id	FK → Report
version	Numer wersji (rosnący)
is_active	Czy jest to aktualnie obowiązująca wersja
schedule_cron	Wyrażenie harmonogramu (częstotliwość wykonania)
created_by	FK → User
created_at	Znacznik czasu utworzenia wersji
change_reason	Opcjonalny opis powodu zmiany (dla audytu)

2.4 TestDefinition

Pojedynczy test wchodzący w skład Validation Profile.

Atrybut	Opis
id	Identyfikator
validation_profile_id	FK → ValidationProfile
name	Nazwa testu (np. "KPI_SPRZEDAZ_TOTAL")
type	SQL_COMPARISON FIELD_VALUE BUSINESS_RULE AVAILABILITY
extraction_config	Konfiguracja odnajdywania wartości w raporcie (selektor semantyczny, strategia fallback — patrz 00_Technical_Feasibility_Study.md)
tolerance_abs	Dopuszczalne odchylenie bezwzględne
tolerance_pct	Dopuszczalne odchylenie procentowe
is_active	Czy test jest aktywny

2.5 SqlValidationQuery

Zapytanie SQL powiązane z testem typu SQL_COMPARISON.

Atrybut	Opis
id	Identyfikator
test_definition_id	FK → TestDefinition
sql_text	Treść zapytania (parametryzowana)
data_source_ref	Referencja do zdefiniowanego, autoryzowanego źródła danych (hurtowni) — nigdy dowolny connection string wpisywany ad-hoc
expected_result_mapping	Mapowanie wyniku zapytania na wartość porównywaną

2.6 BusinessRule

Reguła biznesowa powiązana z testem typu BUSINESS_RULE (patrz 03_Architecture.md, sekcja 3.3).

Atrybut	Opis
id	Identyfikator
test_definition_id	FK → TestDefinition
rule_type	Typ reguły (np. SUM_CHECK, RANGE_CHECK, TREND_DEVIATION, NON_NEGATIVE)
configuration	Parametry reguły (strukturalne, zależne od typu)

2.7 TestExecution (Przebieg testowy)

Pojedyncze wykonanie zestawu testów dla raportu (całego Validation Profile) w danym momencie.

Atrybut	Opis
id	Identyfikator
report_id	FK → Report
validation_profile_id	FK → ValidationProfile (konkretna wersja użyta w tym przebiegu — kluczowe dla audytu)
triggered_by	SCHEDULER MANUAL:{user_id}
started_at, finished_at	Znaczniki czasu
overall_status	PASS FAIL WARNING ERROR (zagregowany status przebiegu)
platform_availability_status	Status dostępności/renderowania raportu (odróżnia awarię platformy od błędu danych — patrz NFR-A-04)

2.8 TestResult (Wynik pojedynczego testu)

Wynik jednego TestDefinition w ramach danego TestExecution.

Atrybut	Opis
id	Identyfikator
test_execution_id	FK → TestExecution
test_definition_id	FK → TestDefinition
status	PASS FAIL WARNING ERROR
report_value	Wartość odczytana z raportu
sql_value	Wartość z hurtowni danych (jeśli dotyczy)
deviation_abs, deviation_pct	Wyliczone odchylenie
extraction_method	Metoda ekstrakcji faktycznie użyta (patrz 00_Technical_Feasibility_Study.md, sekcja 7)
confidence_score	Wskaźnik pewności odczytu (0.0-1.0)
raw_evidence_ref	Referencja do zapisanego dowodu (zrzut ekranu / fragment DOM / plik eksportu)

2.9 HealthScoreSnapshot

Zagregowany, okresowy wskaźnik jakości raportu (do szybkiego renderowania trendów bez przeliczania całej historii na żądanie — denormalizacja celowa dla wydajności, patrz 05_Non-Functional_Requirements.md, NFR-P-05).

Atrybut	Opis
id	Identyfikator
report_id	FK → Report
snapshot_date	Data agregacji
health_score	Wartość wskaźnika (0-100)
pass_count, fail_count, warning_count	Liczniki składowe

2.10 AIInsight

Interpretacja wygenerowana przez moduł AI, powiązana z przebiegiem/wynikiem testu — **nigdy nie modyfikuje** encji TestResult/TestExecution (patrz 06_AI_Module.md).

Atrybut	Opis
id	Identyfikator
test_execution_id	FK → TestExecution
explanation	Wyjaśnienie w języku naturalnym
evidence	Dane źródłowe, na których oparto wyjaśnienie (strukturalne, JSON)
probable_causes	Lista prawdopodobnych przyczyn
suggested_actions	Sugerowane działania naprawcze
cluster_id	Opcjonalne powiązanie z grupą podobnych błędów
model_version, prompt_version	Wersjonowanie dla audytu i odtwarzalności
user_feedback	Opcjonalna ocena trafności przez użytkownika (feedback loop)
created_at	Znacznik czasu

2.11 Notification

Rekord wysłanego powiadomienia.

Atrybut	Opis
id	Identyfikator
test_execution_id	FK → TestExecution
channel	EMAIL inne kanały organizacyjne
recipient	Odbiorca
sent_at	Znacznik czasu
status	Status dostarczenia

2.12 User / Role

Atrybut	Opis
User.id, User.username, User.email, User.role Role	Konto użytkownika zsynchronizowane z SSO organizacji ADMINISTRATOR ANALYST_OWNER VIEWER (patrz 04_Functional_Requirements.md, RBAC)

2.13 AuditLogEntry

Append-only log audytowy (patrz 05_NonFunctional_Requirements.md, NFR-C-01).

Atrybut	Opis
id	Identyfikator
actor_id	FK → User (lub SYSTEM)
action	Typ zdarzenia (np. PROFILE_UPDATED, TEST_TRIGGERED_MANUALLY)
entity_type, entity_id	Encja, której dotyczy zdarzenie
before_state, after_state	Stan przed/po (JSON)
timestamp	Znacznik czasu

3. Diagram relacji (skrót)

Pełny diagram — `uml/ERD.puml`. Kluczowe relacje:

```

BusinessArea (1) — (N) Report
Report (1) — (N) ValidationProfile [wersje w czasie]
ValidationProfile (1) — (N) TestDefinition
TestDefinition (1) — (0..1) SqlValidationQuery
TestDefinition (1) — (0..1) BusinessRule
Report (1) — (N) TestExecution
ValidationProfile (1) — (N) TestExecution [wersja użyta w danym przebiegu]
TestExecution (1) — (N) TestResult
TestDefinition (1) — (N) TestResult
TestExecution (1) — (0..N) AIInsight
TestExecution (1) — (0..N) Notification
Report (1) — (N) HealthScoreSnapshot
User (1) — (N) ValidationProfile [created_by]
User/SYSTEM (1) — (N) AuditLogEntry

```

4. Strategia retencji i partycjonowania

Ze względu na skalę docelową (≥ 1000 raportów, wiele przebiegów testowych dziennie, wieloletnia historia), tabele `TestExecution`, `TestResult` i `AuditLogEntry` są kandydatami do **partycjonowania po czasie** (np. partycje miesięczne w PostgreSQL, `pg_partman` lub natywne partycjonowanie deklaratywne) w celu:

- utrzymania wydajności zapytań na “świeżych” danych (najczęściej odpytywanych przez Dashboard),
- umożliwienia efektywnego archiwizowania/usuwania starych partycji zgodnie z polityką retencji (Data Governance),
- ograniczenia kosztu operacji `VACUUM/REINDEX` na dużych tabelach.

`HealthScoreSnapshot` jest tabelą celowo zdenormalizowaną (agregaty prekalkulowane cyklicznie, a nie liczone on-the-fly z `TestResult`), aby zapewnić szybkie ładowanie Dashboardu (NFR-P-05) niezależnie od rozmiaru historii szczegółowej.

Rekomendowana wyjściowa polityka retencji (do potwierdzenia z Data Governance): - `TestResult/TestExecution` (dane szczegółowe): min. 24 miesiące w bazie operacyjnej, starsze — archiwizacja do zimnego storage. - `AuditLogEntry`: zgodnie z polityką audytową organizacji (typowo dłużej niż dane operacyjne). - `AIInsight`: powiązane cyklem życia z `TestExecution`.

5. Powiązanie z dalszą dokumentacją

- Pełny diagram ERD — `uml/ERD.puml`.
- Diagram klas domenowych (zachowania, nie tylko dane) — `uml/ClassDiagram.puml`.
- Kontrakty API operujące na powyższych encjach — `08_API.md`.
- Implementacja (SQLAlchemy models, Alembic migrations) — `09_Python_Architecture.md`.

08. API

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Projekt API REST **Odbiorcy:** Zespół Deweloperski, Zespoły integrujące się z ARGUS

1. Zasady projektowe API

1. **REST nad HTTP/JSON**, zgodny ze specyfikacją **OpenAPI 3.1** — dokumentacja API generowana automatycznie z kodu (FastAPI generuje schemat OpenAPI natywnie na bazie modeli Pydantic, co eliminuje ryzyko rozjazdu między dokumentacją a implementacją).
 2. **Wersjonowanie w ścieżce** (/api/v1/...) — pozwala na wprowadzanie zmian niekompatybilnych wstecz bez łamania istniejących integracji (np. przyszłych integracji z innymi systemami banku).
 3. **Zasoby, nie akcje** — endpointy modelowane wokół rzeczowników (/reports, /validation-profiles, /test-executions), zgodnie z konwencją REST; wyjątki (akcje typu “uruchom test”) modelowane jako pod-zasoby/operacje (POST /reports/{id}/test-executions).
 4. **Spójna struktura odpowiedzi błędu** — zgodna z RFC 7807 (application/problem+json): type, title, status, detail, instance, ułatwiająca jednolitą obsługę błędów po stronie klienta (GUI).
 5. **Paginacja kursorowa** dla list mogących urosnąć do dużych rozmiarów (test-executions, audit-log) — stabilniejsza niż offsetowa przy dużym, stale rosnącym wolumenie danych.
 6. **Idempotencja** dla operacji tworzących zasoby wywoływanych z potencjalnym retry po stronie klienta (np. ręczne wyzwolenie testu) — poprzez nagłówek Idempotency-Key.
 7. **Autoryzacja na poziomie zasobu** — każdy endpoint modyfikujący dane weryfikuje rolę RBAC (patrz 04_Functional_Requirements.md, FR-060) poprzez zależność FastAPI (Depends(require_role(...))).
-

2. Uwierzytelnianie i autoryzacja

- Uwierzytelnianie oparte o token wydany po SSO organizacji (np. OIDC/OAuth2 Authorization Code Flow z Azure AD/AD FS — do potwierdzenia z zespołem IAM w Fazie 0), przekazywany jako Bearer token w nagłówku Authorization.
 - API waliduje token (podpis, ważność, aud/iss) i mapuje tożsamość na encję User z przypisaną rolą RBAC.
 - Komunikacja API ↔ workery (wewnętrzna, service-to-service) zabezpieczona osobnym mechanizmem (mTLS lub token serwisowy w sieci wewnętrznej) — nieeksponowana na zewnątrz.
-

3. Zasoby główne i endpointy

3.1 Reports — katalog raportów

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/reports	Lista raportów (filtrowanie: business_area_id, platform_type, status, wyszukiwanie tekstowe q)	Viewer
GET	/api/v1/reports/{report_id}	Szczegóły raportu, w tym aktualny Validation Profile i ostatni wynik	Viewer
POST	/api/v1/reports	Rejestracja nowego raportu w katalogu	Administrator
PATCH	/api/v1/reports/{report_id}	Aktualizacja metadanych raportu (właściciel, obszar biznesowy, aktywność)	Administrator
GET	/api/v1/business-areas	Lista obszarów biznesowych (zakładek)	Viewer

3.2 Validation Profiles

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/reports/{report_id}/validation-profiles	Historia wersji profilu dla raportu	Viewer
GET	/api/v1/validation-profiles/{profile_id}	Szczegóły konkretnej wersji profilu	Viewer
POST	/api/v1/reports/{report_id}/validation-profiles	Utworzenie nowej wersji Validation Profile (testy, harmonogram, tolerancje)	Analyst/Owner
POST	/api/v1/validation-profiles/{profile_id}/activate	Aktywacja danej wersji jako obowiązującej	Analyst/Owner
POST	/api/v1/validation-profiles/{profile_id}/clone	Klonowanie profilu jako punkt startowy dla innego raportu (FR-018)	Analyst/Owner

3.3 Test Definitions (w ramach profilu)

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/validation-profiles/{profile_id}/test-definitions	Lista testów w profilu	Viewer
POST	/api/v1/validation-profiles/{profile_id}/test-definitions	Dodanie testu (SQL / wartość pola / reguła biznesowa)	Analyst/Owner
POST	/api/v1/test-definitions/validate	Walidacja poprawności definicji testu przed zapisem (Test Definition Engine) — sprawdzenie składni SQL, poprawności konfiguracji ekstrakcji	Analyst/Owner
DELETE	/api/v1/test-definitions/{test_id}	Dezaktywacja testu (soft-delete, zachowanie w historii)	Analyst/Owner

3.4 Test Executions — wykonania testów

Metoda	Endpoint	Opis	Rola min.
POST	/api/v1/reports/{report_id}/test-executions	Ręczne uruchomienie testu dla raportu (wymaga Idempotency-Key)	Analyst/Owner
POST	/api/v1/business-areas/{area_id}/test-executions	Ręczne uruchomienie testów dla całego obszaru biznesowego	Analyst/Owner
GET	/api/v1/test-executions	Lista przebiegów testowych (filtrowanie: report_id, status, zakres dat; paginacja kursorowa)	Viewer
GET	/api/v1/test-executions/{execution_id}	Szczegóły przebiegu wraz z wynikami poszczególnych testów (TestResult)	Viewer
GET	/api/v1/test-executions/{execution_id}/status	Lekki endpoint statusu (do odpytywania “w toku” z GUI)	Viewer

3.5 Health & Confidence Scores

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/reports/{report_id}/health-score?from=&to=	Trend Health Score raportu w zadanym okresie	Viewer
GET	/api/v1/business-areas/{area_id}/health-score	Zagregowany Health Score obszaru biznesowego	Viewer
GET	/api/v1/dashboard/summary	Zagregowane podsumowanie dla ekranu głównego (liczniki statusów, trend globalny) — endpoint zoptymalizowany pod NFR-P-05	Viewer

3.6 AI Insights

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/test-executions/{execution_id}/ai-insight	Interpretacja AI powiązana z przebiegiem (jeśli wygenerowana)	Viewer
GET	/api/v1/ai-insights/clusters	Lista klastrów podobnych błędów (FR-041)	Viewer
POST	/api/v1/ai-insights/{insight_id}/feedback	Ocena trafności sugestii AI przez użytkownika (feedback loop)	Analyst/Owner

3.7 Notifications

Metoda	Endpoint	Opis	Rola min.
GET	/api/v1/notifications	Historia wysłanych powiadomień (filtrowanie)	Analyst/Owner
POST	/api/v1/validation-profiles/{profile_id}/notification-rules	Konfiguracja reguł powiadomień/eskalacji dla profilu	Analyst/Owner

3.8 Administracja

Metoda	Endpoint	Opis	Rola min.
GET/POST	/api/v1/admin/users	Zarządzanie użytkownikami i rolami (RBAC)	Administrator
GET/POST	/api/v1/admin/connectors	Zarządzanie katalogiem platform/connectorów (bez ekspozycji sekretów)	Administrator
GET	/api/v1/admin/audit-log	Przegląd logu audytowego (filtrowanie, paginacja kursorowa)	Administrator
GET	/api/v1/admin/system-metrics	Skrócony widok metryk operacyjnych wewnątrz aplikacji (uzupełniający Grafanę)	Administrator

4. Przykładowy kontrakt — wynik przebiegu testowego

Poniżej ilustracyjny (skrótowy) kształt odpowiedzi GET /api/v1/test-executions/{execution_id}, prezentujący kluczowe pola istotne dla warstwy Presentation:

```
{
  "execution_id": "alb2c3d4",
  "report_id": "rep-00231",
  "report_name": "Sprzedaż / Konsorcja Kredytowe",
  "platform_type": "SSRS",
  "triggered_by": "SCHEDULER",
  "started_at": "2026-07-18T02:00:04Z",
  "finished_at": "2026-07-18T02:00:19Z",
  "overall_status": "WARNING",
  "platform_availability_status": "AVAILABLE",
  "test_results": [
    {
      "test_definition_id": "td-0091",
      "test_name": "KPI_SPRZEDAZ_TOTAL",
      "status": "WARNING",
      "report_value": "1204500.00",
      "sql_value": "1189300.00",
      "deviation_pct": 1.28,
      "tolerance_pct": 1.00,
      "extraction_method": "DOM_ANALYSIS",
      "confidence_score": 0.94
    }
  ],
  "ai_insight_available": true
}
```

Uwaga projektowa: pole extraction_method i confidence_score są celowo eksponowane na poziomie API (a nie tylko wewnętrznie), zgodnie z wymogiem transparentności (FR-033, UC-13) — użytkownik biznesowy musi mieć możliwość oceny, jak bardzo ufać danemu wynikowi.

5. Obsługa błędów

Zgodnie z RFC 7807, przykład:

```
{
  "type": "https://argus.bank.com.pl/errors/platform-unavailable",
  "title": "Platforma raportowa niedostępna",
}
```

```
"status": 503,  
"detail": "SSRS Report Server nie odpowiedział w oczekiwanym czasie (timeout 30s).",  
"instance": "/api/v1/reports/rep-00231/test-executions/alb2c3d4"  
}
```

Kategorie błędów standaryzowane w całym API: validation-error (400), unauthorized (401), forbidden (403), not-found (404), conflict (409, np. równoległa edycja tego samego profilu), platform-unavailable (503, zgodnie z NFR-A-04), internal-error (500).

6. Webhooki / integracje wychodzące (rozszerzenie przyszłościowe)

Poza zakresem MVP, ale architektonicznie przewidziane (zgodnie z zasadą Open/Closed): możliwość rejestracji webhooków (POST /api/v1/admin/webhooks) powiadamiających zewnętrzne systemy organizacji (np. centralny system ticketowy) o zdarzeniach test_execution.failed — zaprojektowane jako rozszerzenie Notification Service o kolejny INotificationChannel, bez zmian w rdzeniu.

7. Powiązanie z dalszą dokumentacją

- Encje leżące pod powyższymi zasobami — 07_Data_Model.md.
- Implementacja API w FastAPI, struktura routerów — 09_Python_Architecture.md.
- Wymagania bezpieczeństwa dot. uwierzytelniania i szyfrowania transportu — 05_NonFunctional_Requirements.md.
- Diagram sekwencji ilustrujący przepływ wywołań dla scenariusza wykonania testu — uml/SequenceDiagram.puml.

09. Python Architecture

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Architektura kodu Python i stack technologiczny **Odbiorcy:** Zespół Deweloperski, Architektura IT

1. Stack technologiczny — pełna lista i uzasadnienie

Warstwa	Technologia	Uzasadnienie
Język	Python 3.13+	Dojrzały ekosystem bibliotek do automatyzacji przeglądarki, przetwarzania danych i integracji AI; natywne wsparcie dla <code>asyncio</code> kluczowe dla równoległości Browser Automation na skalę 1000 raportów; nowoczesne typowanie (generics, <code>TypedDict</code> , <code>Protocol</code>) wspierające zasady SOLID i statyczną analizę.
Framework API	FastAPI	Natywna integracja z Pydantic (walidacja danych, automatyczna dokumentacja OpenAPI), natywny <code>async/await</code> , wbudowany mechanizm Dependency Injection (<code>Depends</code>) spójny z zasadą DI z <code>03_Architecture.md</code> , bardzo dobra wydajność względem alternatyw synchronicznych (Flask/Django REST).
Automatyzacja przeglądarki	Playwright (Python, async API)	Uzasadnienie wyboru względem Selenium — patrz <code>00_Technical_Feasibility_Study.md</code> , sekcja 3.1–3.2. Natywne wsparcie <code>async</code> , <code>auto-waiting</code> , przechwytywanie sieci, stabilna obsługa wielu kontekstów przeglądarki na proces.
ORM / dostęp do danych	SQLAlchemy 2.x (styl <code>async</code>, <code>Mapped/mapped_column</code>)	Standard de facto w ekosystemie Python enterprise, pełne wsparcie typowania statycznego (MyPy), wzorzec Unit of Work/Repository łatwy do zaimplementowania nad SQLAlchemy Session, wsparcie dla trybu asynchronicznego zgodnego z resztą stacku.
Migracje schematu	Alembic	Standardowy, w pełni zintegrowany z SQLAlchemy mechanizm wersjonowania schematu bazy — kluczowy dla audytowalnych, powtarzalnych wdrożeń w środowisku bankowym (<code>Dev→Test→UAT→Prod</code>).

Warstwa	Technologia	Uzasadnienie
Baza danych	PostgreSQL	Dojrzałość, natywne partycjonowanie deklaratywne (istotne przy skali historii wyników — patrz 07_Data_Model.md), wsparcie JSONB (elastyczne przechowywanie extraction_config, evidence AI), silne wsparcie transakcyjne, brak kosztów licencyjnych, powszechna akceptacja w środowiskach bankowych jako alternatywa dla komercyjnych silników.
Cache / broker kolejki	Redis	Niskie opóźnienia, wsparcie jako broker dla Celery, przechowywanie stanu sesji/rate-limiting, cache wyników zapytań agregujących do Dashboardu (NFR-P-05).
Orkiestracja zadań asynchronicznych	Celery (z Redis lub RabbitMQ jako broker)	Rekomendacja wyjściowa: Celery + Redis dla MVP (prostota operacyjna), z możliwością migracji na RabbitMQ jako broker w fazie skalowania produkcyjnego (Faza 4, patrz 11_Roadmap.md), jeśli wymagana będzie wyższa gwarancja dostarczenia i bardziej zaawansowane routing zadań. Celery wybrany zamiast samego APScheduler, ponieważ APScheduler dobrze sprawdza się jako harmonogram (triggering), ale nie jako rozproszony worker pool wykonawczy — stąd rekomendacja hybrydowa: APScheduler wewnątrz komponentu Scheduler (odpowiedzialność: kiedy uruchomić) + Celery jako warstwa wykonawcza (Test Orchestrator → zadania Celery na worker pool, odpowiedzialność: jak i gdzie wykonać, z retry/backoff/circuit breaker).
Walidacja danych / konfiguracji	Pydantic v2	Wysoka wydajność (rdzeń w Rust), natywna integracja z FastAPI, pydantic-settings do zarządzania konfiguracją zgodnie z 12-Factor App (patrz 03_Architecture.md, sekcja 5.2).
Framework AI	LangChain (moduł AI wyłącznie)	Zgodnie z wymogiem projektowym — używany punktowo do orkiestracji łańcucha: budowa promptu → wywołanie modelu → walidacja odpowiedzi strukturalnej (schema JSON) → zapis. Alternatywa rozważana: lekki własny wrapper nad klientem Azure OpenAI, jeśli złożoność łańcuchów w Fазie 3 okaże się niewystarczająco duża, by uzasadnić zależność od LangChain — decyzja do potwierdzenia w Fазie 3 na bazie rzeczywistych potrzeb (unikanie nadmiarowej złożoności — zasada YAGNI).
Dostawca modelu AI	Azure OpenAI (rekomendacja)	Patrz uzasadnienie w 06_AI_Module.md, sekcja 5.3 oraz 05_NonFunctional_Requirements.md, NFR-SEC-07.
Automatyzacja przeglądarki — CV/ OCR (fallback)	OpenCV (Computer Vision) + Tesseract / pytesseract (OCR)	Standardowe, sprawdzone biblioteki open-source do warstwy fallbacku Value Extraction Engine (patrz 00_Technical_Feasibility_Study.md, sekcja 6.6–6.7); brak kosztów licencyjnych, możliwość pracy w pełni on-prem.

Warstwa	Technologia	Uzasadnienie
Integracja XMLA (opcjonalna)	pythonnet + ADOMD.NET lub dedykowany mikroservis .NET	Patrz 00_Technical_Feasibility_Study.md, sekcja 3.4 — most technologiczny do XMLA Endpoint, izolowany jako opcjonalny connector, nieobowiązkowy dla MVP.
Frontend / GUI	React + TypeScript	Dojrzały ekosystem komponentowy, dobre wsparcie dla wykresów/trendów (biblioteki typu Recharts/Chart.js), łatwość integracji z zespołami frontendowymi banku posiadającymi doświadczenie w React; TypeScript zapewnia bezpieczeństwo typów spójne z podejściem statycznego typowania po stronie backendu.
Konteneryzacja	Docker / Docker Compose (Dev/Test), Kubernetes (docelowo Prod)	Docker Compose wystarczający i szybki do uruchomienia dla środowisk deweloperskich/testowych; Kubernetes rekomendowany dla produkcji ze względu na potrzebę horyzontalnego skalowania worker poola Playwright (patrz 10_Deployment.md).
Testy	pytest (+ pytest-asyncio, pytest-cov, playwright fixtures)	Standard w ekosystemie Python, bogaty ekosystem pluginów, czytelna składnia parametryzacji testów kontraktowych connectorów (NFR-M-03).
Statyczna analiza	Ruff (linting + formatting) + MyPy (strict mode)	Ruff zastępuje jednocześnie flake8/isort/black przy znacząco wyższej wydajności (implementacja w Rust) — szybki feedback w CI; MyPy strict wymusza pełne typowanie zgodne z zasadami SOLID/Clean Architecture (interfejsy jako Protocol/ABC).
CI/CD	GitHub Actions	Zgodnie z rekomendacją projektową; pipeline: lint → type-check → testy jednostkowe/kontraktowe → build obrazu Docker → skanowanie podatności (SCA/SAST) → publikacja do wewnętrznego rejestru.
Obserwowalność	Prometheus + Grafana + OpenTelemetry	Zgodnie z 05_NonFunctional_Requirements.md, sekcja Observability — standard branżowy, dobra integracja z FastAPI (prometheus-fastapi-instrumentator) i Celery (celery-prometheus-exporter lub instrumentacja własna).
Zarządzanie zależnościami	uv (rekomendacja) lub Poetry	uv rekomendowany ze względu na istotnie wyższą wydajność instalacji/resolwingu zależności (implementacja w Rust) oraz rosnącą dojrzałość w ekosystemie 2025/2026; Poetry jako sprawdzona alternatywa, jeśli standardy wewnętrzne organizacji tego wymagają — decyzja do ostatecznego zatwierdzenia z zespołem platformowym w Fazie 0.
Kontener DI	dependency-injector	Jawny, czytelny Composition Root (patrz 03_Architecture.md, sekcja 3.4), wsparcie dla konfiguracji kontenera per środowisko (dev/test/prod) oraz łatwe podmienianie implementacji w testach (Fake/Mock connectory).

2. Struktura katalogów (Clean Architecture / Ports & Adapters)

```

argus/
├── src/
│   ├── argus/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   ├── value_objects/
│   │   │   ├── services/
│   │   │   └── ports/
│   │   │       # DOMAIN LAYER – zero zależności od frameworków
│   │   │       # Report, ValidationProfile, TestDefinition, TestResult, ...
│   │   │       # ExtractedValue, HealthScore, ConfidenceScore, Tolerance
│   │   │       # ValidationEngine, BusinessRuleEngine (logika czysta)
│   │   │       # Interfejsy (Protocol): IReportRepository,
│   │   │       # IValidationProfileRepository, IResultsRepository,
│   │   │       # IReportConnector, IExtractionStrategy, ISqlValidator,
│   │   │       # INotificationChannel, IAIProvider
│   │   ├── application/
│   │   │   # APPLICATION LAYER – przypadki użycia / orkiestracja
│   │   │   ├── use_cases/
│   │   │   │   # np. RunTestExecutionUseCase, CreateValidationProfileUseCase
│   │   │   ├── orchestration/
│   │   │   │   # TestOrchestrator, Scheduler (logika harmonogramowania)
│   │   │   └── ai/
│   │   │       # AIAnalysisEngine (orkiestracja wywołań, non-blocking)
│   │   ├── infrastructure/
│   │   │   # INFRASTRUCTURE LAYER – adaptery
│   │   │   ├── connectors/
│   │   │   │   ├── power_bi/
│   │   │   │   │   # PowerBICollector + strategie ekstrakcji specyficzne
│   │   │   │   ├── ssrs/
│   │   │   │   │   # SSRSConnector + strategie ekstrakcji specyficzne
│   │   │   │   └── factory.py
│   │   │   │   # ConnectorFactory (Registry Pattern)
│   │   │   ├── browser_automation/
│   │   │   │   # Cienka warstwa nad Playwright (pule kontekstów, SS0)
│   │   │   ├── extraction/
│   │   │   │   # Implementacje IExtractionStrategy (DOM, CV, OCR, Export, API)
│   │   │   │   ├── sql_validation/
│   │   │   │   │   # SQLValidationEngine (bezpieczne wykonanie zapytań)
│   │   │   │   ├── persistence/
│   │   │   │   │   ├── models/
│   │   │   │   │   │   # Modele SQLAlchemy (mapowanie 1:1 z domain/entities)
│   │   │   │   │   ├── repositories/
│   │   │   │   │   │   # Implementacje portów repozytoriów
│   │   │   │   │   └── migrations/
│   │   │   │   │       # Alembic
│   │   │   │   ├── notifications/
│   │   │   │   │   # Implementacje INotificationChannel (Email, ...)
│   │   │   │   ├── ai_providers/
│   │   │   │   │   # Implementacja IAIProvider (Azure OpenAI + LangChain)
│   │   │   │   ├── audit/
│   │   │   │   │   # Audit & Logging (append-only writer)
│   │   │   │   └── monitoring/
│   │   │   │       # Instrumentacja Prometheus/OpenTelemetry
│   │   ├── presentation/
│   │   │   # PRESENTATION LAYER
│   │   │   ├── api/
│   │   │   │   ├── routers/
│   │   │   │   │   # reports.py, validation_profiles.py, test_executions.py, ...
│   │   │   │   ├── schemas/
│   │   │   │   │   # Pydantic request/response models (DTO, odseparowane od domain)
│   │   │   │   │   ├── dependencies.py
│   │   │   │   │   │   # Depends() – auth, RBAC, DI wiring per request
│   │   │   │   └── workers/
│   │   │   │       # Celery task definitions (cienkie, delegujące do use_cases)
│   │   ├── config/
│   │   │   # Pydantic Settings, konfiguracja per środowisko
│   │   └── composition_root.py
│   │       # Wiring kontenera DI – jedyne miejsce łączące porty z adapterami
│   ├── frontend/
│   │   # Aplikacja React + TypeScript (GUI)
│   ├── tests/
│   │   ├── unit/
│   │   │   # Testy domain/ i application/ (z Fake/Mock adapterami)
│   │   ├── contract/
│   │   │   # Testy kontraktowe każdego IReportConnector (NFR-M-03)
│   │   ├── integration/
│   │   │   # Testy z realną bazą (testcontainers) i mockowaną platformą BI
│   │   └── e2e/
│   │       # Testy end-to-end na środowisku Test/UAT (ograniczony zakres)
│   └── docker/

```

```
|   ├── Dockerfile.api
|   ├── Dockerfile.worker
|   └── docker-compose.yml
|
|── .github/workflows/                # Pipeline CI/CD (GitHub Actions)
|── pyproject.toml
|── alembic.ini
```

Uzasadnienie: struktura fizycznych katalogów odwzorowuje 1:1 warstwy logiczne z 03_Architecture.md, sekcja 2 — deweloper poruszający się po repozytorium natychmiast widzi, do której warstwy Clean Architecture należy dany plik, co ułatwia utrzymanie reguły zależności (import-linter / narzędzie do wymuszania granic warstw rekomendowane jako część CI — patrz sekcja 5).

3. Dependency Injection — Composition Root

```
# composition_root.py – ilustracja koncepcyjna, nie kod produkcyjny do wdrożenia wprost
from dependency_injector import containers, providers

class ApplicationContainer(containers.DeclarativeContainer):
    config = providers.Configuration()

    # Infrastructure – repozytoria
    report_repository = providers.Singleton(SqlAlchemyReportRepository, session_factory=...)
    validation_profile_repository = providers.Singleton(SqlAlchemyValidationProfileRepository, ...)

    # Infrastructure – connectory (Registry przez Factory)
    connector_factory = providers.Singleton(
        ConnectorFactory,
        connectors={
            "POWER_BI": providers.Factory(PowerBIConnector, browser_engine=...),
            "SSRS": providers.Factory(SSRSConnector, browser_engine=...),
        },
    )

    # Application – use case'y komponowane z portów
    run_test_execution_use_case = providers.Factory(
        RunTestExecutionUseCase,
        connector_factory=connector_factory,
        extraction_chain=...,
        validation_engine=...,
        results_repository=...,
        ai_engine=...,
        notification_service=...,
    )
```

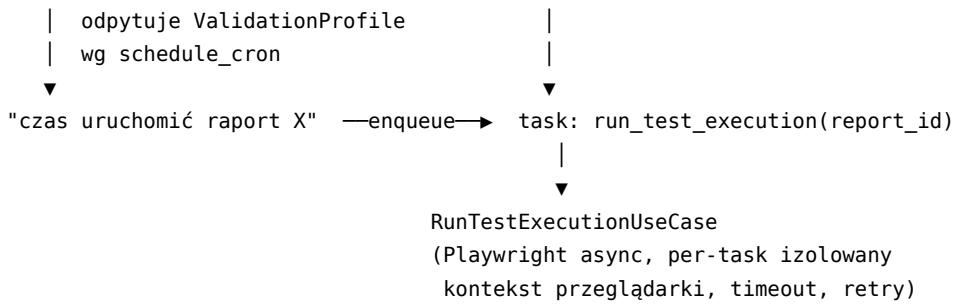
W testach jednostkowych ten sam ApplicationContainer jest konfigurowany z implementacjami Fake/Mock (np. FakeReportConnector zwracający zdefiniowane w teście dane), co pozwala testować RunTestExecutionUseCase w pełnej izolacji od Playwright, sieci czy bazy danych — kluczowe dla realizacji NFR-M-02 (pokrycie testami warstwy domenowej $\geq 85\%$).

4. Model wykonawczy i współbieżność

4.1 Dwuwarstwowy model harmonogramu i wykonania

APScheduler (Scheduler)

Celery (worker pool, N replik)



- **APScheduler** odpowiada wyłącznie za “kiedy” (trigger), publikując zadania do kolejki Celery — rozdzielenie odpowiedzialności zgodne z SRP.
- **Celery workery** skalowane horyzontalnie (liczba replik konfigurowalna, patrz 10_Deployment.md), każdy worker zarządza własną pulą kontekstów przeglądarki Playwright (nie całych instancji przeglądarki — reużycie procesu przeglądarki między zadaniami dla wydajności, przy izolacji sesji przez oddzielne konteksty).
- Limit współbieżności per worker oraz globalny limit równoległych połączeń do hurtowni danych są konfigurowalne (config/), zgodnie z NFR-P-04 (ochrona hurtowni przed przeciążeniem).

4.2 Asynchroniczność w kodzie

Cały łańcuch krytyczny dla wydajności (browser_automation, extraction, wywołania HTTP do API platform, klient AI) implementowany w asyncio (natywne wsparcie Playwright async API i httpx.AsyncClient), z jawnymi timeoutami na każdym poziomie (asyncio.wait_for) zgodnie z wymogiem odpornościowym z 03_Architecture.md, sekcja 3.6.

5. Wymuszanie granic architektonicznych w CI

Aby zapobiec erozji architektury w czasie (częsty problem długoletnich projektów enterprise), pipeline CI zawiera dedykowany krok **import-linter** (lub równoważne narzędzie), wymuszający reguły:

- domain/ nie importuje niczego z infrastructure/ ani presentation/,
- application/ nie importuje bezpośrednio z infrastructure/ (wyłącznie przez porty zdefiniowane w domain/ports/),
- presentation/ nie zawiera logiki domenowej (wyłącznie mapowanie DTO ↔ wywołanie use case).

Naruszenie którejkolwiek reguły blokuje merge — architektura jest w ten sposób **wymuszana narzędziowo, nie tylko opisana w dokumentacji**.

6. Zarządzanie kosztem wywołań AI

Zgodnie z 06_AI_Module.md, sekcja 7 (ryzyko kosztowe), warstwa application/ai/ implementuje jawną politykę selektywnego wywoływania modelu — AIAnalysisEngine jest wyzwalany przez TestOrchestrator **wyłącznie** dla przebiegów o statusie innym niż pełny PASS lub oznaczonych przez Anomaly Detector jako odbiegające od trendu, minimalizując liczbę (a więc i koszt) wywołań LLM przy pełnej skali 1000 raportów.

7. Powiązanie z dalszą dokumentacją

- Warstwy i wzorce architektoniczne odwzorowane w powyższej strukturze — 03_Architecture.md.
- Encje domenowe mapowane do domain/entities/ i infrastructure/persistence/models/ — 07_Data_Model.md.
- Endpointy presentation/api/routers/ — 08_API.md.
- Środowiska uruchomieniowe, obrazy Docker, pipeline CI/CD — 10_Deployment.md.

10. Deployment

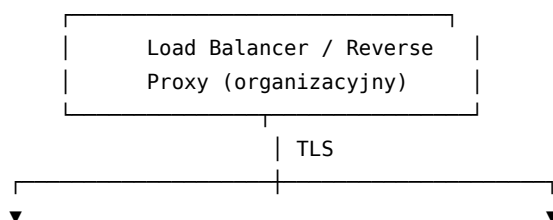
Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Środowiska, CI/CD, infrastruktura, Disaster Recovery **Odbiorcy:** Architektura IT, Zespół Infrastruktury/Platform, Security Office

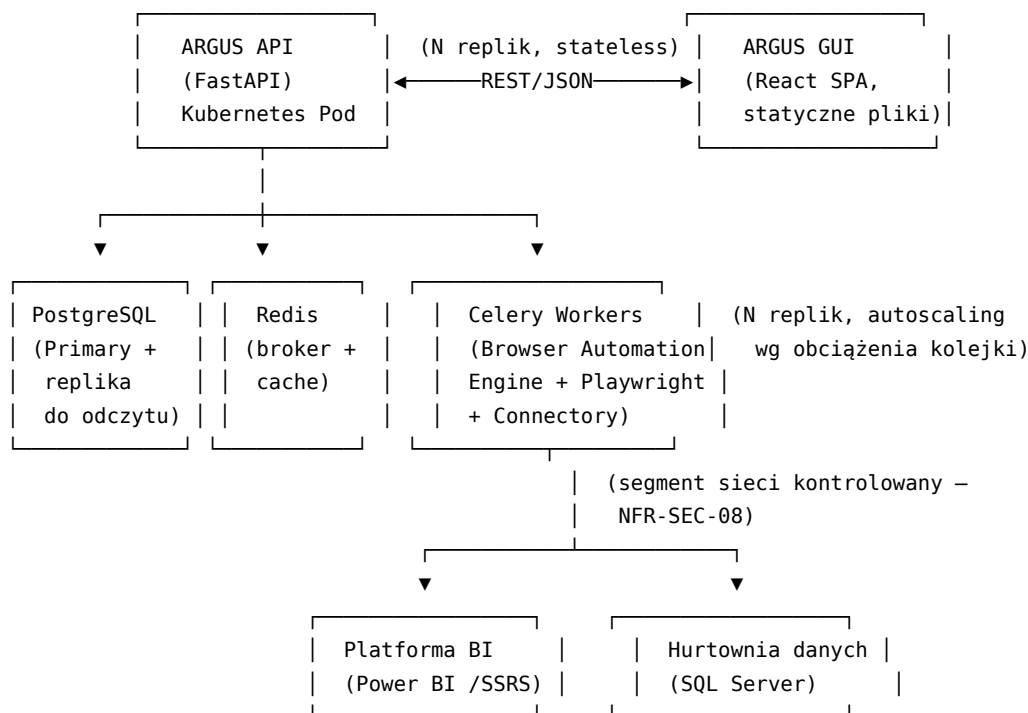
1. Środowiska

Środowisko	Cel	Charakterystyka
Dev	Praca deweloperska, integracja bieżąca	Docker Compose lokalnie/na współdzielonym hoście deweloperskim; dane testowe/syntetyczne; mockowane connectory platform BI dopuszczalne dla szybkiej iteracji.
Test	Testy automatyczne (integracyjne, kontraktowe), testy zespołu QA	Zbliżone do Prod pod względem topologii, ograniczony worker pool; połączenie z testowym/kopią środowiska Platformy BI, jeśli dostępne, w innym wypadku z reprezentatywnym podzbiorem raportów produkcyjnych (read-only).
UAT	Akceptacja biznesowa, przegląd bezpieczeństwa	Konfiguracja zbliżona do Prod, ograniczony podzbiór raportów produkcyjnych, pełna ścieżka SSO/autoryzacji jak w Prod.
Prod	Produkcja	Pełna skala (docelowo 1000 raportów), pełny worker pool, pełny monitoring i alerting, wysoka dostępność (NFR-A-01).

Wszystkie środowiska konteneryzowane (NFR-PO-01), różnice wyłącznie konfiguracyjne (zmienne środowiskowe / wpisy w magazynie sekretów), zgodnie z zasadą 12-Factor App.

2. Topologia infrastruktury (on-prem, zgodnie z założeniem projektowym)





Warstwa obserwowalności (równoległa do powyższych):

Prometheus ← metryki ← (API, Workery, Scheduler)

Grafana ← dashboardy operacyjne

OpenTelemetry Collector ← trace'y rozproszone

Centralny system logów organizacji ← logi ustrukturyzowane (JSON)

Magazyn sekretów (Vault / Azure Key Vault / rozwiązanie wewnętrzne)

← konto techniczne, connection stringi, klucze API (NFR-SEC-02)

Pełny, formalny diagram wdrożenia — patrz `uml/DeploymentDiagram.puml`.

2.1 Uzasadnienie kluczowych decyzji topologicznych

- **API jako komponent bezstanowy (stateless)**, umożliwiającą skalowanie horyzontalne i pracę za load balancerem bez sticky session — zgodnie z NFR-S-02.
- **Wydzielenie Celery Workers jako osobnej warstwy skalowanej niezależnie od API** — obciążenie generowane przez Browser Automation (CPU/RAM na instancję przeglądarki) ma zupełnie inną charakterystykę niż obciążenie API, więc niezależne skalowanie obu warstw optymalizuje koszt infrastruktury.
- **Segmentacja sieciowa workerów** — workery mają dostęp wyłącznie do Platformy BI i hurtowni danych (NFR-SEC-08), nie do szerszej sieci korporacyjnej, minimalizując powierzchnię ataku w razie kompromitacji kontenera wykonawczego.
- **PostgreSQL z repliką do odczytu** — przygotowanie pod przyszłe obciążenia analityczne/raportowe generowane przez sam Dashboard ARGUS bez obciążania instancji operacyjnej (NFR-S-02).

3. CI/CD — pipeline (GitHub Actions)

Push / Pull Request

|

▼

[1] Lint & Format Check

– Ruff



Kroki 1–9 wykonywane automatycznie na każdy Pull Request; wdrożenie do UAT/Prod podlega standardowemu procesowi zarządzania zmianą organizacji (poza automatycznym pipeline, zgodnie z procedurami banku).

4. Zarządzanie sekretami i kontem technicznym

- Konto techniczne (dostęp do Platformy BI i hurtowni danych) zarządzane zgodnie z polityką IAM organizacji — rekomendacja: **Group Managed Service Account (gMSA)** w środowisku Active Directory, jeśli dostępne, jako mechanizm eliminujący konieczność ręcznej rotacji haseł kontenerowych; alternatywnie konto serwisowe z hasłem rotowanym automatycznie przez magazyn sekretów.
- Sekrety wstrzykiwane do kontenerów wyłącznie w runtime (np. jako zamontowane pliki/zmienne środowiskowe pobierane z Vault/Key Vault przy starcie), nigdy zapisane w obrazie Docker ani w repozytorium (NFR-SEC-02).
- Rotacja sekretów — zgodnie z polityką organizacji, wspierana architektonicznie przez brak twardego cache'owania danych uwierzytelniających dłużej niż czas życia sesji roboczej workera.

5. Skalowanie

Komponent	Strategia skalowania
ARGUS API	Horyzontalne (liczba replik Kubernetes Pod), na podstawie obciążenia CPU/liczby requestów.
Celery Workers (Browser Automation)	Horyzontalne, na podstawie długości kolejki zadań (autoscaling oparty o metrykę głębokości kolejki Redis/RabbitMQ) — kluczowe dla realizacji NFR-P-01 (pełny cykl 1000 raportów w oknie $\leq 4h$).
PostgreSQL	Wertykalne w pierwszej kolejności; read-replika dla obciążeń odczytowych Dashboardu w miarę potrzeb.
Redis	Wertykalne / Redis Cluster w miarę wzrostu obciążenia kolejki i cache.

Docelowa liczba workerów i parametry autoscalingu wymagają kalibracji empirycznej w ramach Proof of Concept (Faza 0, patrz 11_Roadmap.md) — dokument nie zakłada z góry konkretnej liczby, ponieważ zależy ona od rzeczywistej charakterystyki wydajnościowej raportów w środowisku organizacji.

6. Disaster Recovery i kopie zapasowe

Aspekt	Rekomendacja wyjściowa
Kopie zapasowe PostgreSQL	Pełne kopie okresowe + Point-in-Time Recovery (WAL archiving), zgodnie ze standardem organizacji dla baz produkcyjnych.
RPO (Recovery Point Objective)	Do potwierdzenia z zespołem infrastruktury — rekomendacja wyjściowa: ≤ 1 godzina dla danych operacyjnych.
RTO (Recovery Time Objective)	Do potwierdzenia z zespołem infrastruktury — rekomendacja wyjściowa: ≤ 4 godziny.
Odtwarzalność konfiguracji	Cała konfiguracja aplikacji (poza sekretami) wersjonowana w repozytorium (Infrastructure as Code — manifesty Kubernetes/Helm chart), umożliwiając pełne odtworzenie środowiska z repozytorium + magazynu sekretów.
Odporność na awarię pojedynczego workera	Nie wymaga procedur DR — architektonicznie izolowana (NFR-A-02), automatyczne wznowienie zadania przez Celery.

Ostateczne wartości RPO/RTO oraz procedury DR muszą zostać zatwierdzone przez zespół infrastruktury/Business Continuity organizacji jako część standardowego procesu dla systemów klasy Enterprise — niniejszy dokument dostarcza rekomendacji wyjściowej do tej dyskusji, nie ostatecznej decyzji.

7. Integracja z monitoringiem organizacyjnym

- Metryki Prometheus eksponowane na standardowym endpointcie `/metrics` każdego komponentu, zbierane przez istniejącą infrastrukturę Prometheus organizacji (jeśli dostępna) lub dedykowaną instancję projektu.
- Logi w formacie JSON przekazywane do centralnego systemu logowania organizacji (np. ELK/Splunk — zależnie od standardu wewnętrznego, do potwierdzenia w Fazie 0).
- Alerting (Alertmanager/Grafana Alerting) skonfigurowany dla scenariuszy krytycznych: worker pool wyczerpany, wskaźnik błędów connectora powyżej progu, opóźnienie kolejki powyżej progu, niedostępność API.

8. Powiązanie z dalszą dokumentacją

- Wymagania нефunkcjonalne realizowane przez powyższą infrastrukturę — 05_NonFunctional_Requirements.md.
- Stack technologiczny i struktura kodu — 09_Python_Architecture.md.
- Diagram wdrożenia (formalny UML) — uml/DeploymentDiagram.puml.
- Plan fazowy wdrożenia produkcyjnego — 11_Roadmap.md.
- Ryzyka związane z infrastrukturą i operacjami — 12_Risks.md.

11. Roadmap

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Plan wdrożenia w fazach **Odbiorcy:** Project Manager, Sponsor, Architektura IT, Zespół Deweloperski

1. Podejście do wdrożenia

Wdrożenie ARGUS jest planowane **fazowo, w modelu przyrostowym (incremental delivery)**, minimalizując ryzyko i skracając czas do pierwszej wartości biznesowej. Każda faza dostarcza działający, użyteczny przyrost systemu, zamiast jednorazowego, długiego cyklu “big bang”. Kolejność faz odzwierciedla zależności techniczne zidentyfikowane w 00_Technical_Feasibility_Study.md (najpierw potwierdzenie fundamentów integracyjnych, dopiero potem skalowanie i funkcje zaawansowane).

2. Faza 0 — Discovery & Proof of Concept

Cel: Zweryfikować w praktyce kluczowe założenia techniczne z 00_Technical_Feasibility_Study.md na realnym środowisku organizacji, zanim podjęte zostaną nieodwracalne decyzje architektoniczne.

Zadanie	Rezultat
Potwierdzenie modelu wdrożenia Power BI (on-prem / hybrydowy) i dostępnych licencji (Premium/PPU)	Odpowiedź na otwarte pytania z 00_Technical_Feasibility_Study.md, sekcja 8
Potwierdzenie mechanizmu SSO (Kerberos/NTLM) i modelu konta technicznego	Zatwierdzony model autoryzacji dla connectorów
PoC: Browser Automation (Playwright) — otwarcie po jednym reprezentatywnym raporcie PBI i SSRS, odczyt przykładowego KPI	Potwierdzenie wykonalności technicznej warstwy integracyjnej
PoC: pomiar rzeczywistego czasu otwarcia/renderowania raportów	Kalibracja założeń wydajnościowych z 05_NonFunctional_Requirements.md
Ustalenie polityk sieciowych i dostępu do magazynu sekretów	Odblokowanie prac Fazy 1
Decyzja Security Office ws. dostawcy modelu AI (Azure OpenAI vs. on-prem)	Wejście do 06_AI_Module.md, sekcja 5.3
Akceptacja niniejszej dokumentacji architektonicznej przez Architekturę IT	Formalne rozpoczęcie developmentu

Kryterium wyjścia z Fazy 0: Wszystkie otwarte pytania z 00_Technical_Feasibility_Study.md, sekcja 8, mają udzieloną odpowiedź; PoC potwierdza wykonalność Browser Automation w środowisku organizacji.

3. Faza 1 — MVP (Minimum Viable Product)

Cel: Dostarczyć działający rdzeń systemu dla ograniczonego, reprezentatywnego podzbioru raportów (jeden obszar biznesowy), dowodzący pełnego cyklu: definicja testu → wykonanie → wynik → prezentacja.

Zakres	Szczegóły
Moduły rdzeniowe	Test Orchestrator, Report Platform Connector Layer (PowerBICConnector, SSRSCConnector — warstwa Browser Automation), Value Extraction Engine (Warstwa 0-1: API/model danych + DOM, bez CV/OCR), Validation Engine, SQL Validation Engine, Results Repository, Configuration/Validation Profile Repository
GUI	Katalog raportów (jeden obszar biznesowy), konfiguracja Validation Profile (podstawowa), przegląd wyników, historia
Harmonogram	Scheduler + wykonanie sekwencyjne/ograniczenie równoległości (bez pełnego autoscalingu)
Bezpieczeństwo	RBAC podstawowy, SSO, konto techniczne, magazyn sekretów
AI	Poza zakresem MVP (patrz Faza 3)
Audyt/Monitoring	Podstawowy log audytowy, podstawowe metryki (bez pełnego Grafana/OpenTelemetry)

Kryterium wyjścia: System automatycznie testuje wszystkie raporty jednego wybranego obszaru biznesowego, zgodnie z harmonogramem, z poprawnym porównaniem wartości raport↔SQL i prezentacją wyniku w GUI.

4. Faza 2 — Scale-out i pełna funkcjonalność testowa

Cel: Rozszerzenie na cały bieżący katalog (~300 raportów), pełna funkcjonalność definiowania testów i reguł biznesowych, wprowadzenie równoległości.

Zakres	Szczegóły
Rozszerzenie katalogu	Objęcie wszystkich obszarów biznesowych, pełna migracja/inwentaryzacja katalogu raportów
Test Definition Engine	Pełna walidacja definicji testów, klonowanie profili (FR-018), edytor testów SQL z podglądem
Business Rule Engine	Pełny zestaw typów reguł (sumy kontrolne, zakresy, trend)
Value Extraction Engine	Dodanie Warstwy 1b (eksport strukturalny) i Warstwy 2 (Computer Vision) fallbacku
Wykonanie równoległe	Worker pool Celery, konfigurowalna współbieżność, mechanizmy odpornościowe (retry, circuit breaker)
Notification Service	Pełna funkcjonalność (deduplikacja, reguły eskalacji)
Health Score / Confidence Score	Pełne wyliczanie i prezentacja trendów
Monitoring	Pełna integracja Prometheus/Grafana/OpenTelemetry

Kryterium wyjścia: Pełny katalog ~300 raportów objęty automatycznym, cyklicznym testowaniem, z równoległym wykonaniem i pełnym monitoringiem operacyjnym.

5. Faza 3 — Moduł AI

Cel: Wprowadzenie warstwy interpretacyjnej AI zgodnie z granicami opisanymi w 06_AI_Modułe.md.

Zakres	Szczegóły
AI Analysis Engine	Context Builder, LLM Orchestration Layer, Response Validator
Integracja z dostawcą modelu	Zgodnie z decyzją Fazy 0 (Azure OpenAI / on-prem)
Wyjaśnialność	Wymuszenie strukturalnej odpowiedzi z polem evidence, logowanie kontekstu wywołań (NFR-C-04)
Anomaly Detector	Detekcja odchyłeń od wzorca historycznego
Clustering Engine	Grupowanie podobnych błędów (FR-041)
GUI	Ekran AI Insights, wizualne oznaczenie treści AI (NFR-U-02), mechanizm feedbacku użytkownika
Guardraile bezpieczeństwa	Testy akceptacyjne potwierdzające niemożność zmiany statusu PASS/FAIL przez AI (FR-043)

Kryterium wyjścia: Moduł AI generuje wyjaśnialne interpretacje dla przebiegów FAIL/WARNING, zweryfikowane pod kątem zgodności z zasadą nadrzędną (AI nie decyduje), zaakceptowane przez Security Office.

6. Faza 4 — Pełne wdrożenie produkcyjne (skala 1000 raportów)

Cel: Osiągnięcie docelowej skali i pełnej gotowości operacyjnej klasy Enterprise.

Zakres	Szczegóły
Skalowanie infrastruktury	Pełny autoscaling worker pool, weryfikacja NFR-P-01 ($\leq 4h$ dla 1000 raportów) w warunkach zbliżonych do produkcyjnych
Wysoka dostępność	Pełna redundancja komponentów, weryfikacja SLA (NFR-A-01)
Disaster Recovery	Pełne wdrożenie i testy procedur DR (patrz 10_Deployment.md, sekcja 6)
Bezpieczeństwo	Pełny audyt bezpieczeństwa, testy penetracyjne (NFR-SEC-06)
Dokumentacja operacyjna	Runbooki dla zespołu utrzymania, procedury eskalacji
Rollout	Stopniowe włączanie kolejnych raportów/obszarów aż do pełnego pokrycia katalogu (docelowo ≥ 1000)

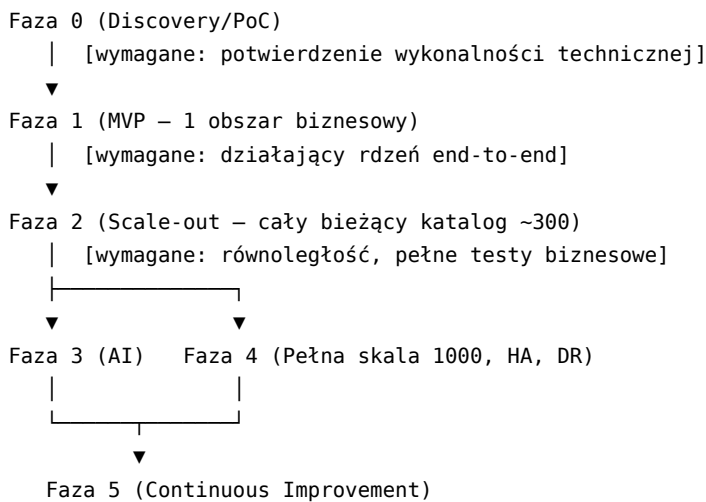
Kryterium wyjścia: System w pełni produkcyjny, obsługujący docelową skalę raportów, zaakceptowany przez wszystkich interesariuszy z 01_Project_Overview.md, sekcja 5.

7. Faza 5 — Ciągłe doskonalenie (Continuous Improvement)

Faza nieograniczona w czasie, obejmująca:

- Rozszerzanie katalogu raportów wraz z jego naturalnym wzrostem organizacyjnym (powyżej 1000).
- Iteracyjne doskonalenie promptów/modelu AI na bazie zebranego feedbacku użytkowników (FR-041, mechanizm oceny trafności sugestii).
- Ewentualne dodanie kolejnych connectorów platform raportowych zgodnie z potrzebami biznesowymi (architektonicznie przewidziane — patrz 03_Architecture.md, sekcja 6).
- Optymalizacja kosztowa (infrastruktura, wywołania AI) na bazie rzeczywistych danych operacyjnych.
- Regularny przegląd i aktualizacja rejestru ryzyk (12_Risks.md).

8. Zależności między fazami (skrót)



Fazy 3 i 4 mogą być realizowane częściowo równolegle przez odrębne strumienie zespołu, ponieważ nie mają twardej zależności technicznej między sobą (moduł AI jest architektonicznie non-blocking względem rdzenia systemu — patrz 06_AI_Module.md, sekcja 5.1) — decyzja o równoległej realizacji należy do Project Managera na bazie dostępności zasobów zespołu.

9. Powiązanie z dalszą dokumentacją

- Uzasadnienie biznesowe wdrożenia fazowego — 02_Business_Problem.md.
- Kryteria sukcesu projektu jako całości — 01_Project_Overview.md, sekcja 9.
- Ryzyka mogące wpłynąć na harmonogram powyższych faz — 12_Risks.md.

12. Risks

Projekt: ARGUS — Automated Reporting Governance & Unified Surveillance Platform **Dokument:** Rejestr ryzyk **Odbiorcy:** Sponsor, Project Manager, Architektura IT, Security Office

1. Metodologia

Każde ryzyko oceniane jest w skali **Prawdopodobieństwo × Wpływ** (Niskie / Średnie / Wysokie), z przypisanym właścicielem i strategią mitygacji. Rejestr podlega cyklicznemu przeglądowi (rekomendacja: na początku każdej fazy z 11_Roadmap.md oraz ad-hoc przy istotnych zmianach).

2. Rejestr ryzyk

ID	Ryzyko	Kategoria	Prawdop.	Wpływ	Mitygacja	Właściciel
R-01	Zmiana layoutu/struktury DOM raportu przez Microsoft (aktualizacja produktu Power BI/SSRS) psuje selektory ekstrakcji na dużą skalę	Techniczne	Średnie	Wysoki	Strategia wielowarstwowego fallbacku (00_Technical_Feasibility_Study.md, sekcja 7), selektory semantyczne, automatyczna detekcja podejrzanej zmiany strukturalnej, wersjonowanie Validation Profile	Zespół Deweloperski
R-02	Niedostępność/degradacja wydajności Platformy BI pod obciążeniem generowanym przez ARGUS (zbyt agresywny harmonogram testów)	Techniczne / Operacyjne	Średnie	Wysoki	Konfigurowalny limit współbieżności, Circuit Breaker, testy obciążeniowe w Fazie 0/2, harmonogramy poza godzinami szczytu tam, gdzie to możliwe	Zespół Deweloperski, Administrator Platformy BI

ID	Ryzyko	Kategoria	Prawdop.	Wpływ	Mitygacja	Właściciel
R-03	Brak licencji Premium/PPU uniemożliwiający wykorzystanie connectora XMLA dla części raportów	Techniczne / Licencyjne	Wysokie	Niski	Architektura nie zakłada zależności od XMLA jako warstwy obowiązkowej — Browser Automation pozostaje warstwą bazową uniwersalną niezależnie od licencji (00_Technical_Feasibility_Study.md, sekcja 4)	Architektura IT
R-04	Problemy z integracją SSO/Kerberos w kontekście headless przeglądarki (typowe wyzwanie w środowiskach on-prem z NTLM/Kerberos)	Techniczne	Średnie	Wysoki	Wczesna weryfikacja w Fazie 0 (PoC), ścisła współpraca z zespołem IAM, rezerwowy plan (dedykowane konto serwisowe z bezpośrednim dostępem, jeśli delegacja/impersonacja niedostępna)	Zespół Deweloperski, IAM
R-05	Niewystarczające zaangażowanie administratorów/analitików w definiowanie Validation Profiles — niskie pokrycie katalogu testami	Organizacyjne	Średnie	Wysoki	Uproszczony UX konfiguracji, klonowanie profili (FR-018), wsparcie AI w propozycji wstępnej konfiguracji (FR-019), jasna komunikacja wartości biznesowej, wsparcie sponsora w promowaniu adopcji	Project Manager, Właściciele obszarów biznesowych
R-06	Nadmierna liczba fałszywych alarmów (zbyt wąskie tolerancje / niska jakość ekstrakcji) prowadząca do utraty zaufania użytkowników biznesowych do systemu	Biznesowe / Techniczne	Średnie	Wysoki	Confidence Score jako wskaźnik jawnie prezentowany, iteracyjne strojenie tolerancji w Fazie 2, mechanizm feedbacku (ocena trafności) już od Fazy 3	Analitycy biznesowi, Zespół Deweloperski

ID	Ryzyko	Kategoria	Prawdop.	Wpływ	Mitygacja	Właściciel
R-07	Halucynacja lub niepożądana treść generowana przez moduł AI	Techniczne / Reputacyjne	Niskie-Średnie	Średni	Wymóg strukturalnej odpowiedzi z evidence, Response Validator, jednoznaczne oznaczenie treści AI, brak wpływu AI na status PASS/FAIL (06_AI_Module.md)	Zespół Deweloperski, Security Office
R-08	Wyciek danych wrażliwych do zewnętrznego dostawcy modelu AI	Bezpieczeństwo / Compliance	Niskie	Wysoki	Minimalizacja danych przekazywanych do modelu, kontraktowe gwarancje dostawcy, opcja wariantu w pełni on-prem, decyzja Security Office w Fazie 0	Security Office, Data Governance
R-09	Niedoszacowanie infrastruktury wykonawczej (worker pool) względem rzeczywistego czasu renderowania raportów przy skali 1000	Techniczne / Wydajnościowe	Średnie	Wysoki	Kalibracja empiryczna w PoC (Faza 0), autoscaling oparty o głębokość kolejki, monitoring trendu czasu wykonania od Fazy 1	Architektura IT, Zespół Infrastruktury
R-10	Opóźnienia w uzyskaniu dostępów sieciowych/kontowych w regulowanym środowisku bankowym	Organizacyjne / Proceduralne	Wysokie	Średni	Wczesne zgłoszenie wymagań dostępowych w Fazie 0, jasno zdefiniowana lista otwartych pytań (00_Technical_Feasibility_Study.md, sekcja 8) przekazana odpowiednim zespołom na starcie projektu	Project Manager

ID	Ryzyko	Kategoria	Prawdop.	Wpływ	Mitygacja	Właściciel
R-11	Konto techniczne o nadmiernych uprawnieniach staje się celem/ wektorem ataku	Bezpieczeństwo	Niskie	Wysoki	Least privilege (NFR-SEC-01), segmentacja sieciowa (NFR-SEC-08), zarządzany magazyn sekretów, regularny przegląd uprawnień	Security Office
R-12	Rozrost wolumenu danych historycznych degraduje wydajność zapytań/ Dashboardu szybciej niż zakładano	Techniczne	Średnie	Średni	Partycjonowanie od Fazy 2 (07_Data_Model.md, sekcja 4), monitoring wzrostu wolumenu, zdefiniowana polityka retencji/ archiwizacji	Zespół Deweloperski, DBA
R-13	Zmiana zakresu projektu w trakcie realizacji (np. objęcie dodatkowych platform raportowych wcześniej niż planowano)	Zarządcze	Średnie	Średni	Architektura Open/Closed minimalizuje koszt techniczny takiej zmiany (03_Architecture.md, sekcja 6); formalny proces zarządzania zmianą zakresu po stronie PM	Project Manager
R-14	Niewystarczające testy kontraktowe connectorów prowadzą do regresji przy aktualizacjach zależności (np. nowa wersja Playwright)	Techniczne / Jakościowe	Średnie	Średni	NFR-M-03 (obowiązkowe testy kontraktowe), pipeline CI blokujący merge przy regresji, kontrolowany proces aktualizacji zależności	Zespół Deweloperski

ID	Ryzyko	Kategoria	Prawdop.	Wpływ	Mitygacja	Właściciel
R-15	Rotacja/odejście kluczowych członków zespołu deweloper-skiego w trakcie wieloletniego projektu	Organizacyjne	Średnie	Średni	Kompletna, aktualizowana dokumentacja architektoniczna (niniejsze repozytorium) jako źródło wiedzy niezależne od osób, wysokie pokrycie testami ułatwiające onboarding, code review jako mechanizm transferu wiedzy	Project Manager, Tech Lead

3. Macierz Prawdopodobieństwo × Wpływ (skrót wizualny)

	Wpływ: Niski	Wpływ: Średni	Wpływ: Wysoki
Prawdop.: Wysokie	—	R-10	R-03*
Prawdop.: Średnie	—	R-12, R-13, R-14, R-15	R-01, R-02, R-04, R-05, R-06, R-09
Prawdop.: Niskie	—	R-07	R-08, R-11

* R-03 ma wysokie prawdopodobieństwo wystąpienia ograniczenia licencyjnego, ale niski wpływ, ponieważ architektura nie zakłada zależności od XMLA jako warstwy obowiązkowej.

Ryzyka w prawym górnym i środkowym górnym obszarze macierzy (wysoki wpływ przy średnim/wysokim prawdopodobieństwie: **R-01, R-02, R-04, R-05, R-06, R-09**) wymagają najwyższej uwagi zespołu projektowego i powinny być monitorowane aktywnie od Fazy 0.

4. Powiązanie z dalszą dokumentacją

- Mitygacje techniczne szczegółowo opisane w 00_Technical_Feasibility_Study.md (R-01, R-03) oraz 03_Architecture.md (R-13, R-14).
- Otwarte pytania Fazy 0 bezpośrednio adresujące ryzyka R-03, R-04, R-08, R-10 — 00_Technical_Feasibility_Study.md, sekcja 8.
- Wymagania niefunkcjonalne mitygujące ryzyka bezpieczeństwa i wydajności — 05_Non-Functional_Requirements.md.
- Harmonogram faz, w którym poszczególne ryzyka są aktywnie adresowane — 11_Roadmap.md.

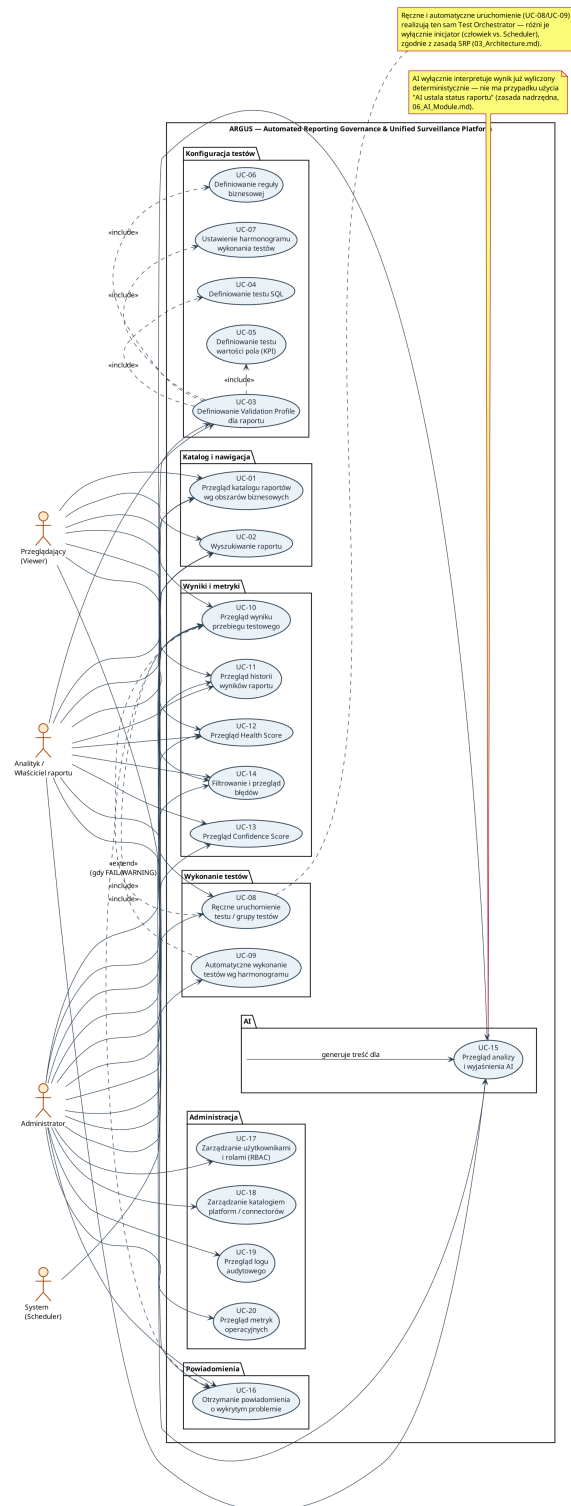
13. Diagramy UML

Poniższy rozdział zawiera wszystkie siedem diagramów UML zdefiniowanych dla projektu ARGUS. Każdy diagram jest zaprezentowany w trzech częściach: **(1)** opis merytoryczny — co diagram przedstawia, **(2)** uzasadnienie projektowe — dlaczego przyjęto taki, a nie inny sposób modelowania, oraz **(3)** wyrenderowany diagram wraz z pełnym kodem źródłowym PlantUML (katalog `uml/` w repozytorium). Diagramy nie korzystają z notacji Mermaid — zgodnie z wymogiem projektowym, wszystkie zostały przygotowane w PlantUML.

13.1 Diagram przypadków użycia (Use Case Diagram)

Opis. Diagram przedstawia pięciu aktorów systemu ARGUS (trzech ludzkich: Administrator, Analityk/Właściciel raportu, Przeglądający; oraz dwóch niebędących ludźmi: Scheduler i AI Analysis Engine) oraz kompletną listę 20 przypadków użycia (UC-01-UC-20) zdefiniowanych w dokumencie `04_Functional_Requirements.md`. Przypadki użycia są pogrupowane w pakiety odzwierciedlające obszary funkcjonalne: Katalog i nawigacja, Konfiguracja testów, Wykonanie testów, Wyniki i metryki, AI, Powiadomienia, Administracja.

Uzasadnienie projektowe. Rozdzielenie aktora "System (Scheduler)" od aktorów ludzkich jest celowe — podkreśla, że automatyczne wykonanie testów (UC-09) jest architektonicznie równoważne wykonaniu ręcznemu (UC-08), różniąc się wyłącznie inicjatorem, co odzwierciedla zasadę Single Responsibility opisaną w `03_Architecture.md` (Scheduler wyłącznie harmonogramuje, Test Orchestrator wykonuje). Aktor "AI Analysis Engine" jest wyodrębniony jako osobny aktor wspierający (a nie fragment aktora Systemu), aby wizualnie podkreślić na poziomie wymagań jego rolę interpretacyjną, spójną z zasadą nadrzędną modułu AI opisaną w `06_AI_Module.md` — AI dostarcza przypadek użycia UC-15, ale nie uczestniczy w UC-08/UC-09 (wykonanie i werdykt testu). Relacje `<>` pomiędzy UC-03 a UC-04/UC-05/UC-06/UC-07 pokazują, że zdefiniowanie kompletnego Validation Profile obowiązkowo obejmuje zdefiniowanie co najmniej jednego testu (FR-011 — raport bez testu nie jest objęty harmonogramem).



Rysunek 13.1: Diagram przypadków użycia (Use Case Diagram) — plik źródłowy `uml/Use-Case.puml`.

Pełny kod źródłowy PlantUML (uml/UseCase.puml):

```
' =====
' ARGUS – Diagram przypadków użycia (Use Case Diagram)
' =====
' OPIS:
' Diagram przedstawia pięciu aktorów systemu ARGUS (trzech ludzkich: Administrator,
' Analityk/Właściciel raportu, Przeglądający; oraz dwóch niebędących ludźmi:
' Scheduler i AI Analysis Engine) oraz kompletną listę 20 przypadków użycia
' (UC-01–UC-20) zdefiniowanych w dokumencie 04_Functional_Requirements.md.
' Przypadki użycia są pogrupowane w pakiety odzwierciedlające obszary
' funkcjonalne: Katalog i nawigacja, Konfiguracja testów, Wykonanie testów,
' Wyniki i metryki, AI, Powiadomienia, Administracja.
'
' UZASADNIENIE PROJEKTOWE:
' Rozdzielenie aktora "System (Scheduler)" od aktorów ludzkich jest celowe –
' podkreśla, że automatyczne wykonanie testów (UC-09) jest architektonicznie
' równoważne wykonaniu ręcznemu (UC-08), różniąc się wyłącznie inicjatorem,
' co odzwierciedla zasadę Single Responsibility opisaną w 03_Architecture.md
' (Scheduler wyłącznie harmonogramuje, Test Orchestrator wykonuje).
' Aktor "AI Analysis Engine" jest wyodrębniony jako osobny aktor wspierający
' (a nie fragment aktora Systemu), aby wizualnie podkreślić na poziomie
' wymagań jego rolę interpretacyjną, spójną z zasadą nadrzędną modułu AI
' opisaną w 06_AI_Module.md – AI dostarcza przypadek użycia UC-15, ale nie
' uczestniczy w UC-08/UC-09 (wykonanie i werdykt testu).
' Relacje <<include>> pomiędzy UC-03 a UC-04/UC-05/UC-06/UC-07 pokazują, że
' zdefiniowanie kompletnego Validation Profile obowiązkowo obejmuje
' zdefiniowanie co najmniej jednego testu (FR-011 – raport bez testu nie
' jest objęty harmonogramem).
' =====

@startuml UseCase

skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 12
skinparam ArrowColor #2C3E50
skinparam usecase {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
    FontColor #1B2631
}
skinparam actor {
    BackgroundColor #FDEBD0
    BorderColor #A04000
}
skinparam packageStyle rectangle
left to right direction

actor "Administrator" as Admin
actor "Analityk /\nWłaściciel raportu" as Analyst
actor "Przeglądający\n(Viewer)" as Viewer
actor "System\n(Scheduler)" as Scheduler
actor "AI Analysis\nEngine" as AI

rectangle "ARGUS – Automated Reporting Governance & Unified Surveillance Platform" {

    package "Katalog i nawigacja" {
        usecase "UC-01\nPrzegląd katalogu raportów\nnwg obszarów biznesowych" as UC01
        usecase "UC-02\nWyszukiwanie raportu" as UC02
    }
}
```

```

package "Konfiguracja testów" {
    usecase "UC-03\nDefiniowanie Validation Profile\ndla raportu" as UC03
    usecase "UC-04\nDefiniowanie testu SQL" as UC04
    usecase "UC-05\nDefiniowanie testu\nwartości pola (KPI)" as UC05
    usecase "UC-06\nDefiniowanie reguły\nbiznesowej" as UC06
    usecase "UC-07\nUstawienie harmonogramu\nwykonania testów" as UC07
}

package "Wykonanie testów" {
    usecase "UC-08\nRęczne uruchomienie\ntestu / grupy testów" as UC08
    usecase "UC-09\nAutomatyczne wykonanie\ntestów wg harmonogramu" as UC09
}

package "Wyniki i metryki" {
    usecase "UC-10\nPrzegląd wyniku\nprzebiegu testowego" as UC10
    usecase "UC-11\nPrzegląd historii\nwyników raportu" as UC11
    usecase "UC-12\nPrzegląd Health Score" as UC12
    usecase "UC-13\nPrzegląd Confidence Score" as UC13
    usecase "UC-14\nFiltrowanie i przegląd\nbłędów" as UC14
}

package "AI" {
    usecase "UC-15\nPrzegląd analizy\ni wyjaśnienia AI" as UC15
}

package "Powiadomienia" {
    usecase "UC-16\nOtrzymanie powiadomienia\no wykrytym problemie" as UC16
}

package "Administracja" {
    usecase "UC-17\nZarządzanie użytkownikami\ni rolami (RBAC)" as UC17
    usecase "UC-18\nZarządzanie katalogiem\nplatform / connectorów" as UC18
    usecase "UC-19\nPrzegląd logu\naudytowego" as UC19
    usecase "UC-20\nPrzegląd metryk\noperacyjnych" as UC20
}
}

Viewer --> UC01
Viewer --> UC02
Viewer --> UC10
Viewer --> UC11
Viewer --> UC12
Viewer --> UC14
Viewer --> UC15

Analyst --> UC01
Analyst --> UC02
Analyst --> UC03
Analyst --> UC08
Analyst --> UC10
Analyst --> UC11
Analyst --> UC12
Analyst --> UC13
Analyst --> UC14
Analyst --> UC15
Analyst --> UC16

Admin --> UC01
Admin --> UC02
Admin --> UC03
Admin --> UC08

```

```
Admin --> UC10
Admin --> UC11
Admin --> UC12
Admin --> UC13
Admin --> UC14
Admin --> UC15
Admin --> UC16
Admin --> UC17
Admin --> UC18
Admin --> UC19
Admin --> UC20
```

```
Scheduler --> UC09
```

```
UC03 .> UC04 : <<include>>
UC03 .> UC05 : <<include>>
UC03 .> UC06 : <<include>>
UC03 .> UC07 : <<include>>
```

```
UC09 .> UC10 : <<include>>
UC08 .> UC10 : <<include>>
UC10 .> UC16 : <<extend>>\n(gdy FAIL/WARNING)
```

```
AI --> UC15 : generuje treść dla
```

```
note right of UC08
```

```
    Ręczne i automatyczne uruchomienie (UC-08/UC-09)
    realizują ten sam Test Orchestrator – różni je
    wyłącznie inicjator (człowiek vs. Scheduler),
    zgodnie z zasadą SRP (03_Architecture.md).
```

```
end note
```

```
note bottom of UC15
```

```
    AI wyłącznie interpretuje wynik już wyliczony
    deterministycznie – nie ma przypadku użycia
    "AI ustala status raportu" (zasada nadrzędna,
    06_AI_Module.md).
```

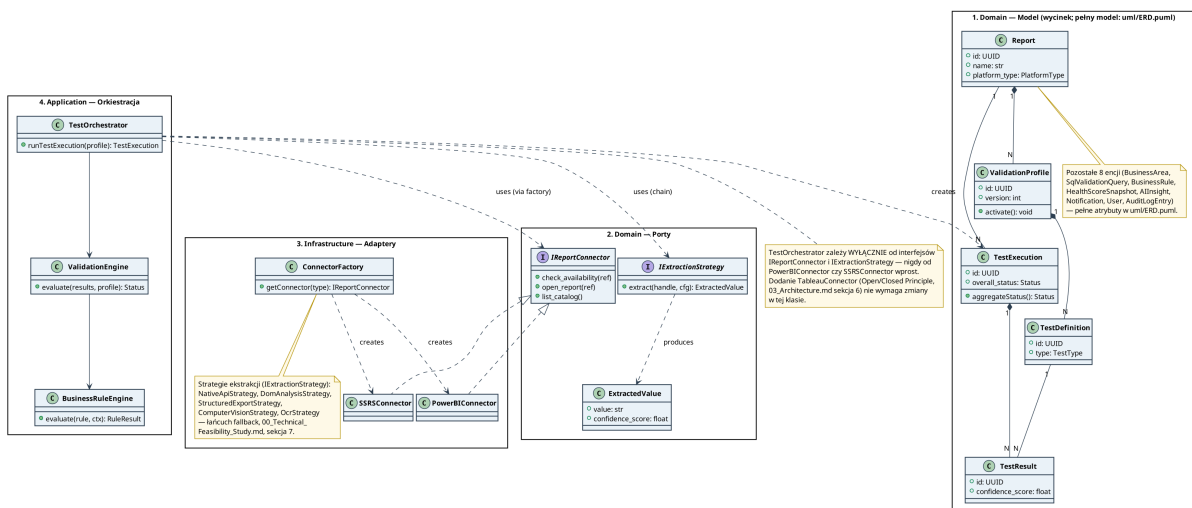
```
end note
```

```
@enduml
```

13.2 Diagram klas (Class Diagram)

Opis. Diagram celowo NIE powtarza kompletnego modelu danych (13 encji z pełnymi atrybutami — to jest wyczerpująco pokazane w `uml/ERD.puml` oraz `07_Data_Model.md`), lecz koncentruje się na strukturze obiektowej i wzorcach projektowych architektury ARGUS, w czterech ułożonych pionowo pasmach: (1) reprezentatywny wycinek modelu domenowego (encje kluczowe dla zrozumienia cyklu życia testu: `Report` → `ValidationProfile` → `TestDefinition` → `TestExecution` → `TestResult`); (2) porty domenowe — interfejsy `IReportConnector` i `IExtractionStrategy`; (3) adaptery infrastrukturalne implementujące te porty dla Power BI i SSRS oraz zestaw strategii ekstrakcji (Strategy Pattern); (4) warstwę aplikacyjną (`TestOrchestrator`, `ValidationEngine`, `BusinessRuleEngine`) spinającą całość w jeden przepływ.

Uzasadnienie projektowe. Interfejsy `IReportConnector` oraz `IExtractionStrategy` są celowo umieszczone w warstwie Domain (jako porty), a ich implementacje (`PowerBICConnector`, `SSRSConnector`, `DomAnalysisStrategy` itd.) w warstwie Infrastructure — zgodnie z zasadą Dependency Inversion (SOLID) oraz regułą zależności Clean Architecture (`03_Architecture.md`, sekcja 2): zależność wskazuje zawsze “do wewnątrz” — `TestOrchestrator` zależy od abstrakcji `IReportConnector`, nigdy od konkretnej klasy `PowerBICConnector`. `ConnectorFactory` realizuje wzorzec Registry + Factory (`03_Architecture.md`, sekcja 1.5) — dodanie nowej platformy (np. `TableauConnector`) wymaga wyłącznie nowej klasy implementującej `IReportConnector` i wpisu w fabryce, bez zmian w `TestOrchestrator` (Open/Closed Principle), co ten diagram odzwierciedla brakiem jakiegokolwiek zależności `TestOrchestrator` → konkretna implementacja connectora. Kompozycja `ValidationProfile` *— `TestDefinition` (a nie zwykła asocjacja) podkreśla, że testy nie mają sensu bytu poza swoim profilem — usunięcie/ wersjonowanie profilu pociąga za sobą cykl życia testów. Diagram jest celowo ułożony w czterech poziomych pasmach (od góry: Model domenowy → Porty → Adaptory → Aplikacja), a nie w kolumnach obok siebie — układ pionowy jest czytelny na stronie A4 dokumentu PDF i lepiej odwzorowuje kierunek zależności Clean Architecture (`03_Architecture.md`, sekcja 2). Redukcja liczby klas względem pełnego modelu danych jest świadomym kompromisem czytelności — cel tego diagramu to zrozumienie WZORCÓW i STRUKTURY, nie inwentaryzacja pól (do tego służy `ERD.puml`).



Rysunek 13.2: Diagram klas (Class Diagram) — plik źródłowy `uml/ClassDiagram.puml`.

Pełny kod źródłowy PlantUML (uml/ClassDiagram.puml):

```

' =====
' ARGUS – Diagram klas (Class Diagram)
' =====
' OPIS:
' Diagram celowo NIE powtarza kompletnego modelu danych (13 encji z pełnymi
' atrybutami – to jest wyczerpująco pokazane w uml/ERD.puml oraz
' 07_Data_Model.md), lecz koncentruje się na strukturze obiektowej i wzorcach
' projektowych architektury ARGUS, w czterech ułożonych pionowo pasmach:
' (1) reprezentatywny wycinek modelu domenowego (encje kluczowe dla
' zrozumienia cyklu życia testu: Report → ValidationProfile →
' TestDefinition → TestExecution → TestResult);
' (2) porty domenowe – interfejsy IReportConnector i IExtractionStrategy;
' (3) adaptery infrastrukturalne implementujące te porty dla Power BI i SSRS
' oraz zestaw strategii ekstrakcji (Strategy Pattern);
' (4) warstwę aplikacyjną (TestOrchestrator, ValidationEngine,
' BusinessRuleEngine) spinającą całość w jeden przepływ.
'
' UZASADNIENIE PROJEKTOWE:
' Interfejsy IReportConnector oraz IExtractionStrategy są celowo umieszczone
' w warstwie Domain (jako porty), a ich implementacje (PowerBIConnector,
' SSRSConnector, DomAnalysisStrategy itd.) w warstwie Infrastructure –
' zgodnie z zasadą Dependency Inversion (SOLID) oraz regułą zależności
' Clean Architecture (03_Architecture.md, sekcja 2): zależność wskazuje
' zawsze "do wewnątrz" – TestOrchestrator zależy od abstrakcji
' IReportConnector, nigdy od konkretnej klasy PowerBIConnector.
' ConnectorFactory realizuje wzorzec Registry + Factory (03_Architecture.md,
' sekcja 1.5) – dodanie nowej platformy (np. TableauConnector) wymaga
' wyłącznie nowej klasy implementującej IReportConnector i wpisu w fabryce,
' bez zmian w TestOrchestrator (Open/Closed Principle), co ten diagram
' odzwierciedla brakiem jakiegokolwiek zależności TestOrchestrator →
' konkretna implementacja connectora.
' Kompozycja ValidationProfile *-- TestDefinition (a nie zwykła asocjacja)
' podkreśla, że testy nie mają sensu bytu poza swoim profilem – usunięcie/
' wersjonowanie profilu pociąga za sobą cykl życia testów.
' Diagram jest celowo ułożony w czterech poziomych pasmach (od góry: Model
' domenowy → Porty → Adaptory → Aplikacja), a nie w kolumnach obok siebie –
' układ pionowy jest czytelny na stronie A4 dokumentu PDF i lepiej odwzorowuje
' kierunek zależności Clean Architecture (03_Architecture.md, sekcja 2).
' Redukcja liczby klas względem pełnego modelu danych jest świadomym
' kompromisem czytelności – cel tego diagramu to zrozumienie WZORCÓW
' i STRUKTURY, nie inwentaryzacja pól (do tego służy ERD.puml).
' =====

```

```
@startuml ClassDiagram
```

```

skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 12
skinparam classFontStyle bold
skinparam nodesep 20
skinparam ranksep 40
skinparam class {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
    ArrowColor #2C3E50
}
skinparam interface {
    BackgroundColor #FCF3CF
    BorderColor #7D6608

```

```

}
skinparam note {
    BackgroundColor #FEF9E7
    BorderColor #B7950B
}
skinparam packageStyle rectangle

package "1. Domain – Model (wycinek; pełny model: uml/ERD.puml)" as PKG1 {
    class Report {
        +id: UUID
        +name: str
        +platform_type: PlatformType
    }

    class ValidationProfile {
        +id: UUID
        +version: int
        +activate(): void
    }

    class TestDefinition {
        +id: UUID
        +type: TestType
    }

    class TestExecution {
        +id: UUID
        +overall_status: Status
        +aggregateStatus(): Status
    }

    class TestResult {
        +id: UUID
        +confidence_score: float
    }

    Report "1" *-- "N" ValidationProfile
    ValidationProfile "1" *-- "N" TestDefinition
    Report "1" -- "N" TestExecution
    TestExecution "1" *-- "N" TestResult
    TestDefinition "1" -- "N" TestResult

    note bottom of Report
        Pozostałe 8 encji (BusinessArea,
        SqlValidationQuery, BusinessRule,
        HealthScoreSnapshot, AIInsight,
        Notification, User, AuditLogEntry)
        – pełne atrybuty w uml/ERD.puml.
    end note
}

package "2. Domain – Porty" as PKG2 {
    interface IReportConnector {
        +check_availability(ref)
        +open_report(ref)
        +list_catalog()
    }

    interface IExtractionStrategy {
        +extract(handle, cfg): ExtractedValue
    }
}

```

```

class ExtractedValue {
    +value: str
    +confidence_score: float
}

IExtractionStrategy ..> ExtractedValue : produces
}

package "3. Infrastructure – Adaptery" as PKG3 {
    class PowerBICollector implements IReportCollector
    class SSRSConnector implements IReportCollector
    class ConnectorFactory {
        +getConnector(type): IReportCollector
    }

    ConnectorFactory ..> PowerBICollector : creates
    ConnectorFactory ..> SSRSConnector : creates

    note bottom of ConnectorFactory
        Strategie ekstrakcji (IExtractionStrategy):
        NativeApiStrategy, DomAnalysisStrategy,
        StructuredExportStrategy,
        ComputerVisionStrategy, OcrStrategy
        – łańcuch fallback, 00_Technical_
        Feasibility_Study.md, sekcja 7.
    end note
}

package "4. Application – Orkiestracja" as PKG4 {
    class TestOrchestrator {
        +runTestExecution(profile): TestExecution
    }

    class ValidationEngine {
        +evaluate(results, profile): Status
    }

    class BusinessRuleEngine {
        +evaluate(rule, ctx): RuleResult
    }

    TestOrchestrator ..> IReportCollector : uses (via factory)
    TestOrchestrator ..> IExtractionStrategy : uses (chain)
    TestOrchestrator --> ValidationEngine
    ValidationEngine --> BusinessRuleEngine
    TestOrchestrator ..> TestExecution : creates
}

' --- Wymuszenie ścisłego pionowego układu czterech pasm ---
TestResult -[hidden]d- IReportCollector
IReportCollector -[hidden]d- ExtractedValue
ExtractedValue -[hidden]d- PowerBICollector
ConnectorFactory -[hidden]d- TestOrchestrator

note as N1
    TestOrchestrator zależy WYŁĄCZNIE od interfejsów
    IReportCollector i IExtractionStrategy – nigdy od
    PowerBICollector czy SSRSConnector wprost.
    Dodanie TableauConnector (Open/Closed Principle,
    03_Architecture.md sekcja 6) nie wymaga zmiany
    w tej klasie.
end note

```

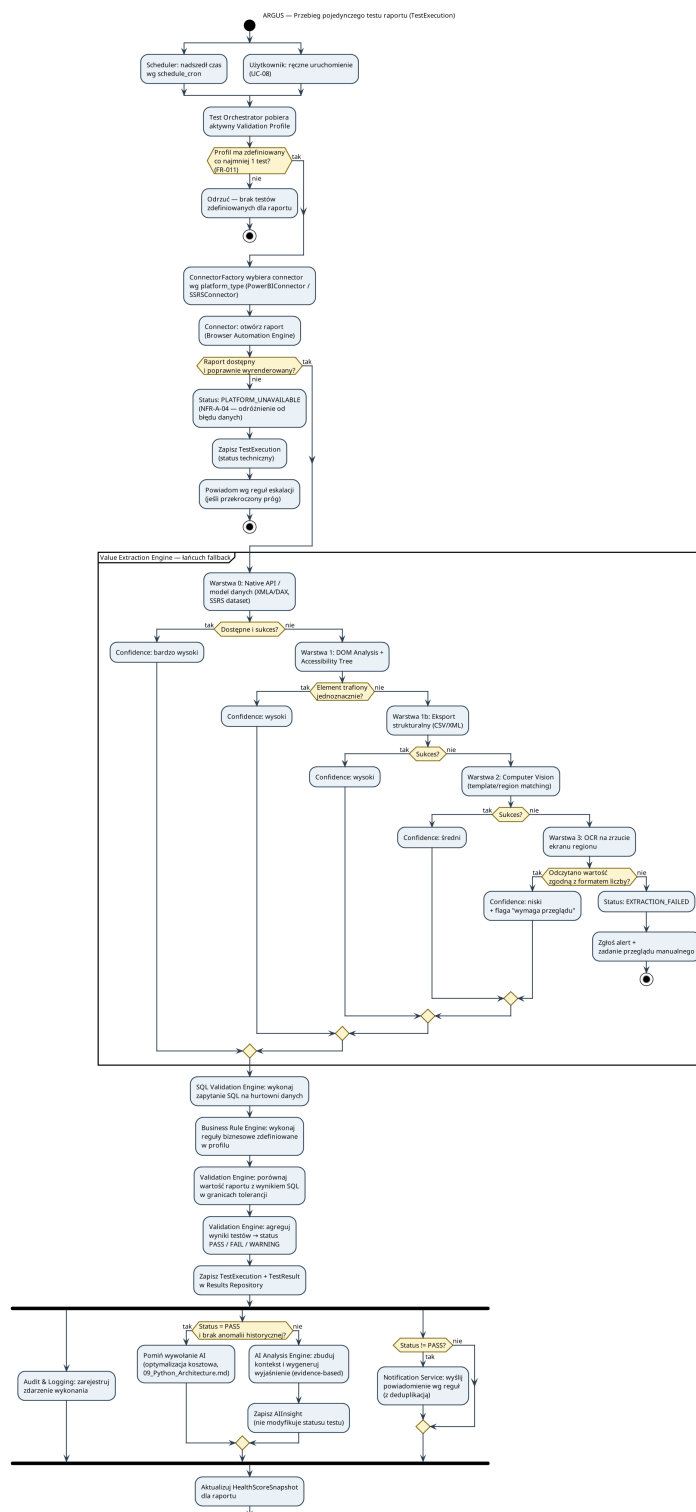
TestOrchestrator .. N1

@enduml

13.3 Diagram aktywności (Activity Diagram)

Opis. Diagram przedstawia pełny przepływ pojedynczego przebiegu testowego (TestExecution) — od wyzwolenia (harmonogram lub ręcznie) przez otwarcie raportu, wielowarstwową strategię ekstrakcji wartości KPI (fallback opisany w 00_Technical_Feasibility_Study.md, sekcja 7), walidację SQL i reguł biznesowych, aż po zapis wyniku, opcjonalną analizę AI i powiadomienie.

Uzasadnienie projektowe. Diagram celowo eksponuje jako osobną, wczesną decyzję “Czy raport dostępny i wyrenderowany poprawnie?” — rozdzielenie testu dostępności/renderowania od testu wartości KPI odzwierciedla wymóg NFR-A-04 (odróżnienie awarii platformy od błędu danych, aby uniknąć fałszywych alarmów o “błędym raporcie”, gdy w rzeczywistości to platforma BI jest niedostępna). Pętla wielowarstwowego fallbacku ekstrakcji jest modelowana jako aktywność strukturalna z decyzjami sekwencyjnymi (a nie jedna aktywność “wyodrębnij wartość”), aby jawnie pokazać zespołowi deweloperskiemu kolejność strategii i punkty, w których ustalany jest Confidence Score — kluczowe dla implementacji Value Extraction Engine zgodnie z 00_Technical_Feasibility_Study.md. Wywołanie AI Analysis Engine jest umieszczone jako aktywność równoległa (fork) względem zapisu wyniku i wysłania powiadomienia podstawowego, odzwierciedlając architektoniczną zasadę non-blocking z 06_AI_Module.md, sekcja 5.1 — niedostępność AI nie wstrzymuje zapisu wyniku testu.



Rysunek 13.3: Diagram aktywności (Activity Diagram) — plik źródłowy `uml/ActivityDiagram.puml`.

Pełny kod źródłowy PlantUML (uml/ActivityDiagram.puml):

```
' =====
' ARGUS – Diagram aktywności (Activity Diagram)
' =====
' OPIS:
' Diagram przedstawia pełny przepływ pojedynczego przebiegu testowego
' (TestExecution) – od wyzwolenia (harmonogram lub ręcznie) przez otwarcie
' raportu, wielowarstwową strategię ekstrakcji wartości KPI (fallback opisany
' w 00_Technical_Feasibility_Study.md, sekcja 7), walidację SQL i reguł
' biznesowych, aż po zapis wyniku, opcjonalną analizę AI i powiadomienie.
'
' UZASADNIENIE PROJEKTOWE:
' Diagram celowo eksponuje jako osobną, wczesną decyzję "Czy raport dostępny
' i wyrenderowany poprawnie?" – rozdzielenie testu dostępności/renderowania
' od testu wartości KPI odzwierciedla wymóg NFR-A-04 (odróżnienie awarii
' platformy od błędu danych, aby uniknąć fałszywych alarmów o "błędym
' raporcie", gdy w rzeczywistości to platforma BI jest niedostępna).
' Pętla wielowarstwowego fallbacku ekstrakcji jest modelowana jako aktywność
' strukturalna z decyzjami sekwencyjnymi (a nie jedna aktywność "wyodrębnij
' wartość"), aby jawnie pokazać zespołowi deweloperskiemu kolejność strategii
' i punkty, w których ustalany jest Confidence Score – kluczowe dla
' implementacji Value Extraction Engine zgodnie z 00_Technical_Feasibility_
' Study.md.
' Wywołanie AI Analysis Engine jest umieszczone jako aktywność równoległa
' (fork) względem zapisu wyniku i wysłania powiadomienia podstawowego,
' odzwierciedlając architektoniczną zasadę non-blocking z 06_AI_Module.md,
' sekcja 5.1 – niedostępność AI nie wstrzymuje zapisu wyniku testu.
' =====
```

```
@startuml ActivityDiagram
```

```
skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 11
skinparam activity {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
    DiamondBackgroundColor #FCF3CF
    DiamondBorderColor #7D6608
}
skinparam ArrowColor #2C3E50
```

```
title ARGUS – Przebieg pojedynczego testu raportu (TestExecution)
```

```
start
```

```
split
    :Scheduler: nadszedł czas\nwg schedule_cron;
split again
    :Użytkownik: ręczne uruchomienie\n(UC-08);
end split
```

```
:Test Orchestrator pobiera\naktywny Validation Profile;
```

```
if (Profil ma zdefiniowany\nco najmniej 1 test?\n(FR-011)) then (nie)
    :Odrzuć – brak testów\nzdefiniowanych dla raportu;
    stop
else (tak)
endif
```

```

:ConnectorFactory wybiera connector\nwg platform_type (PowerBIConnector /\nSSRSConnector);

:Connector: otwórz raport\n(Browser Automation Engine);

if (Raport dostępny\ni poprawnie wyrenderowany?) then (nie)
  :Status: PLATFORM_UNAVAILABLE\n(NFR-A-04 – odróżnienie od\nbłędu danych);
  :Zapisz TestExecution\n(status techniczny);
  :Powiedom wg reguł eskalacji\n(jeśli przekroczony próg);
  stop
else (tak)
endif

partition "Value Extraction Engine – łańcuch fallback" {
  :Warstwa 0: Native API /\nmodel danych (XMLA/DAX,\nSSRS dataset);
  if (Dostępne i sukces?) then (tak)
    :Confidence: bardzo wysoki;
  else (nie)
    :Warstwa 1: DOM Analysis +\nAccessibility Tree;
    if (Element trafiony\njednoznacznie?) then (tak)
      :Confidence: wysoki;
    else (nie)
      :Warstwa 1b: Eksport\nstrukturalny (CSV/XML);
      if (Sukces?) then (tak)
        :Confidence: wysoki;
      else (nie)
        :Warstwa 2: Computer Vision\n(template/region matching);
        if (Sukces?) then (tak)
          :Confidence: średni;
        else (nie)
          :Warstwa 3: OCR na zrzucie\nekranu regionu;
          if (Odczytano wartość\nzgodną z formatem liczby?) then (tak)
            :Confidence: niski\n+ flaga "wymaga przeglądu";
          else (nie)
            :Status: EXTRACTION_FAILED;
            :Zgłoś alert +\nzadanie przeglądu manualnego;
            stop
          endif
        endif
      endif
    endif
  endif
endif
endif
}

:SQL Validation Engine: wykonaj\nzapytanie SQL na hurtowni danych;

:Business Rule Engine: wykonaj\nreguły biznesowe zdefiniowane\nw profilu;

:Validation Engine: porównaj\nwartość raportu z wynikiem SQL\nw granicach tolerancji;

:Validation Engine: agreguj\nwyniki testów → status\nPASS / FAIL / WARNING;

:Zapisz TestExecution + TestResult\nw Results Repository;

fork
  :Audit & Logging: zarejestruj\nzdarzenie wykonania;
fork again
  if (Status = PASS\ni brak anomalii historycznej?) then (tak)
    :Pomiń wywołanie AI\n(optimalizacja kosztowa,\n09_Python_Architecture.md);
  else (nie)
    :AI Analysis Engine: zbuduj\nkontekst i wygeneruj\nwyjaśnienie (evidence-based);
    :Zapisz AIInsight\n(nie modyfikuje statusu testu);
  endif

```

```
fork again
  if (Status != PASS?) then (tak)
    :Notification Service: wyślij\npowiadomienie wg reguł\n(z deduplikacją);
  else (nie)
  endif
end fork

:Aktualizuj HealthScoreSnapshot\ndla raportu;

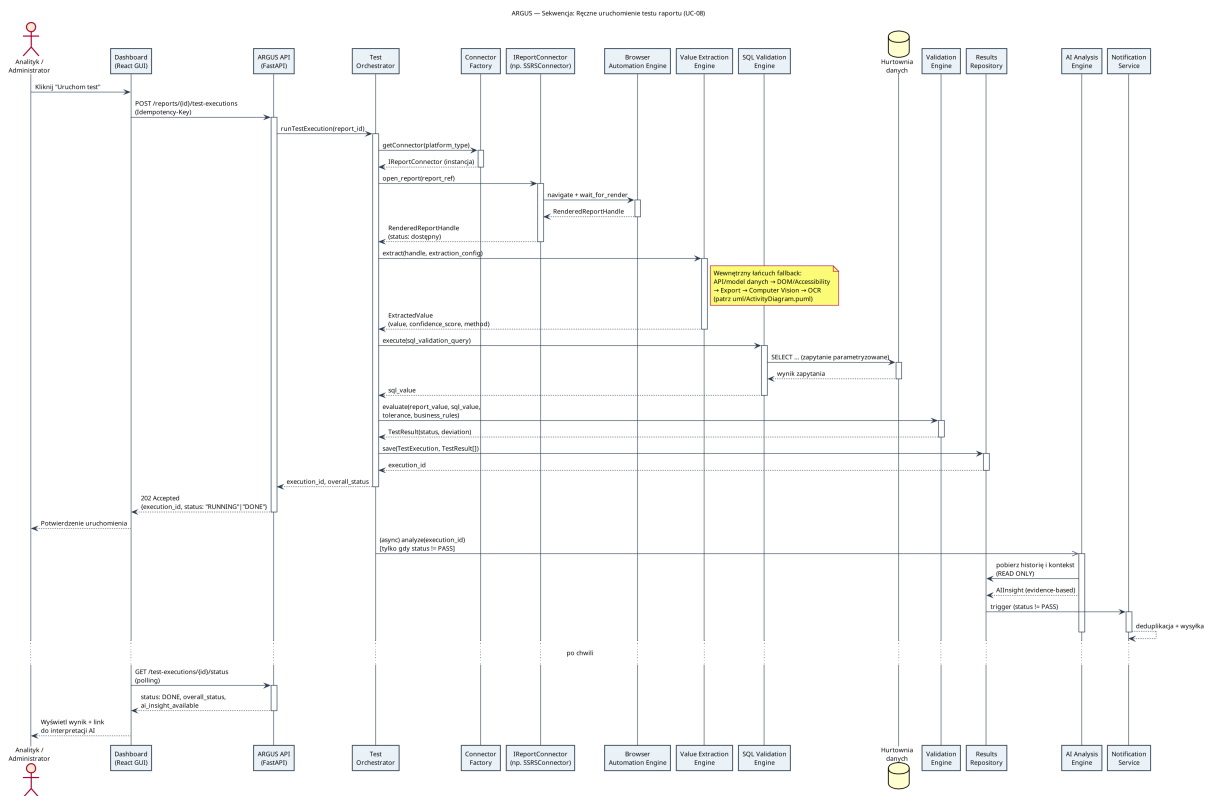
stop

@enduml
```

13.4 Diagram sekwencji (Sequence Diagram)

Opis. Diagram przedstawia scenariusz “Ręczne uruchomienie testu dla raportu” (UC-08, POST /api/v1/reports/{report_id}/test-executions z 08_API.md), pokazując pełną interakcję między komponentami: GUI → API → Test Orchestrator → warstwa connectorów/ekstrakcji → silniki walidacji → repozytorium wyników → moduł AI (asynchronicznie) → Notification Service, aż po odświeżenie statusu w GUI przez odpytywanie (polling).

Uzasadnienie projektowe. Wywołanie AI Analysis Engine jest zamodelowane jako komunikat asynchroniczny (strzałka otwarta, “-»” z activate/deactivate poza główną osią czasu odpowiedzi HTTP) — celowo, aby pokazać, że odpowiedź API do użytkownika (202 Accepted / status przebiegu) NIE czeka na zakończenie generowania interpretacji AI, zgodnie z zasadą non-blocking z 06_AI_Module.md. Interakcja z ConnectorFactory jest pokazana jako osobny krok (a nie bezpośrednie wywołanie konkretnego connectora przez Test Orchestrator), aby zilustrować w praktyce Dependency Inversion — Test Orchestrator zna wyłącznie interfejs IReportConnector, wybór konkretnej implementacji (Power BI / SSRS) jest hermetyzowany w fabryce (03_Architecture.md, sekcja 3.1). Test Definition Engine jest pokazany jako wywołany wcześniej, w momencie zapisu Validation Profile (nie w tym scenariuszu wykonania) — dla czytelności pominięty na tym diagramie i pokazany domyślnie w opisie tekstowym UC-03/UC-04 w 04_Functional_Requirements.md.



Rysunek 13.4: Diagram sekwencji (Sequence Diagram) — plik źródłowy `uml/SequenceDiagram.puml`.

Pełny kod źródłowy PlantUML (uml/SequenceDiagram.puml):

```
' =====
' ARGUS – Diagram sekwencji (Sequence Diagram)
' =====
' OPIS:
' Diagram przedstawia scenariusz "Ręczne uruchomienie testu dla raportu"
' (UC-08, POST /api/v1/reports/{report_id}/test-executions z 08_API.md),
' pokazując pełną interakcję między komponentami: GUI → API → Test
' Orchestrator → warstwa connectorów/ekstrakcji → silniki walidacji →
' repozytorium wyników → moduł AI (asynchronicznie) → Notification Service,
' aż po odświeżenie statusu w GUI przez odpytywanie (polling).
'
' UZASADNIENIE PROJEKTOWE:
' Wywołanie AI Analysis Engine jest zamodelowane jako komunikat asynchroniczny
' (strzałka otwarta, "->>" z activate/deactivate poza główną osią czasu
' odpowiedzi HTTP) – celowo, aby pokazać, że odpowiedź API do użytkownika
' (202 Accepted / status przebiegu) NIE czeka na zakończenie generowania
' interpretacji AI, zgodnie z zasadą non-blocking z 06_AI_Module.md.
' Interakcja z ConnectorFactory jest pokazana jako osobny krok (a nie
' bezpośrednie wywołanie konkretnego connectora przez Test Orchestrator),
' aby zilustrować w praktyce Dependency Inversion – Test Orchestrator zna
' wyłącznie interfejs IReportConnector, wybór konkretnej implementacji
' (Power BI / SSRS) jest hermetyzowany w fabryce (03_Architecture.md,
' sekcja 3.1). Test Definition Engine jest pokazany jako wywoływany
' wcześniej, w momencie zapisu Validation Profile (nie w tym scenariuszu
' wykonania) – dla czytelności pominięty na tym diagramie i pokazany
' domyślnie w opisie tekstowym UC-03/UC-04 w 04_Functional_Requirements.md.
' =====
```

```
@startuml SequenceDiagram
```

```
skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 11
skinparam sequence {
    ArrowColor #2C3E50
    LifelineBorderColor #2C3E50
    ParticipantBackgroundColor #EAF2F8
    ParticipantBorderColor #2C3E50
    ActorBackgroundColor #FDEBD0
}
```

```
title ARGUS – Sekwencja: Ręczne uruchomienie testu raportu (UC-08)
```

```
actor "Analityk /\nAdministrator" as User
participant "Dashboard\n(React GUI)" as GUI
participant "ARGUS API\n(FastAPI)" as API
participant "Test\nOrchestrator" as Orch
participant "Connector\nFactory" as Factory
participant "IReportConnector\n(np. SSRSConnector)" as Connector
participant "Browser\nAutomation Engine" as Browser
participant "Value Extraction\nEngine" as Extract
participant "SQL Validation\nEngine" as SqlEngine
database "Hurtownia\ndanych" as DWH
participant "Validation\nEngine" as Valid
participant "Results\nRepository" as Results
participant "AI Analysis\nEngine" as AI
participant "Notification\nService" as Notify
```

```
User -> GUI : Kliknij "Uruchom test"
```

```
GUI -> API : POST /reports/{id}/test-executions\n(Idempotency-Key)
activate API

API -> Orch : runTestExecution(report_id)
activate Orch

Orch -> Factory : getConnector(platform_type)
activate Factory
Factory --> Orch : IReportConnector (instancja)
deactivate Factory

Orch -> Connector : open_report(report_ref)
activate Connector
Connector -> Browser : navigate + wait_for_render
activate Browser
Browser --> Connector : RenderedReportHandle
deactivate Browser
Connector --> Orch : RenderedReportHandle\n(status: dostępny)
deactivate Connector

Orch -> Extract : extract(handle, extraction_config)
activate Extract
note right of Extract
    Wewnętrzny łańcuch fallback:
    API/model danych → DOM/Accessibility
    → Export → Computer Vision → OCR
    (patrz uml/ActivityDiagram.puml)
end note
Extract --> Orch : ExtractedValue\n(value, confidence_score, method)
deactivate Extract

Orch -> SqlEngine : execute(sql_validation_query)
activate SqlEngine
SqlEngine -> DWH : SELECT ... (zapytanie parametryzowane)
activate DWH
DWH --> SqlEngine : wynik zapytania
deactivate DWH
SqlEngine --> Orch : sql_value
deactivate SqlEngine

Orch -> Valid : evaluate(report_value, sql_value,\ntolerance, business_rules)
activate Valid
Valid --> Orch : TestResult(status, deviation)
deactivate Valid

Orch -> Results : save(TestExecution, TestResult[])
activate Results
Results --> Orch : execution_id
deactivate Results

Orch --> API : execution_id, overall_status
deactivate Orch

API --> GUI : 202 Accepted\n{execution_id, status: "RUNNING"|"DONE"}
deactivate API

GUI --> User : Potwierdzenie uruchomienia

Orch --> AI : (async) analyze(execution_id)\n[tylko gdy status != PASS]
activate AI
AI -> Results : pobierz historię i kontekst\n(READ ONLY)
AI --> Results : AIInsight (evidence-based)
```

```
Results -> Notify : trigger (status != PASS)
activate Notify
Notify --> Notify : deduplikacja + wysyłka
deactivate Notify
deactivate AI

...po chwili...

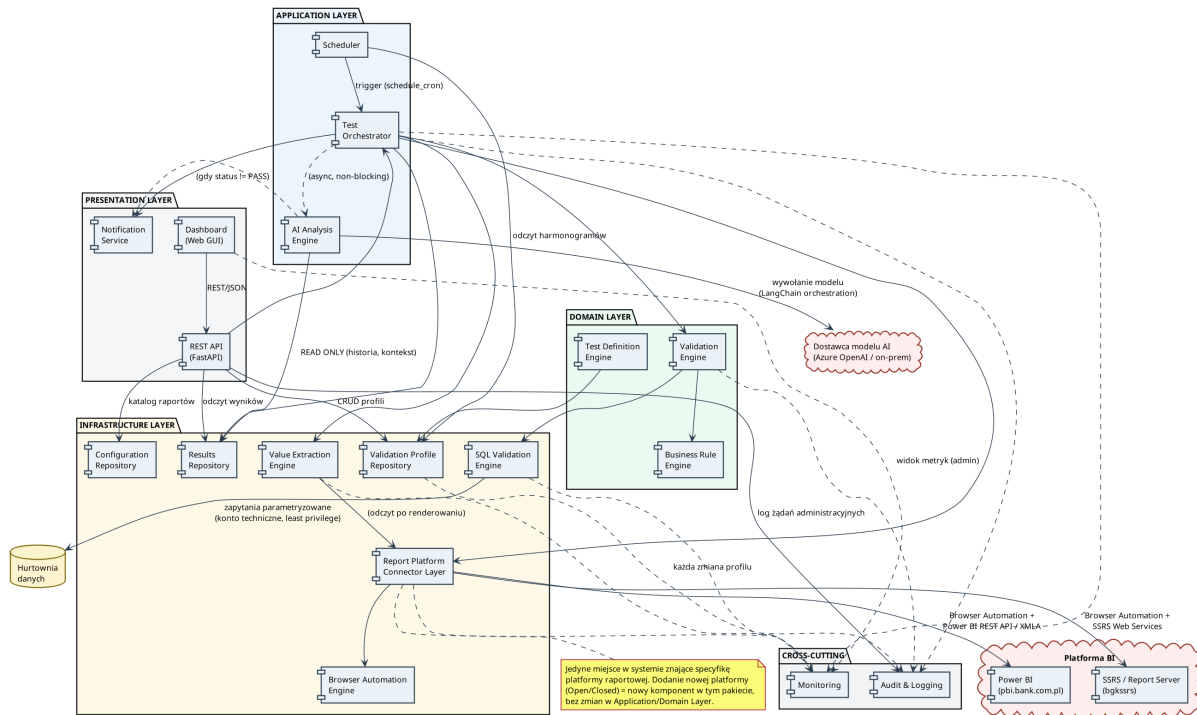
GUI -> API : GET /test-executions/{id}/status\n(polling)
activate API
API --> GUI : status: DONE, overall_status,\nai_insight_available
deactivate API
GUI --> User : Wyświetl wynik + link\ndo interpretacji AI

@enduml
```

13.5 Diagram komponentów (Component Diagram)

Opis. Diagram przedstawia wszystkie 17 głównych modułów systemu zdefiniowanych w 03_Architecture.md, sekcja 4, pogrupowanych wg warstw Clean Architecture (Presentation / Application / Domain / Infrastructure), wraz z ich zależnościami oraz systemami zewnętrznymi (Platforma BI — Power BI i SSRS, hurtownia danych, opcjonalny dostawca modelu AI).

Uzasadnienie projektowe. Grupowanie komponentów w ramki odpowiadające warstwom architektonicznym (a nie płaski graf) jest celowe — pozwala natychmiast zweryfikować wizualnie zgodność z regułą zależności Clean Architecture: żadna strzałka nie powinna prowadzić z warstwy wewnętrznej (Domain) do warstwy zewnętrznej (Infrastructure/Presentation). Moduły cross-cutting (Audit & Logging, Monitoring) są celowo umieszczone poza głównym stosem warstw i połączone z każdym modulem, aby podkreślić ich przekrojowy charakter (03_Architecture.md, sekcja 5) — nie są częścią żadnej pojedynczej warstwy sensu stricto. Systemy zewnętrzne (Power BI, SSRS, Hurtownia danych, Azure OpenAI) są pokazane jako komponenty poza granicą systemu ARGUS, połączone wyłącznie z odpowiednimi adapterami infrastruktury — żaden komponent domenowy ani aplikacyjny nie ma bezpośredniej zależności od świata zewnętrznego, zgodnie z Dependency Inversion Principle.



Rysunek 13.5: Diagram komponentów (Component Diagram) — plik źródłowy `uml/ComponentDiagram.puml`.

Pełny kod źródłowy PlantUML (uml/ComponentDiagram.puml):

```

' =====
' ARGUS – Diagram komponentów (Component Diagram)
' =====
' OPIS:
' Diagram przedstawia wszystkie 17 głównych modułów systemu zdefiniowanych
' w 03_Architecture.md, sekcja 4, pogrupowanych wg warstw Clean Architecture
' (Presentation / Application / Domain / Infrastructure), wraz z ich
' zależnościami oraz systemami zewnętrznymi (Platforma BI – Power BI i SSRS,
' hurtownia danych, opcjonalny dostawca modelu AI).
'
' UZASADNIENIE PROJEKTOWE:
' Grupowanie komponentów w ramki odpowiadające warstwom architektonicznym
' (a nie płaski graf) jest celowe – pozwala natychmiast zweryfikować wizualnie
' zgodność z regułą zależności Clean Architecture: żadna strzałka nie powinna
' prowadzić z warstwy wewnętrznej (Domain) do warstwy zewnętrznej
' (Infrastructure/Presentation). Moduły cross-cutting (Audit & Logging,
' Monitoring) są celowo umieszczone poza głównym stosem warstw i połączone
' z każdym modułem, aby podkreślić ich przekrojowy charakter (03_Architecture.md,
' sekcja 5) – nie są częścią żadnej pojedynczej warstwy sensu stricto.
' Systemy zewnętrzne (Power BI, SSRS, Hurtownia danych, Azure OpenAI) są
' pokazane jako komponenty poza granicą systemu ARGUS, połączone wyłącznie
' z odpowiednimi adapterami infrastruktury – żaden komponent domenowy ani
' aplikacyjny nie ma bezpośredniej zależności od świata zewnętrznego,
' zgodnie z Dependency Inversion Principle.
' =====

@startuml ComponentDiagram

skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 11
skinparam component {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
}
skinparam database {
    BackgroundColor #FCF3CF
    BorderColor #7D6608
}
skinparam cloud {
    BackgroundColor #FDEDEC
    BorderColor #943126
}
skinparam ArrowColor #2C3E50
skinparam packageStyle rectangle

package "PRESENTATION LAYER" #F4F6F7 {
    [Dashboard\n(Web GUI)] as Dashboard
    [REST API\n(FastAPI)] as API
    [Notification\nService] as Notify
}

package "APPLICATION LAYER" #EBF5FB {
    [Scheduler] as Scheduler
    [Test\nOrchestrator] as Orchestrator
    [AI Analysis\nEngine] as AIEngine
}

package "DOMAIN LAYER" #EAFAF1 {

```

```

[Validation\nEngine] as ValidEngine
[Business Rule\nEngine] as RuleEngine
[Test Definition\nEngine] as TestDefEngine
}

package "INFRASTRUCTURE LAYER" #FEF9E7 {
  [Report Platform\nConnector Layer] as ConnectorLayer
  [Browser Automation\nEngine] as BrowserEngine
  [Value Extraction\nEngine] as ExtractEngine
  [SQL Validation\nEngine] as SqlEngine
  [Configuration\nRepository] as ConfigRepo
  [Validation Profile\nRepository] as ProfileRepo
  [Results\nRepository] as ResultsRepo
}

package "CROSS-CUTTING" #F2F3F4 {
  [Audit & Logging] as Audit
  [Monitoring] as Monitoring
}

cloud "Platforma BI" {
  [Power BI\n(pbi.bank.com.pl)] as PBI
  [SSRS / Report Server\n(bgkssrs)] as SSRS
}

database "Hurtownia\ndanych" as DWH
cloud "Dostawca modelu AI\n(Azure OpenAI / on-prem)" as AIProvider

' --- Relacje główne ---
Dashboard --> API : REST/JSON
API --> Orchestrator
API --> ProfileRepo : CRUD profili
API --> ConfigRepo : katalog raportów
API --> ResultsRepo : odczyt wyników
API --> Audit : log żądań administracyjnych

Scheduler --> Orchestrator : trigger (schedule_cron)
Scheduler --> ProfileRepo : odczyt harmonogramów

Orchestrator --> ConnectorLayer
Orchestrator --> ExtractEngine
Orchestrator --> ValidEngine
Orchestrator --> ResultsRepo
Orchestrator ..> AIEngine : (async, non-blocking)
Orchestrator --> Notify : (gdzie status != PASS)

TestDefEngine --> ProfileRepo

ConnectorLayer --> BrowserEngine
ConnectorLayer --> PBI : Browser Automation +\nPower BI REST API / XMLA
ConnectorLayer --> SSRS : Browser Automation +\nSSRS Web Services

ExtractEngine --> ConnectorLayer : (odczyt po renderowaniu)

ValidEngine --> RuleEngine
ValidEngine --> SqlEngine
SqlEngine --> DWH : zapytania parametryzowane\n(konto techniczne, least privilege)

AIEngine --> ResultsRepo : READ ONLY (historia, kontekst)
AIEngine --> AIProvider : wywołanie modelu\n(LangChain orchestration)
AIEngine ..> Notify

```

```
Dashboard ..> Monitoring : widok metryk (admin)

' --- Cross-cutting: powiązania z każdym modułem operacyjnym ---
Orchestrator ..> Audit
Orchestrator ..> Monitoring
ConnectorLayer ..> Monitoring
ExtractEngine ..> Monitoring
SqlEngine ..> Monitoring
ValidEngine ..> Audit
ProfileRepo ..> Audit : każda zmiana profilu

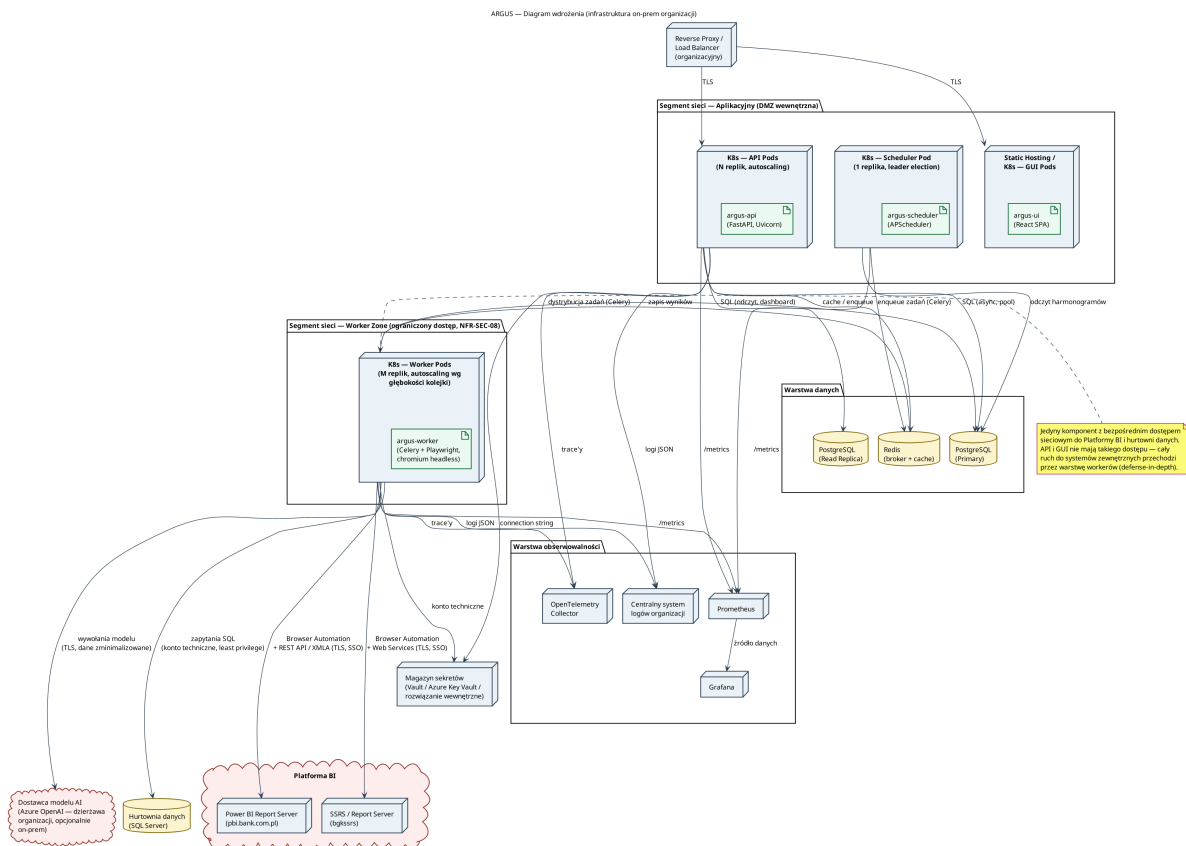
note bottom of ConnectorLayer
    Jedyne miejsce w systemie znające specyfikę
    platformy raportowej. Dodanie nowej platformy
    (Open/Closed) = nowy komponent w tym pakiecie,
    bez zmian w Application/Domain Layer.
end note

@enduml
```

13.6 Diagram wdrożenia (Deployment Diagram)

Opis. Diagram przedstawia fizyczną/logiczną topologię infrastruktury opisaną w 10_Deployment.md, sekcja 2 — węzły Kubernetes (API, GUI, Scheduler, Celery Workers), warstwę danych (PostgreSQL, Redis), warstwę obserwowalności (Prometheus, Grafana, OpenTelemetry Collector, centralny system logów), magazyn sekretów oraz systemy zewnętrzne (Platforma BI, hurtownia danych, opcjonalny dostawca AI), wraz z granicami segmentacji sieciowej. Diagram jest ułożony w siedmiu poziomych pasmach (od góry: Load Balancer → Segment Aplikacyjny → Segment Worker Zone → Warstwa danych → Warstwa obserwowalności → Magazyn sekretów → Systemy zewnętrzne), aby zachować czytelność na stronie A4/landscape dokumentu PDF.

Uzasadnienie projektowe. Węzeł “Segment sieci — Worker Zone” jest celowo wyodrębniony jako osobna granica (boundary) obejmująca wyłącznie Celery Workers — odzwierciedla wymóg NFR-SEC-08 (segmentacja sieciowa, dostęp workerów ograniczony wyłącznie do Platformy BI i hurtowni danych). API i GUI są w osobnym segmencie, bez bezpośredniego dostępu do Platformy BI — cały ruch do zewnętrznych systemów raportowych przechodzi wyłącznie przez warstwę workerów, co minimalizuje powierzchnię ataku zgodnie z zasadą defense-in-depth. Magazyn sekretów jest połączony wyłącznie z węzłami, które faktycznie potrzebują poświadczeń w runtime (Workers — konto techniczne; API — connection string do bazy), nigdy z GUI (statyczne pliki, brak dostępu do sekretów backendu) — obrazuje zasadę least privilege również na poziomie infrastruktury, nie tylko kont użytkowników. Jawne wymuszenie pionowego układu pasm (zamiast naturalnego, szerokiego rozkładu grafu zależności) jest świadomą decyzją redakcyjną — diagram wdrożenia ma zwykle najwięcej równoległych węzłów spośród wszystkich diagramów UML w tym projekcie, więc bez wymuszenia pionowej hierarchii stałby się nieczytelny na wydrukowanej stronie.



Rysunek 13.6: Diagram wdrożenia (Deployment Diagram) — plik źródłowy `uml/DeploymentDiagram.puml`.

Pełny kod źródłowy PlantUML (uml/DeploymentDiagram.puml):

```
' =====
' ARGUS – Diagram wdrożenia (Deployment Diagram)
' =====
' OPIS:
' Diagram przedstawia fizyczną/logiczną topologię infrastruktury opisaną w
' 10_Deployment.md, sekcja 2 – węzły Kubernetes (API, GUI, Scheduler, Celery
' Workers), warstwę danych (PostgreSQL, Redis), warstwę obserwowalności
' (Prometheus, Grafana, OpenTelemetry Collector, centralny system logów),
' magazyn sekretów oraz systemy zewnętrzne (Platforma BI, hurtownia danych,
' opcjonalny dostawca AI), wraz z granicami segmentacji sieciowej. Diagram
' jest ułożony w siedmiu poziomych pasmach (od góry: Load Balancer → Segment
' Aplikacyjny → Segment Worker Zone → Warstwa danych → Warstwa
' obserwowalności → Magazyn sekretów → Systemy zewnętrzne), aby zachować
' czytelność na stronie A4/landscape dokumentu PDF.
'
' UZASADNIENIE PROJEKTOWE:
' Węzeł "Segment sieci – Worker Zone" jest celowo wyodrębniony jako osobna
' granica (boundary) obejmująca wyłącznie Celery Workers – odzwierciedla
' wymóg NFR-SEC-08 (segmentacja sieciowa, dostęp workerów ograniczony
' wyłącznie do Platformy BI i hurtowni danych). API i GUI są w osobnym
' segmencie, bez bezpośredniego dostępu do Platformy BI – cały ruch do
' zewnętrznych systemów raportowych przechodzi wyłącznie przez warstwę
' workerów, co minimalizuje powierzchnię ataku zgodnie z zasadą
' defense-in-depth. Magazyn sekretów jest połączony wyłącznie z węzłami,
' które faktycznie potrzebują poświadczeń w runtime (Workers – konto
' techniczne; API – connection string do bazy), nigdy z GUI (statyczne
' pliki, brak dostępu do sekretów backendu) – obrazuje zasadę least
' privilege również na poziomie infrastruktury, nie tylko kont użytkowników.
' Jawne wymuszenie pionowego układu pasm (zamiast naturalnego, szerokiego
' rozkładu grafu zależności) jest świadomą decyzją redakcyjną – diagram
' wdrożenia ma zwykle najwięcej równoległych węzłów spośród wszystkich
' diagramów UML w tym projekcie, więc bez wymuszenia pionowej hierarchii
' stałby się nieczytelny na wydrukowanej stronie.
' =====
```

```
@startuml DeploymentDiagram
```

```
skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 12
skinparam nodesep 18
skinparam ranksep 35
skinparam node {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
}
skinparam database {
    BackgroundColor #FCF3CF
    BorderColor #7D6608
}
skinparam cloud {
    BackgroundColor #FDEDEC
    BorderColor #943126
}
skinparam artifact {
    BackgroundColor #EAF2F8
    BorderColor #196F3D
}
skinparam ArrowColor #2C3E50
```

```

title ARGUS – Diagram wdrożenia (infrastruktura on-prem organizacji)

' ----- Pasma 1: wejście -----
node "Reverse Proxy /\nLoad Balancer\n(organizacyjny)" as LB

' ----- Pasma 2: segment aplikacyjny -----
package "Segment sieci – Aplikacyjny (DMZ wewnętrzna)" as NetApp {
  together {
    node "K8s – API Pods\n(N replik, autoscaling)" as APIPods {
      artifact "argus-api\n(FastAPI, Uvicorn)" as APIArtifact
    }
    node "Static Hosting /\nK8s – GUI Pods" as GUIPods {
      artifact "argus-ui\n(React SPA)" as GUIArtifact
    }
    node "K8s – Scheduler Pod\n(1 replika, leader election)" as SchedulerPod {
      artifact "argus-scheduler\n(APScheduler)" as SchedulerArtifact
    }
  }
}

' ----- Pasma 3: worker zone -----
package "Segment sieci – Worker Zone (ograniczony dostęp, NFR-SEC-08)" as NetWorker {
  node "K8s – Worker Pods\n(M replik, autoscaling wg\ngłębokości kolejki)" as WorkerPods {
    artifact "argus-worker\n(Celery + Playwright,\nchromium headless)" as WorkerArtifact
  }
}

' ----- Pasma 4: dane -----
package "Warstwa danych" as DataLayer {
  together {
    database "PostgreSQL\n(Primary)" as PgPrimary
    database "PostgreSQL\n(Read Replica)" as PgReplica
    database "Redis\n(broker + cache)" as Redis
  }
}

' ----- Pasma 5: obserwowalność -----
package "Warstwa obserwowalności" as ObsLayer {
  together {
    node "Prometheus" as Prometheus
    node "Grafana" as Grafana
    node "OpenTelemetry\nCollector" as OtelCollector
    node "Centralny system\nlogów organizacji" as CentralLogs
  }
}

' ----- Pasma 6: sekrety -----
node "Magazyn sekretów\n(Vault / Azure Key Vault /\nrozwiązanie wewnętrzne)" as Vault

' ----- Pasma 7: systemy zewnętrzne -----
together {
  cloud "Platforma BI" as PlatformaBI {
    node "Power BI Report Server\n(pbi.bank.com.pl)" as PBI
    node "SSRS / Report Server\n(bgkssrs)" as SSRS
  }
  database "Hurtownia danych\n(SQL Server)" as DWH
  cloud "Dostawca modelu AI\n(Azure OpenAI – dzierżawa\norganizacji, opcjonalnie\nnon-prem)" as AIProvider
}

' ----- Wymuszenie ścisłego układu pasm od góry do dołu -----
LB -[hidden]d- NetApp

```

```

NetApp -[hidden]d- NetWorker
NetWorker -[hidden]d- DataLayer
DataLayer -[hidden]d- ObsLayer
ObsLayer -[hidden]d- Vault
Vault -[hidden]d- PlatformaBI

' ----- Połączenia rzeczywiste -----
LB --> APIPods : TLS
LB --> GUIPods : TLS

APIPods --> PgPrimary : SQL (async, pool)
APIPods --> PgReplica : SQL (odczyt, dashboard)
APIPods --> Redis : cache / enqueue
APIPods --> Vault : connection string

SchedulerPod --> Redis : enqueue zadań (Celery)
SchedulerPod --> PgPrimary : odczyt harmonogramów

Redis --> WorkerPods : dystrybucja zadań (Celery)
WorkerPods --> PgPrimary : zapis wyników
WorkerPods --> Vault : konto techniczne
WorkerPods --> PBI : Browser Automation\n+ REST API / XMLA (TLS, SSO)
WorkerPods --> SSRS : Browser Automation\n+ Web Services (TLS, SSO)
WorkerPods --> DWH : zapytania SQL\n(konto techniczne, least privilege)
WorkerPods --> AIProvider : wywołania modelu\n(TLS, dane zminimalizowane)

APIPods --> Prometheus : /metrics
WorkerPods --> Prometheus : /metrics
SchedulerPod --> Prometheus : /metrics
Prometheus --> Grafana : źródło danych

APIPods --> OtelCollector : trace'y
WorkerPods --> OtelCollector : trace'y

APIPods --> CentralLogs : logi JSON
WorkerPods --> CentralLogs : logi JSON

note right of WorkerPods
    Jedyne komponent z bezpośrednim dostępem
    sieciowym do Platformy BI i hurtowni danych.
    API i GUI nie mają takiego dostępu – cały
    ruch do systemów zewnętrznych przechodzi
    przez warstwę workerów (defense-in-depth).
end note

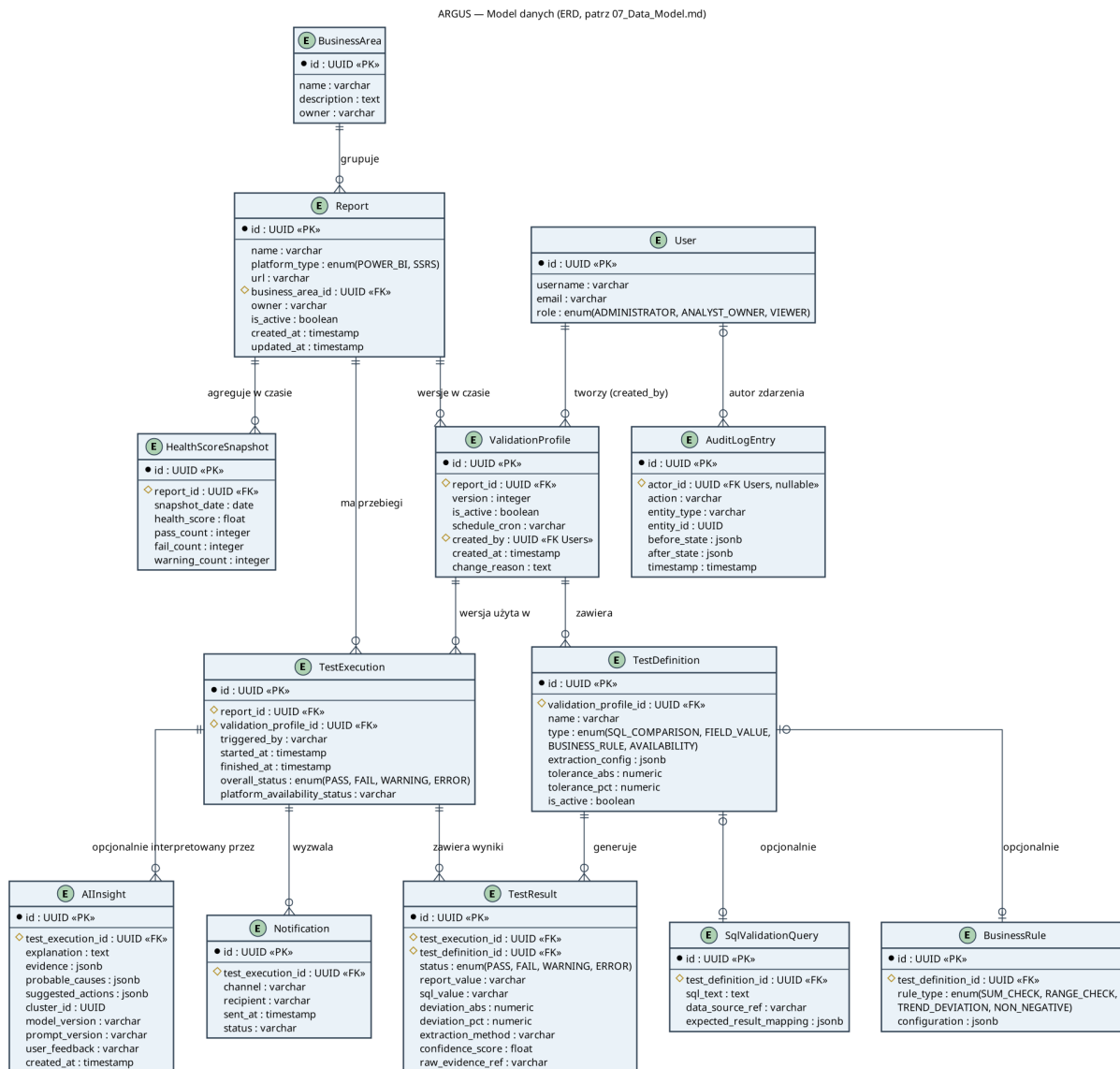
@enduml

```

13.7 Diagram encji (Entity Relationship Diagram — ERD)

Opis. Diagram przedstawia logiczny model danych systemu opisany w 07_Data_Model.md — 13 encji wraz z atrybutami kluczowymi i kardynalnościami relacji, zaimplementowany docelowo w PostgreSQL (SQLAlchemy + Alembic, patrz 09_Python_Architecture.md).

Uzasadnienie projektowe. ValidationProfile jest modelowany w relacji 1:N względem Report (a nie 1:1) celowo — każda zmiana konfiguracji testowej tworzy nową wersję profilu zamiast nadpisywać poprzednią, co jest fundamentem wymogu audytowalności (NFR-C-01, FR-016). Analogicznie TestExecution przechowuje FK zarówno do Report, jak i do konkretnej wersji ValidationProfile użytej w danym przebiegu — pozwala to odtworzyć post factum dokładnie, jaka konfiguracja testowa obowiązywała w momencie wykonania, nawet jeśli profil został później zmieniony (kluczowe dla audytu compliance w instytucji finansowej). HealthScoreSnapshot jest celowo zdenormalizowaną tabelą agregującą (a nie widokiem liczonym on-the-fly z TestResult) — decyzja podyktowana wymogiem wydajnościowym NFR-P-05 (ładowanie Dashboardu ≤2s przy 1000 raportów). AIInsight ma FK wyłącznie do TestExecution (nie do TestResult ani nie ma prawa zapisu do statusu) — odzwierciedla na poziomie modelu danych zasadę nadrzędną z 06_AI_Module.md: AI nie ma technicznej możliwości modyfikacji werdyktu.



Rysunek 13.7: Diagram encji (Entity Relationship Diagram — ERD) — plik źródłowy `uml/ERD.puml`.

Pełny kod źródłowy PlantUML (uml/ERD.puml):

```

' =====
' ARGUS – Diagram encji (Entity Relationship Diagram – ERD)
' =====
' OPIS:
' Diagram przedstawia logiczny model danych systemu opisany w 07_Data_Model.md
' – 13 encji wraz z atrybutami kluczowymi i kardynalnościami relacji,
' zaimplementowany docelowo w PostgreSQL (SQLAlchemy + Alembic, patrz
' 09_Python_Architecture.md).
'
' UZASADNIENIE PROJEKTOWE:
' ValidationProfile jest modelowany w relacji 1:N względem Report (a nie 1:1)
' celowo – każda zmiana konfiguracji testowej tworzy nową wersję profilu
' zamiast nadpisywać poprzednią, co jest fundamentem wymogu audytowalności
' (NFR-C-01, FR-016). Analogicznie TestExecution przechowuje FK zarówno do
' Report, jak i do konkretnej wersji ValidationProfile użytej w danym
' przebiegu – pozwala to odtworzyć post factum dokładnie, jaka konfiguracja
' testowa obowiązywała w momencie wykonania, nawet jeśli profil został
' później zmieniony (kluczowe dla audytu compliance w instytucji finansowej).
' HealthScoreSnapshot jest celowo zdenormalizowaną tabelą agregującą
' (a nie widokiem liczonym on-the-fly z TestResult) – decyzja podyktowana
' wymogiem wydajnościowym NFR-P-05 (Ładowanie Dashboardu ≤2s przy 1000
' raportów). AIInsight ma FK wyłącznie do TestExecution (nie do TestResult
' ani nie ma prawa zapisu do statusu) – odzwierciedla na poziomie modelu
' danych zasadę nadrzędną z 06_AI_Module.md: AI nie ma technicznej
' możliwości modyfikacji werdyktu.
' =====

@startuml ERD

skinparam backgroundColor white
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 11
skinparam linetype ortho
skinparam class {
    BackgroundColor #EAF2F8
    BorderColor #2C3E50
    ArrowColor #2C3E50
}

title ARGUS – Model danych (ERD, patrz 07_Data_Model.md)

entity "BusinessArea" as BusinessArea {
    * id : UUID <<PK>>
    --
    name : varchar
    description : text
    owner : varchar
}

entity "Report" as Report {
    * id : UUID <<PK>>
    --
    name : varchar
    platform_type : enum(POWER_BI, SSRS)
    url : varchar
    # business_area_id : UUID <<FK>>
    owner : varchar
    is_active : boolean
    created_at : timestamp

```

```

    updated_at : timestamp
}

entity "ValidationProfile" as ValidationProfile {
    * id : UUID <<PK>>
    --
    # report_id : UUID <<FK>>
    version : integer
    is_active : boolean
    schedule_cron : varchar
    # created_by : UUID <<FK Users>>
    created_at : timestamp
    change_reason : text
}

entity "TestDefinition" as TestDefinition {
    * id : UUID <<PK>>
    --
    # validation_profile_id : UUID <<FK>>
    name : varchar
    type : enum(SQL_COMPARISON, FIELD_VALUE, \nBUSINESS_RULE, AVAILABILITY)
    extraction_config : jsonb
    tolerance_abs : numeric
    tolerance_pct : numeric
    is_active : boolean
}

entity "SqlValidationQuery" as SqlValidationQuery {
    * id : UUID <<PK>>
    --
    # test_definition_id : UUID <<FK>>
    sql_text : text
    data_source_ref : varchar
    expected_result_mapping : jsonb
}

entity "BusinessRule" as BusinessRule {
    * id : UUID <<PK>>
    --
    # test_definition_id : UUID <<FK>>
    rule_type : enum(SUM_CHECK, RANGE_CHECK, \nTREND_DEVIATION, NON_NEGATIVE)
    configuration : jsonb
}

entity "TestExecution" as TestExecution {
    * id : UUID <<PK>>
    --
    # report_id : UUID <<FK>>
    # validation_profile_id : UUID <<FK>>
    triggered_by : varchar
    started_at : timestamp
    finished_at : timestamp
    overall_status : enum(PASS, FAIL, WARNING, ERROR)
    platform_availability_status : varchar
}

entity "TestResult" as TestResult {
    * id : UUID <<PK>>
    --
    # test_execution_id : UUID <<FK>>
    # test_definition_id : UUID <<FK>>
    status : enum(PASS, FAIL, WARNING, ERROR)
}

```

```

    report_value : varchar
    sql_value : varchar
    deviation_abs : numeric
    deviation_pct : numeric
    extraction_method : varchar
    confidence_score : float
    raw_evidence_ref : varchar
}

entity "HealthScoreSnapshot" as HealthScoreSnapshot {
    * id : UUID <<PK>>
    --
    # report_id : UUID <<FK>>
    snapshot_date : date
    health_score : float
    pass_count : integer
    fail_count : integer
    warning_count : integer
}

entity "AIInsight" as AIInsight {
    * id : UUID <<PK>>
    --
    # test_execution_id : UUID <<FK>>
    explanation : text
    evidence : jsonb
    probable_causes : jsonb
    suggested_actions : jsonb
    cluster_id : UUID
    model_version : varchar
    prompt_version : varchar
    user_feedback : varchar
    created_at : timestamp
}

entity "Notification" as Notification {
    * id : UUID <<PK>>
    --
    # test_execution_id : UUID <<FK>>
    channel : varchar
    recipient : varchar
    sent_at : timestamp
    status : varchar
}

entity "User" as User {
    * id : UUID <<PK>>
    --
    username : varchar
    email : varchar
    role : enum(ADMINISTRATOR, ANALYST_OWNER, VIEWER)
}

entity "AuditLogEntry" as AuditLogEntry {
    * id : UUID <<PK>>
    --
    # actor_id : UUID <<FK Users, nullable>>
    action : varchar
    entity_type : varchar
    entity_id : UUID
    before_state : jsonb
    after_state : jsonb
}

```

```
    timestamp : timestamp
}

BusinessArea ||--o{ Report : "grupuje"
Report ||--o{ ValidationProfile : "wersje w czasie"
ValidationProfile ||--o{ TestDefinition : "zawiera"
TestDefinition |o--o| SqlValidationQuery : "opcjonalnie"
TestDefinition |o--o| BusinessRule : "opcjonalnie"
Report ||--o{ TestExecution : "ma przebiegi"
ValidationProfile ||--o{ TestExecution : "wersja użyta w"
TestExecution ||--o{ TestResult : "zawiera wyniki"
TestDefinition ||--o{ TestResult : "generuje"
TestExecution ||--o{ AIInsight : "opcjonalnie interpretowany przez"
TestExecution ||--o{ Notification : "wyzwała"
Report ||--o{ HealthScoreSnapshot : "agreguje w czasie"
User ||--o{ ValidationProfile : "tworzy (created_by)"
User |o--o{ AuditLogEntry : "autor zdarzenia"

@enduml
```